04820KA001 Mono Slitter 903_V12_1114

Author: Michou
Created: 11/14/2026 11:55:38 AM
Last Modified: 11/14/2026 11:55:38 AM
Comment: Modified to Connect 2nd Diameter Sens

Sysmac Studio Module Version: 1.60.0.64010

1.PLC

1-1.EtherCAT Network Configuration and Unit Operation Settings

Node Address	Network configuration
	Master Master
1	E001 IL EC BK-PAC Rev:0x00000001

Master

Connect from	---
Model name	Master
Product name	Master
Number of Slaves	1
PDO Communications Cycle 1	4000us
PDO Communications Cycle 2	---
Transmission Delay Time	PDO Communications Cycle 1 Estimated result from network configuration setting on Sysmac Studio: 18µs
Reference Clock	Exist
Total Cable Length	1000m
Fail-soft Operation Setting	Fail-soft operation
Wait Time for Slave Startup	30s
PDO communications timeout detection count	2times
Revision Check Method	Setting <= Actual device
Serial Number Check Method	No check

E001

Connect from	Master (port PortB)
Model name	IL EC BK-PAC
Product name	IL EC BK, Inline EtherCAT Fieldbus coupler
Revision	0x00000001
PDO Communications Cycle	PDO Communications Cycle 1 (4000 us)
Node Address	1
Enable/Disable Settings	Enabled
Serial Number	0x00000000
PDO Map Settings	---
Enable Distributed Clock	---
Shift Time Setting	---
Reference Clock	Not exist
Setting Parameters	0xF801:00 EtherCAT Reset Behavior/EtherCAT Reset Behavior 0 0xF802:00 Validate ModuleIdent List/Validate ModuleIdent List 1 0xF803:00 Localbus Auto Restart/Localbus Auto Restart 1 0xF804:00 Startup with remaining System/Startup with remaining System 1
Backup Parameter Settings	

Position	Slot	Module
Node1 : IL EC BK-PAC (E001)		
0	Slot for Inline terminals	IB IL 24 DI 8/HD-ECO(M1)
1	Slot for Inline terminals	IB IL 24 DO 8/HD-ECO(M2)
2	Slot for Inline terminals	Empty slot
3	Slot for Inline terminals	Empty slot
4	Slot for Inline terminals	Empty slot
5	Slot for Inline terminals	Empty slot
6	Slot for Inline terminals	Empty slot
7	Slot for Inline terminals	Empty slot
8	Slot for Inline terminals	Empty slot
9	Slot for Inline terminals	Empty slot
10	Slot for Inline terminals	Empty slot
11	Slot for Inline terminals	Empty slot
12	Slot for Inline terminals	Empty slot
13	Slot for Inline terminals	Empty slot

14	Slot for Inline terminals	Empty slot
15	Slot for Inline terminals	Empty slot
16	Slot for Inline terminals	Empty slot
17	Slot for Inline terminals	Empty slot
18	Slot for Inline terminals	Empty slot
19	Slot for Inline terminals	Empty slot
20	Slot for Inline terminals	Empty slot
21	Slot for Inline terminals	Empty slot
22	Slot for Inline terminals	Empty slot
23	Slot for Inline terminals	Empty slot
24	Slot for Inline terminals	Empty slot
25	Slot for Inline terminals	Empty slot
26	Slot for Inline terminals	Empty slot
27	Slot for Inline terminals	Empty slot
28	Slot for Inline terminals	Empty slot
29	Slot for Inline terminals	Empty slot
30	Slot for Inline terminals	Empty slot
31	Slot for Inline terminals	Empty slot
32	Slot for Inline terminals	Empty slot
33	Slot for Inline terminals	Empty slot
34	Slot for Inline terminals	Empty slot
35	Slot for Inline terminals	Empty slot
36	Slot for Inline terminals	Empty slot
37	Slot for Inline terminals	Empty slot
38	Slot for Inline terminals	Empty slot
39	Slot for Inline terminals	Empty slot
40	Slot for Inline terminals	Empty slot
41	Slot for Inline terminals	Empty slot
42	Slot for Inline terminals	Empty slot
43	Slot for Inline terminals	Empty slot
44	Slot for Inline terminals	Empty slot
45	Slot for Inline terminals	Empty slot
46	Slot for Inline terminals	Empty slot
47	Slot for Inline terminals	Empty slot
48	Slot for Inline terminals	Empty slot
49	Slot for Inline terminals	Empty slot
50	Slot for Inline terminals	Empty slot
51	Slot for Inline terminals	Empty slot
52	Slot for Inline terminals	Empty slot
53	Slot for Inline terminals	Empty slot
54	Slot for Inline terminals	Empty slot
55	Slot for Inline terminals	Empty slot
56	Slot for Inline terminals	Empty slot
57	Slot for Inline terminals	Empty slot
58	Slot for Inline terminals	Empty slot
59	Slot for Inline terminals	Empty slot
60	Slot for Inline terminals	Empty slot
61	Slot for Inline terminals	Empty slot
62	Slot for Inline terminals	Empty slot

E001

Model	IL EC BK-PAC
Product name	IL EC BK, Inline EtherCAT Fieldbus coupler
Revision	0x00000001
Number of modules	2
PDO Map Settings	0x10F3:04, Coupler Input-PD/New Messages Available
	0x18FF:01, Coupler Input-PD/Remaining system active
	0x18FF:09, Coupler Input-PD/InputUpdateCounter
	0xF100:01, Coupler Input-PD/State
	0xF100:02, Coupler Input-PD/Parameter
	0xF100:03, Coupler Input-PD/Ext. Parameter
Setting Parameters	0xF801:00 EtherCAT Reset Behavior/EtherCAT Reset Behavior 0
	0xF802:00 Validate ModuleIdent List/Validate ModuleIdent List 1
	0xF803:00 Localbus Auto Restart/Localbus Auto Restart 1
	0xF804:00 Startup with remaining System/Startup with remaining System 1
Module config send method	Do not send

M1

Model	IB IL 24 DI 8/HD-ECO
Product name	IB IL 24 DI 8/HD-ECO, Inline ECO terminal with 8 digital inputs
Connected position	0
PDO Map Settings	0x6000:01, IB IL 24 DI 8/HD-ECO Input-PD/IN01 (Pin 1.1)
	0x6000:02, IB IL 24 DI 8/HD-ECO Input-PD/IN02 (Pin 2.1)
	0x6000:03, IB IL 24 DI 8/HD-ECO Input-PD/IN03 (Pin 1.2)
	0x6000:04, IB IL 24 DI 8/HD-ECO Input-PD/IN04 (Pin 2.2)
	0x6000:05, IB IL 24 DI 8/HD-ECO Input-PD/IN05 (Pin 1.3)
	0x6000:06, IB IL 24 DI 8/HD-ECO Input-PD/IN06 (Pin 2.3)
	0x6000:07, IB IL 24 DI 8/HD-ECO Input-PD/IN07 (Pin 1.4)
	0x6000:08, IB IL 24 DI 8/HD-ECO Input-PD/IN08 (Pin 2.4)

M2

Model	IB IL 24 DO 8/HD-ECO
Product name	IB IL 24 DO 8/HD-ECO, Inline ECO terminal with 8 digital outputs
Connected position	1
PDO Map Settings	0x7000:01, IB IL 24 DO 8/HD-ECO Output-PD/OUT01 (Pin 1.1)
	0x7000:02, IB IL 24 DO 8/HD-ECO Output-PD/OUT02 (Pin 2.1)
	0x7000:03, IB IL 24 DO 8/HD-ECO Output-PD/OUT03 (Pin 1.2)
	0x7000:04, IB IL 24 DO 8/HD-ECO Output-PD/OUT04 (Pin 2.2)
	0x7000:05, IB IL 24 DO 8/HD-ECO Output-PD/OUT05 (Pin 1.3)

	0x7000:06, IB IL 24 DO 8/HD-ECO Output-PD/OUT06 (Pin 2.3)
	0x7000:07, IB IL 24 DO 8/HD-ECO Output-PD/OUT07 (Pin 1.4)
	0x7000:08, IB IL 24 DO 8/HD-ECO Output-PD/OUT08 (Pin 2.4)

1-2.CPU/Expansion Racks

1-2-1.NX Bus Configuration and Unit Operation Settings

Width:166[mm]

NX Unit Number	1	2	3
NX1P2-9024DT1 NXBus	NX-PF0730 N3	NX-AD2603 N1	NX-EC0122 N2

Rack Number	NX Unit Number	Model name	Product name	Device name
0	0	NX1P2-9024DT1	NX series controller	NXBus
	1	NX-PF0730	Additional I/O Power Supply	N3
	2	NX-AD2603	Analog Input Unit	N1
	3	NX-EC0122	Incremental Encoder Input Unit	N2

NXBus

Model name	NX1P2-9024DT1
Product name	NX series controller
Unit version	1.42
NX Unit Number	0
Serial Number	---
Supply Power/Available Power	3.50 / 10.00W
Width	124mm
Number of mounted Units	3
NX Unit Connection Time	3sec
Serial Number Check Method	No check
Fail-soft operation setting	Fail-soft operation

N3

Model name	NX-PF0730
Product name	Additional I/O Power Supply
Unit version	1.0
NX Unit Number	1
NX Unit Mounting Setting	Enabled
Serial Number	0x00000000
Power consumption	0.85W
Width	12mm

N1

Model name	NX-AD2603
Product name	Analog Input Unit
Unit version	1.0

NX Unit Number		2
NX Unit Mounting Setting		Enabled
Serial Number		0x00000000
Power consumption		1.35W
Width		12mm
I/O allocation settings		0x6000:01 16[bit] Input Data Set 1/Ch1 Analog Input Value
		0x6000:02 16[bit] Input Data Set 1/Ch2 Analog Input Value
Unit operation settings		
	Event Level Setting/Event 1	Unit Over Range
	Event Level Setting/Level Setting of Event 1	Observation
	Event Level Setting/Event 2	Unit Under Range
	Event Level Setting/Level Setting of Event 2	Observation
	Event Level Setting/Event 3	Ch1 Disconnection Detected
	Event Level Setting/Level Setting of Event 3	Minor Fault
	Event Level Setting/Event 4	Ch2 Disconnection Detected
	Event Level Setting/Level Setting of Event 4	Minor Fault
	Input Enable/Disable Setting/Ch1 Enable/Disable	True
	Input Enable/Disable Setting/Ch2 Enable/Disable	True
	Input Range Setting/Ch1 Range Setting	-10 to +10 V
	Input Range Setting/Ch2 Range Setting	-10 to +10 V
	Input Moving Average Time/Ch1 Input Moving Average Time	32000
	Input Moving Average Time/Ch2 Input Moving Average Time	32000

N2

Model name		NX-EC0122
Product name		Incremental Encoder Input Unit
Unit version		1.2
NX Unit Number		3
NX Unit Mounting Setting		Enabled
Serial Number		0x00000000
Power consumption		1.30W
Width		12mm
I/O allocation settings		0x7008:01 16[bit] Output Data Set 1/Ch1 Pulse Period Measurement Function
		0x7004:01 16[bit] Output Data Set 1/Ch1 Latch Function
		0x7000:01 16[bit] Output Data Set 1/Ch1 Encoder Counter Operation Command
		0x6000:01 8[bit] Input Data Set 1/Ch1 Encoder Counter Status
		0x6001:01 8[bit] Input Data Set 1/Ch1 Reset/External Input Status
		0x6002:01 32[bit] Input Data Set 1/Ch1 Encoder Present Position
		0x6008:01 8[bit] Input Data Set 1/Ch1 Pulse Period Measurement Status
		--- 8[bit] Input Data Set 1/Padding
		0x6004:01 16[bit] Input Data Set 1/Ch1 Latch Status
		0x6005:01 32[bit] Input Data Set 1/Ch1 Latch Input 1 Data
		0x6006:01 32[bit] Input Data Set 1/Ch1 Latch Input 2 Data
		0x6003:01 32[bit] Input Data Set 1/Ch1 Pulse Rate
Unit operation settings		
	Counter Type/Ch1 Counter Type	Ring Counter

Maximum Counter Value/Ch1 Maximum Counter Value	2147483647
Minimum Counter Value/Ch1 Minimum Counter Value	-2147483648
Pulse Input Method/Ch1 Pulse Input Method	Phase Differential Pulse x4
Time Window/Ch1 Time Window	100
Average Processing Times/Ch1 Average Processing Times	50
Edge Detection Method/Ch1 Edge Detection Method	Disable the Function
Encoder Count Direction/Ch1 Encoder Count Direction	Positive Direction of Phase A
External Input 0 Function Selection/Ch1 External Input 0 Function Selection	General Input
External Input 0 Logic Selection/Ch1 External Input 0 Logic Selection	N.O. (Normaly Open)
External Input 1 Function Selection/Ch1 External Input 1 Function Selection	General Input
External Input 1 Logic Selection/Ch1 External Input 1 Logic Selection	N.O. (Normaly Open)
External Input 2 Function Selection/Ch1 External Input 2 Function Selection	General Input
External Input 2 Logic Selection/Ch1 External Input 2 Logic Selection	N.O. (Normaly Open)

1-3.I/O Map

Port	Description	R/W	Data Type	Variable	Variable Comment	Variable Type
EtherCAT Network Configuration						
Node1 IL EC BK-PAC						
Coupler Input-PD_New Messages Available_10F3_04		R	BOOL			
Coupler Input-PD_Remaining system active_18FF_01		R	BOOL			
Coupler Input-PD_InputUpdateCounter_18FF_09		R	ARRAY [0..1] OF BOOL			
Coupler Input-PD_State_F100_01		R	UINT			
Coupler Input-PD_Parameter_F100_02		R	UINT			
Coupler Input-PD_Ext. Parameter_F100_03		R	UINT			
Slot 0 IB IL 24 DI 8/HD-ECO						
IB IL 24 DI 8/HD-ECO Input-_IN01 (Pin 1.1)_6000_01		R	BOOL	I_OR_Position_Sens	OverRider position sens	Global Variables
IB IL 24 DI 8/HD-ECO Input-_IN02 (Pin 2.1)_6000_02		R	BOOL	I_OR_Hoover_Mode	OverRider Hoover Mode	Global Variables
IB IL 24 DI 8/HD-ECO Input-_IN03 (Pin 1.2)_6000_03		R	BOOL	I_OR_Auto_OS	OverRider in automatic	Global Variables
IB IL 24 DI 8/HD-ECO Input-_IN04 (Pin 2.2)_6000_04		R	BOOL	I_OR_Cont_OS	OverRider in continuous	Global Variables
IB IL 24 DI 8/HD-ECO Input-_IN05 (Pin 1.3)_6000_05		R	BOOL	I_T_one_Tension OK	T-one Alarm o (15-16): tension established (replaces I_SlowSens_PH)	Global Variables
IB IL 24 DI 8/HD-ECO Input-_IN06 (Pin 2.3)_6000_06		R	BOOL	I_FanTemp_Ok	Motor (fan) thermal OK	Global Variables
IB IL 24 DI 8/HD-ECO Input-_IN07 (Pin 1.4)_6000_07		R	BOOL	I_WebBreak_PH	Paper break photocell	Global Variables

	IB IL 24 DI 8/HD-ECO Input-_IN08 (Pin 2.4) _6000_08		R	BOOL	I_RsetAlarm	Alarm reset	Global Variables
Slot 1 IB IL 24 DO 8/HD-ECO							
	IB IL 24 DO 8/ HD-ECO Output_OUT01 (Pin 1.1)_7000_01		W	BOOL	O_Alarm_Array	Alarm lamp	Global Variables
	IB IL 24 DO 8/ HD-ECO Output_OUT02 (Pin 2.1)_7000_02		W	BOOL			
	IB IL 24 DO 8/ HD-ECO Output_OUT03 (Pin 1.2)_7000_03		W	BOOL	O_RwUnload	Unloaders enable - opening	Global Variables
	IB IL 24 DO 8/ HD-ECO Output_OUT04 (Pin 2.2) _7000_04		W	BOOL	O_FrontGuard_By Pass	Front guard bypass	Global Variables
	IB IL 24 DO 8/ HD-ECO Output_OUT05 (Pin 1.3)_7000_05		W	BOOL			
	IB IL 24 DO 8/ HD-ECO Output_OUT06 (Pin 2.3) _7000_06		W	BOOL			
	IB IL 24 DO 8/ HD-ECO Output_OUT07 (Pin 1.4)_7000_07		W	BOOL			
	IB IL 24 DO 8/ HD-ECO Output_OUT08 (Pin 2.4) _7000_08		W	BOOL			
CPU/Expansion Racks							
Built-in I/O Built-in I/O Settings							
	Input Bit 00	Input Bit 00	R	BOOL	I_ControlVoltage	In_0 ControlVoltageON	Global Variables
	Input Bit 01	Input Bit 01	R	BOOL	I_Start_PB	In_1 Start button	Global Variables
	Input Bit 02	Input Bit 02	R	BOOL	I_Stop_PB	In_2 Stop button	Global Variables
	Input Bit 03	Input Bit 03	R	BOOL	I_Motor_Thermal	In_3 Motor thermal switch	Global Variables
	Input Bit 04	Input Bit 04	R	BOOL	I_Prot_ByPass	In_4 Guards-bypassed selector switch - Under Panel	Global Variables

	Input Bit 05	Input Bit 05	R	BOOL	I_Hub_Position	In_5 Hub/chuck in extraction position	Global Variables
	Input Bit 06	Input Bit 06	R	BOOL	I_FrontGuard_Closed	In_6 Front guard closed	Global Variables
	Input Bit 07	Input Bit 07	R	BOOL	I_Jog_PB	In_7 Jog Push button	Global Variables
	Input Bit 08	Input Bit 08	R	BOOL	I_VFD_Alarm	In_8 Inverter in alarm	Global Variables
	Input Bit 09	Input Bit 09	R	BOOL	I_FrontGuard_Open	In_9 Front guard open	Global Variables
	Input Bit 10	Input Bit 10	R	BOOL	I_AutoStop_OS	In_10 Stop on metre counter	Global Variables
	Input Bit 11	Input Bit 11	R	BOOL	I_Input_11_Spare_1255	In_11 SPARE (wire 12.55) Under Panel	Global Variables
	Input Bit 12	Input Bit 12	R	BOOL	I_WebBreak_Monitor	In_12 Paper break monitoring ON	Global Variables
	Input Bit 13	Input Bit 13	R	BOOL	I_Side_Door_Closed	In_Side Door is Closed	Global Variables
	Output Bit 00	Output Bit 00	RW	BOOL	O_MotorStart	Q_0 Motor start	Global Variables
	Output Bit 01	Output Bit 01	RW	BOOL	O_Motor_Lamp	Q_1 Motor ON lamp	Global Variables
	Output Bit 02	Output Bit 02	RW	BOOL	O_Brake	Q_2 Brake command	Global Variables
	Output Bit 03	Output Bit 03	RW	BOOL	O_CoolingFan	Q_3 Brake and motor fans	Global Variables
	Output Bit 04	Output Bit 04	RW	BOOL	O_OverRider_Hold	Q_4 OverRider Lock	Global Variables
	Output Bit 05	Output Bit 05	RW	BOOL	O_AccDec2	Q_5 Second acc / dec ramp	Global Variables
	Output Bit 06	Output Bit 06	RW	BOOL	O_OR_Auto_Lamp	Q_6 OverRider auto/continuous lamp	Global Variables
	Output Bit 07	Output Bit 07	RW	BOOL	O_RwLoading_Enable	Q_7 Unloaders Load enable - closing	Global Variables
	Output Bit 08	Output Bit 08	RW	BOOL	O_AcousticSignal	Q_8 Horn / Acoustic signal	Global Variables
	Output Bit 09	Output Bit 09	RW	BOOL	O_OverRider_Up	Q_9 Raise OverRider ~10mm Over Round Rewinder roll	Global Variables
OptionBoard Option Board Settings							
OptionBoard 1 NX1W-MAB221							
	Ch1 Analog Input Value	Value of analog input 1	R	INT	Pot1_Work_Velocity	Potentiometer 1 value - Work speed -	Global Variables
	Ch2 Analog Input Value	Value of analog input 2	R	INT	Pot2_Slow_Velocity	Potentiometer 2 value - Final speed -	Global Variables
	Ch1 Analog Output	Value of analog	RW	INT	AO_TOne_SetPoin	Analog O to T-one	Global

	Value	output 1			t	Tension SetPoint [Kg]	Variables
	Ch2 Analog Output Value	Value of analog output 2	RW	INT	InvRef	Inverter speed reference	Global Variables
NXBusMaster NX Bus Master							
	Unit Status (Under management of Master)						
	N3 NX Unit Registration Status	Status whether the NX Unit is registered to Unit configuration information or not.	R	BOOL			
	N3 NX Unit Message Enabled Status	Status whether communications with the NX Unit is possible or not.	R	BOOL			
	N3 NX Unit I/O Data Active Status	Status whether the NX Unit I/O data is controlled or not.	R	BOOL			
	N3 NX Unit Error Status	Status whether an error occurs on the NX Unit.	R	BOOL			
	N3 Time Stamp of Synchronous Input	Time information when the NX Unit has synchronous input refresh (in ns)	R	ULINT			
	N3 TimeStamp of Synchronous Output	Time information when the NX Unit has synchronous output refresh (in ns)	R	ULINT			
	N1 NX Unit Registration Status	Status whether the NX Unit is registered to Unit configuration information or not.	R	BOOL			
	N1 NX Unit Message Enabled Status	Status whether communications with the NX Unit is possible or not.	R	BOOL			
	N1 NX Unit I/O Data Active Status	Status whether the NX Unit I/O data is controlled or not.	R	BOOL			
	N1 NX Unit Error Status	Status whether an error occurs on the NX Unit.	R	BOOL			
	N1 Time Stamp of Synchronous Input	Time information when the NX Unit has synchronous input refresh (in	R	ULINT			

		ns)					
	N1 TimeStamp of Synchronous Output	Time information when the NX Unit has synchronous output refresh (in ns)	R	ULINT			
	N2 NX Unit Registration Status	Status whether the NX Unit is registered to Unit configuration information or not.	R	BOOL			
	N2 NX Unit Message Enabled Status	Status whether communications with the NX Unit is possible or not.	R	BOOL			
	N2 NX Unit I/O Data Active Status	Status whether the NX Unit I/O data is controlled or not.	R	BOOL			
	N2 NX Unit Error Status	Status whether an error occurs on the NX Unit.	R	BOOL			
	N2 Time Stamp of Synchronous Input	Time information when the NX Unit has synchronous input refresh (in ns)	R	ULINT			
	N2 TimeStamp of Synchronous Output	Time information when the NX Unit has synchronous output refresh (in ns)	R	ULINT			
Unit1 NX-PF0730							
Unit2 NX-AD2603							
	Ch1 Analog Input Value	Value of analog input 1	R	INT	AI_Diam_Unwinder	Analog value - UnWinder Diameter	Global Variables
	Ch2 Analog Input Value	Value of analog input 2	R	INT	AI_Diam_Rewinder	Analog value - ReWinder Diameter	Global Variables
Unit3 NX-EC0122							
	Ch1 Encoder Counter Status	Aggregated status of Encoder Counter 1	R	BYTE			
	Ch1 Counter Enabled	Encoder Counter 1 is operating.	R	BOOL			
	Ch1 Internal Reset Completed	Internal reset is completed for Encoder Counter 1.	R	BOOL			
	Ch1 Internal Latch Completed	Internal latch is completed for Encoder Counter 1.	R	BOOL			

Ch1 Preset Completed	Presetting is completed for Encoder Counter 1.	R	BOOL			
Ch1 Preset Command Value Invalid Flag	Preset value error flag for Encoder Counter 1.	R	BOOL			
Ch1 Counter Underflow Flag	Encoder Counter 1 underflow flag	R	BOOL			
Ch1 Counter Overflow Flag	Encoder Counter1 overflow flag	R	BOOL			
Ch1 Count Direction Flag	Encoder Counter 1 direction flag	R	BOOL			
Ch1 Reset/External Input Status	Aggregated status of reset/external inputs for encoder input 1	R	BYTE			
Ch1 External Input 0 Status	External input 0 status for encoder input 1	R	BOOL	I_Vacuum_OK	Extractor fan thermal OK	Global Variables
Ch1 External Input 1 Status	External input 1 status for encoder input 1	R	BOOL			
Ch1 External Input 2 Status	External input 2 status for encoder input 1	R	BOOL			
Ch1 External Input Enabled	External input enabled status for encoder input 1	R	BOOL			
Ch1 External Reset Enabled	External reset enabled status for encoder input 1	R	BOOL			
Ch1 Phase Z Reset Enabled	Phase Z reset enabled status for encoder input 1	R	BOOL			
Ch1 External Reset Completed Flag	External reset completed flag for encoder input 1	R	BOOL			
Ch1 Phase Z Reset Completed Flag	Phase Z reset completed flag for encoder input 1	R	BOOL			
Ch1 Encoder Present Position	Encoder 1 current position	R	DINT	Encoder_Pos	Encoder counter	Global Variables
Ch1 Pulse Rate	Counter 1 pulse rate	R	UDINT	PulseRate	Pulses per 100 ms (encoder unit, HMI diagnostic)	Global Variables
Ch1 Latch Status	Aggregated latch status of encoder input 1	R	WORD			
Ch1 Latch Input 1 Enabled	Latch input 1 enabled or disabled status of encoder input 1	R	BOOL			
Ch1 Latch Input 1	Latch input 1	R	BOOL			

Completed Flag	completed flag of encoder input 1					
Ch1 Latch Input 2 Enabled	Latch input 2 enabled or disabled status of encoder input 1	R	BOOL			
Ch1 Latch Input 2 Completed Flag	Latch input 2 completed flag of encoder input 1	R	BOOL			
Ch1 Latch Input 1 Data	The value latched by latch input 1 of external input/ phase Z input of encoder input 1	R	DINT			
Ch1 Latch Input 2 Data	The value latched by latch input 2 of external input/ phase Z input of encoder input 1	R	DINT			
Ch1 Pulse Period Measurement Status	Aggregated status of pulse period measurements of encoder input 1	R	BYTE			
Ch1 Pulse Period Measurement Enabled	Pulse period measurement for encoder input 1 is enabled.	R	BOOL			
Ch1 Pulse Period Measurement Value Clear Completed	Pulse period measurement value for encoder input 1 is cleared.	R	BOOL			
Ch1 Pulse Period Measurement Value Overflow Flag	Pulse period measurement value for encoder input 1 is overflowed.	R	BOOL			
Ch1 Encoder Counter Operation Command	Aggregated byte data of counter operation command for encoder input 1	W	WORD			
Ch1 Counter Enable	Enable or disable the counter for encoder input 1	W	BOOL	Encoder_On	Encoder enable	Global Variables
Ch1 Internal Reset Execution	Execute the internal reset for encoder input 1	W	BOOL	Encoder_Reset	Encoder reset	Global Variables
Ch1 Internal Latch Execution	Execute the internal latch for encoder input 1	W	BOOL			
Ch1 Preset Execution	Execute presetting for encoder input 1	W	BOOL			
Ch1 External Reset Enable	Enable the external reset for encoder	W	BOOL			

	input 1					
Ch1 Phase Z Reset Enable	Enable the phase Z reset for encoder input 1	W	BOOL			
Ch1 External Reset Completed Flag Clear	Clear the external reset completed flag for encoder input 1	W	BOOL			
Ch1 Phase Z Reset Completed Flag Clear	Clear the phase Z reset completed flag for encoder input 1	W	BOOL			
Ch1 Latch Function	Aggregated data of latch function for encoder input 1	W	WORD			
Ch1 Latch Input 1 Enable	Enable or disable the latch input 1 for encoder input 1	W	BOOL			
Ch1 Latch Input 1 Trigger Condition	Latch input 1 trigger condition for encoder input 1	W	BOOL			
Ch1 Latch Input 1 Trigger Selection	Toggle the external input and the phase Z input for the latch input 1 of encoder input 1	W	BOOL			
Ch1 Latch Input 2 Enable	Enable or disable the latch input 2 for encoder input 1	W	BOOL			
Ch1 Latch Input 2 Trigger Condition	Latch input 2 trigger condition for encoder input 1	W	BOOL			
Ch1 Latch Input 2 Trigger Selection	Toggle the external input and the phase Z input for the latch input 2 of encoder input 1	W	BOOL			
Ch1 Pulse Period Measurement Function	Aggregated byte data of the Pulse Period Measurement Function for encoder input 1	W	WORD			
Ch1 Pulse Period Measurement Enable	Enable the Pulse Period Measurement Function for encoder input 1	W	BOOL			
Ch1 Pulse Period Measurement Value Clear	Clear the pulse period measurement value for encoder input 1	W	BOOL			
Ch1 Pulse Period	Clear the pulse	W	BOOL			

	Measurement Value Overflow Flag Clear	period measurement overflow flag for encoder input 1

1-4.Controller Setup

1-4-1.Operation Settings

Operation Settings

Basic Settings	
Startup mode	RUN mode
Battery-related error detection	Do not use
Start delay time at startup	0 s
Memory card diagnosis at startup	Do not check
Program transfer by system-defined variable	Do not use
Password	
Restore by system-defined variable	Do not use
Password	
Instruction Error Output	Do not use
Write protection at startup	Do not use
Data backup prohibition to the SD Memory Card	Do not use
FINS Settings	
Local Network Table	No settings
Remote Network Table	No settings
Gateway counter	Standard (2)

Event Level Settings

*: Event level changed from the default value

Category	Event Name	Event Code	Event Level	
Automation Playback	Capacity Warning of Variable Log Save Destination	0x64050000	Observation	
Automation Playback	Cycle with No Variable Sampling	0x64070000	Observation	
Automation Playback	No Variable Log Concurrency	0x64060000	Observation	
Automation Playback	Upper Limit of Variable Sampling	0x95790000	Minor fault	
Automation Playback	Variable Log Overwritten	0x95760000	Observation	
Automation Playback	Variable Log Save Failed	0x152C0000	Observation	
CJ-series Analog I/O Units	A/D Conversion Error	0x04600000	Minor fault	
CJ-series Analog I/O Units	Cold Junction Sensor Error	0x04610000	Minor fault	
CJ-series Analog I/O Units	I/O Number Specification Error in Adjustment Mode	0x348C0000	Observation	
CJ-series Analog I/O Units	Input Disconnection Detected	0x64780000	Minor fault	
CJ-series Analog I/O Units	Input Error	0x647A0000	Minor fault	
CJ-series Analog I/O Units	Input Number Specification Error in Adjustment Mode	0x34820000	Observation	
CJ-series Analog I/O Units	Input Value Exceeded Adjustment Range in Adjustment Mode	0x34810000	Observation	
CJ-series Analog I/O Units	Output Number Specification Error in Adjustment Mode	0x34880000	Observation	
CJ-series Analog I/O Units	Output Set Value Error	0x64790000	Minor fault	
CJ-series Analog I/O Units	Zero/Span Adjustment Period End	0x647D0000	Observation	
CJ-series Analog I/O Units	Zero/span Adjustment Period Notice	0x647E0000	Observation	
CJ-series CompoNet Units	Communications Error	0x84600000	Minor fault	
CJ-series CompoNet Units	Repeater Unit Communications Error	0x84610000	Minor fault	
CJ-series DeviceNet Units	File Read/Write Error	0x048A0000	Observation	
CJ-series DeviceNet Units	Invalid Message Timer List Error	0x148C0000	Observation	
CJ-series DeviceNet Units	Remote I/O Communications Error	0x84750000	Minor fault	
CJ-series DeviceNet Units	Routing Table Error	0x34BC0000	Minor fault	
CJ-series DeviceNet Units	Unit Memory Error	0x04880000	Minor fault	
CJ-series EtherNet/IP Units	Link OFF Detected	0x84E20000	Observation	
CJ-series EtherNet/IP Units	Multiple Switches ON Error	0x54AE0000	Minor fault	
CJ-series Temperature Control Units	Cold Junction Sensor Error	0x04680000	Observation	
CJ-series Temperature Control Units	CT Overflow	0x64850000	Observation	
CJ-series Temperature Control Units	Heater Burnout Alarm	0x64860000	Observation	
CJ-series Temperature Control Units	Sensor Error	0x64840000	Observation	
CJ-series Temperature Control Units	Setting Error	0x34940000	Observation	
CPU Unit Built-in EtherNet/IP Port	Link OFF Detected	0x84060000	Observation	
CPU Unit Built-in EtherNet/IP Port	Tag Data Link Connection Timeout	0x84090000	Minor fault	
DB Connection	Execution Log Save Failed	0x14D20000	Minor fault	
DB Connection	SQL Execution Failure Log Save Failed	0x14D30000	Minor fault	
EtherCAT	Ring Disconnection Detected	0x84390000	Observation	
Ethernet	Link OFF Detected	0x85D50000	Observation	
Motion Control Function	Acceleration Warning	0x644E0000	Observation	
Motion Control Function	Actual Position Overflow	0x64540000	Observation	
Motion Control Function	Actual Position Underflow	0x64550000	Observation	
Motion Control Function	Command Position Overflow	0x64520000	Observation	
Motion Control Function	Command Position Underflow	0x64530000	Observation	

Motion Control Function	Deceleration Warning	0x644F0000	Observation	
Motion Control Function	Negative Torque Warning	0x64510000	Observation	
Motion Control Function	Notice of Insufficient Travel Distance to Achieve Blending Transit Velocity	0x94200000	Observation	
Motion Control Function	Positive Torque Warning	0x64500000	Observation	
Motion Control Function	Slave Observation Detected	0x74320000	Observation	
Motion Control Function	Velocity Warning	0x644D0000	Observation	
OPC UA	Execution Log Save Failed	0x15000000	Minor fault	
OPC UA	Server Certificate Expiration Notice	0x35D20000	Observation	
OPC UA	Server certificate Expired	0x35D10000	Minor fault	
SECS/GEM	GEM Service Log Save Failed	0x14E10000	Observation	
SECS/GEM	Invalid SD Memory Card	0x14E40000	Observation	
Self-Diagnosis	Battery-backup Memory Check Error	0x10090000	Minor fault	
Self-Diagnosis	Expanded Storage Data Corrupted	0x10140000	Observation	
Self-Diagnosis	Expanded Storage Life	0x00140000	Observation	
Self-Diagnosis	Low Battery Voltage	0x000B0000	Minor fault	
Self-Diagnosis	Non-volatile Memory Life Warning	0x00150000	Minor fault	
Self-Diagnosis	Non-volatile Memory Life Warning	0x00650000	Minor fault	
Self-Diagnosis	Real-Time Clock Stopped	0x00070000	Minor fault	
Self-Diagnosis	SD Memory Card Data Corrupted	0x10060000	Observation	
Self-Diagnosis	SD Memory Card Life Exceeded	0x00100000	Observation	
Self-Diagnosis	SD Memory Card Restored or Formatted	0x10040000	Observation	
Self-Diagnosis	Slow Fan	0x00120000	Minor fault	

1-4-2.Built-in EtherNet/IP Port Settings

Built-in EtherNet/IP Port Settings

TCP/IP Settings	
IP address settings	Fixed setting
IP address	192.168.250.10
Subnet mask	255.255.255.0
Default gateway	
DNS	Do not use
Priority DNS server	
Secondary DNS server	
Domain name	
Host Name - IP Address	No settings
Keep Alive	Use
Keep Alive monitoring time	300 sec
Linger option	Do not specify
IP Router Table	No settings
LINK Settings	
LINK settings	Auto
FTP Settings	
FTP server	Do not use
Port No.	21
Login name	
Password	
NTP Settings	
NTP server clock information	Do not get
Port No.	123
Server specifying method	IP address
IP address	
Host name	
NTP operation timing	Specify a time.
Time	00:00:00
Interval	60 min
Timeout time	10 sec
SNMP Settings	
SNMP service	Do not use
Port No.	161
Address	
Location	
Send a recognition trap.	FALSE
Recognition method	IP address
IP address	0.0.0.0
Host name	
Community name	public
Recognition 2	Do not use
Recognition method	IP address
IP address	0.0.0.0
Host name	
Community name	public

SNMP Trap Settings	
SNMP trap	Do not use
Port No.	162
Specifying method	IP address
IP address	
Host name	
Community name	public
Version	SNMPv2C
Trap 2	Do not use
Specifying method	IP address
IP address	
Host name	
Community name	public
Version	SNMPv2C
FINS Settings	
Node address of built-in EtherNet/IP port	10
FINS/UDP port number	9600
IP address-FINS address conversion method	Automatic generation
IP Address Table	No settings
Dynamically change remote IP	Convert

1-4-3.Built-in I/O Settings

Built-in I/O Settings

Input Filter Settings	
Input Filter Settings No.0 Input Filter	1 ms
Input Filter Settings No.0 Operation Mode	ON and OFF filters
Input Filter Settings No.1 Input Filter	1 ms
Input Filter Settings No.1 Operation Mode	ON and OFF filters
Input Filter Settings No.2 Input Filter	1 ms
Input Filter Settings No.2 Operation Mode	ON and OFF filters
Input Filter Settings No.3 Input Filter	1 ms
Input Filter Settings No.3 Operation Mode	ON and OFF filters
Load Rejection Output Settings	
Load Rejection Output Settings No.0 Outputs at Load Rejection	Turn OFF
Load Rejection Output Settings No.1 Outputs at Load Rejection	Turn OFF
Load Rejection Output Settings No.2 Outputs at Load Rejection	Turn OFF
Load Rejection Output Settings No.3 Outputs at Load Rejection	Turn OFF
Load Rejection Output Settings No.4 Outputs at Load Rejection	Turn OFF
Load Rejection Output Settings No.5 Outputs at Load Rejection	Turn OFF
Load Rejection Output Settings No.6 Outputs at Load Rejection	Turn OFF
Load Rejection Output Settings No.7 Outputs at Load Rejection	Turn OFF
Load Rejection Output Settings No.8 Outputs at Load Rejection	Turn OFF
Load Rejection Output Settings No.9 Outputs at Load Rejection	Turn OFF

1-4-4.Option Board Settings

Option Board Settings

Option Board 1	
Option board	NX1W-MAB22110
Option Board 2	
Option board	Not mounted

1-4-5.Memory Settings

Memory Settings for CJ-series Units

Enable	Area Type	Size (Number of Words)	Retain
Enabled	CIO	6144	Not retained
Enabled	WR	512	Not retained
Enabled	HR	512	Retained
Enabled	DM	4096	Retained

1-8.POUs

1-8-1.Programs

1-8-1-1.TaglMonoTRM

1-8-1-1-1.Variables

Name	Data Type	Initial Value	AT	Retain	Constant	Comment
VAR						
SpdDelta	REAL			False	False	Minimum or maximum speed SpdDeltaerence
TensProfile	REAL			False	False	
UwSlowDiamCalc	REAL			False	False	V7c: speed-dependent unwinder slow-down diameter (mm)
SonarRunMem	bool			False	False	
EndStopReq	BOOL			False	False	V12: end-of-roll stop requested (AutoStopCmd or unwinder diameter)
EndRampOK	BOOL			False	False	V12: end-of-roll ramp usable (FinalStopDecel, FF active, running)
EndRampHard	BOOL			False	False	V12: safety net during the ramp -> immediate stop
EndStopNow	BOOL			False	False	V12: immediate end-of-roll stop (Motor_Manager rung 4)
UwDiamAtLatch	REAL			False	False	Unwinder diameter when the slow-down point was latched (release check)
CntUwWasEmpty	BOOL			True	False	Metre reset: unwinder roll was empty since the last reset
CntStartMem	BOOL			False	False	Metre reset: StartPB_OK memory
CntPulseReq	BOOL			False	False	Metre reset: start of the reset pulse
CntPulseDone	BOOL			False	False	Metre reset: pulse given, waiting for counter ~0
CntAutoReset	BOOL			False	False	Metre reset: pulse to Encoder_Reset (Motor_Manager rung 2)
T_CntNewRoll	TON			False	False	Metre reset: new parent roll confirmed 3 s
T_CntResetPulse	TP			False	False	Metre reset: 50 ms reset pulse
T_CntDone	TON			False	False	Metre reset: safety 2 s after the pulse
TensTarget	REAL			False	False	Tension Manager: target before rate limit (kg)
TensStep	REAL			False	False	Tension Manager: max change per 4 ms scan (kg)
TensAOReal	REAL			False	False	Tension Manager: AO counts before INT conversion
SonarInit	bool			False	False	
AlarmMem	ARRAY[0..32] OF bool			False	False	
UwSlowDiamLatch	BOOL			False	False	V7c: unwinder diameter slow-down reached (frozen)
FFRate	REAL			False	False	V7f: ramp rate (reference per 4 ms scan)
FFRateSet	REAL			False	False	V7f: rate allowed to still stop at the target
FFJerk	REAL			False	False	V7f: max rate change per scan
SpdScanCnt	INT			False	False	V9: scan counter for the 100 ms s

						peed window
TensGSMMem	REAL			False	False	
TensWidthMem	REAL			False	False	
TensKraftMem	BOOL			False	False	
HornRun	BOOL			False	False	V9: slow-down / final speed running
HornRunMem	BOOL			False	False	V9: edge memory
HornStartMem	BOOL			False	False	V9: start edge memory
HornTicks	INT			False	False	V9: scans since slow-down start
HornCnt	INT			False	False	V9: scans in the beep period
HornPer	INT			False	False	V9: beep period in scans
HornStartScans	INT			False	False	V9: start beep in scans
HornLenScans	INT			False	False	V9: short beep in scans
HornSlowBeep	BOOL			False	False	V9: slow-down beep
HornStopPend	BOOL			False	False	V9: stopped after a slow-down, waiting for standstill
HornStopTicks	INT			False	False	V9: stop beep counter
LongActive	BOOL			False	False	V9: long-baseline learning running
LongValid	BOOL			False	False	V9: long baseline >= 20 mm growth, replaces the windows
LongStartPos	DINT			False	False	V9: encoder at the start of the long baseline
LongStartD	REAL			False	False	V9: rewinder diameter at the start (mm)
LongGrow_mm	REAL			False	False	V9: growth since the start (mm)
LongPls	REAL			False	False	V9: pulses since the start
LongK	REAL			False	False	V9: K from the long baseline
RevW	REAL			False	False	V9: encoder pulses summed in the current revolution
RevDP	DINT			False	False	V9: encoder pulses this scan
RevPrevPos	DINT			False	False	V9: encoder last scan
RevSum	REAL			False	False	V9: sum of raw DT35 distance x pulses
RevMin	REAL			False	False	V9: min raw distance in the revolution
RevMax	REAL			False	False	V9: max raw distance in the revolution
RwDiamRevOk	BOOL			False	False	V9: one full revolution averaged since the last fault
UwEmptyTicks	INT			False	False	V9: unwinder empty time (100 ms ticks)
UwFullTicks	INT			False	False	V9: new roll seen time (100 ms ticks)
GsmArmed	BOOL			False	False	V9: roll was empty -> next new roll asks the GSM question
T_JogStartBlk	TOF			False	False	V9: START blocked while JOG held + 2 s
StartHeldBlk	BOOL			False	False	V9: START pressed during the block, wait for release
StartPB_OK	BOOL			False	False	V9: START push-button accepted (replaces I_Start_PB in General 7/8/10)
FFSoftMem	BOOL			False	False	V9: soft stop last scan (rung B)
FFSlowAcc	BOOL			False	False	V9: gentle ramp-up after a cancelled soft stop
FFStartEdge	BOOL			False	False	V9: START push-button edge (rung B)
FFStartMem	BOOL			False	False	V9: START last scan (rung B)
SpdDeltaMin	REAL			False	False	Minimum speed difference

FFSqrtArg	REAL			False	False	SQRT argument (REAL)
SpdDeltaMax	REAL			False	False	Maximum speed SpdDeltaerence
SoftStopMem	BOOL			False	False	v6r: soft stop edge memory
SoftStopRatio	REAL			False	False	v6r: InvRef counts per m/min, frozen at the soft stop start
MemPos	DINT			False	False	Previous encoder count
MatChangeCnt	INT			False	False	v6s: consecutive learning windows far from RwMat_K
DeltaPulse	DINT			False	False	Encoder count difference
SlowMem	BOOL			False	False	SlowDown edge memory
DecelDiam	REAL			False	False	Rewinder diameter used by the decel model (mm, min 100)
TeachSwap	REAL			False	False	Temporary for UNDO swap
SlowStartDiam	REAL			False	False	Diameter at slow-down start (mm)
BrakeHold	BOOL			False	False	Keep T-one in auto during a normal stop (O_Brake)
StopNow	BOOL			False	False	Immediate stop request (replaces I_Stop_PB in General rung 9)
StopPB_Mem	BOOL			False	False	STOP button edge memory
T_StopLong	TON			False	False	STOP held 1 s = immediate stop
T_SoftStop	TON			False	False	Soft stop timeout (120 s)
BrakeHoldTicks	INT			False	False	BrakeHold duration after stop (100 ms ticks, max 150)
StopStartDiam	REAL			False	False	Diameter at stop command (mm)
DecelK_Meas	REAL			False	False	Measured decel per mm of diameter
SlowMeasActive	BOOL			False	False	Slow-down measurement running
TeachTicks	INT			False	False	TEACH CORE stable time (100 ms ticks)
SlowStartPos	DINT			False	False	Encoder at slow-down start
SlowStartSpd_Pps	REAL			False	False	Speed at slow-down start (pulses/s)
SlowMeas_Pls	REAL			False	False	Measured slow-down length (pulses)
SlowDecelMeas_mm	REAL			False	False	Measured slow-down decel (mm/s2)
SlowOneRamp	BOOL			False	False	Slow-down uses one ramp (manual or learned)
Velocity	REAL			False	False	Speed in [mm/s]
Alarm_Array	ARRAY[0..32] OF BOOL			False	False	Alarms
T_PlcOn	TON			False	False	PLC active timeout
ControlVoltage	BOOL			False	False	Auxiliaries OK
StartMotor_Cmd	BOOL			False	False	Motor start command
HubExtractEnable	BOOL			False	False	Hub extraction enable command
Brake_Cmd	BOOL			False	False	Brake command
T_StopVent	TOF			False	False	Fans stop timer
TimStopVen	TIME			False	False	Fans stop timeout
MotorCooling	BOOL			False	False	Fans command
OverRider_Auto	BOOL			False	False	OverRider in automatic
OverRider_Hoover	BOOL			False	False	OverRider in Hoover Mode
OverRider_Lock	BOOL			False	False	OverRider lock command
GSM_LearnFault	BOOL			False	False	Paper [Grams / Square Meter] LearnFailCnt ≥ RwLearnFailMax
TickCnt	INT			False	False	100 ms tick scan counter (25 x 4 ms)

RunWasJog	BOOL			False	False	Run used jog / manual / slow button (2nd ramp)
StopLearnValid	BOOL			False	False	This stop may be learned
SpdBand_Pps	REAL			False	False	Speed band of the trim ramp (pulses/s)
TrimDecel_Pps2	REAL			False	False	Trim ramp decel (pulses/s2)
SpdKnee_Pps	REAL			False	False	Speed where coarse ramp hands over to trim ramp (pulses/s)
OverRider_RaiseUp	BOOL			False	False	OverRider raise command
Pot1_Mem	INT			False	False	Work potentiometer value
VelSet_mms	REAL			False	False	Set speed in mm/s
Vfd_SP_Modif	REAL			False	False	Inverter speed modifier
RwAutoLimAct	BOOL			False	False	v6y: rewinder AUTO speed limit active (taken over at standstill)
UwTeachTicks	INT			False	False	Unwinder TEACH CORE samples (100 ms)
T_PosPresOff	TON			False	False	OverRider photocell OFF timer
T_PosPresOn	TON			False	False	OverRider photocell ON timer
TimPosPresOff	TIME			False	False	OverRider photocell OFF timeout
TimPosPresOn	TIME			False	False	OverRider photocell ON timeout
RollEnd_Active	BOOL			False	False	Roll diameter photocell intercepted
Output_Ratio	REAL			False	False	Analog output ratio
Input_Ratio	REAL			False	False	Analog input ratio
T_StartLow	TON			False	False	Slow start timer
UwDiamReached	BOOL			False	False	Roll diameter reached
Index_A	INT			False	False	
AlarmStrobe	BOOL			False	False	Alarm message display strobe
T_Display	TON			False	False	Alarm display timer
RangeOK	BOOL			False	False	Range reached
Vel_Max_mms	REAL			False	False	Maximum speed in mm/s
Slow_Inhibit	BOOL			False	False	Inhibit slow button
Decel_Ramp	REAL			False	False	Deceleration ramp
Accel_Ramp	REAL			False	False	Acceleration ramp
Pot2_Mem	INT			False	False	Final potentiometer value
Current_mm	REAL			False	False	Current millimetres
EncoderPulse_to_mm	REAL			False	False	mm per encoder pulse
Stop	BOOL			False	False	Stop
T_EncoderStill	TON			False	False	(?) Encoder stopped timer
Analog_Res	UINT			True	False	Analog board resolution
Mt_To_mm	REAL			False	False	m/min to mm/s factor
Init_Ramp	BOOL			True	False	(?) Ramp initialisation
SlowByDiam	BOOL			False	False	Slow down by diameter
UwStopByDiam	BOOL			False	False	Stop by diameter
T_Screen_Delay	TON			False	False	(?) Start page timer
T_PulStop	TON			False	False	(?) Stop button timer
Reading_Max	REAL	2000		True	False	Maximum sensor reading (full scale)
DeadZone	REAL	60		True	False	Sensor dead zone
In_UwDiameter	REAL			True	False	UnWinder Diameter Analog input voltage
Tick	BOOL			False	False	100 ms tick
MmPerPulse	REAL			False	False	mm per encoder pulse
RwDistRaw_mm	REAL			False	False	Rewinder Unfiltered distance (mm)
RwFiltInit	BOOL			False	False	Rewinder Filter initialised

SpdInit	BOOL			False	False	Speed init
EncPrev	DINT			False	False	Encoder at previous tick
SpdAct_Pps	REAL			False	False	Actual Speed (pulses/s)
LearnInit	BOOL			False	False	Learn initialised
LearnWinRunning	BOOL			False	False	Motor ran whole window
LearnWinTicks	INT			False	False	Learn window counter
LearnWin_Pls	DINT			False	False	Pulses in window
LearnWinDiamDelta	REAL			False	False	Diameter change in window
MatK_Window	REAL			False	False	Material factor this window
MotorRunMem	BOOL			False	False	Motor start edge memory
StopMeasActive	BOOL			False	False	Stop measurement running
StopStartPos	DINT			False	False	Encoder at stop command
StopStartSpd_Pps	REAL			False	False	Speed at stop command (pulses/s)
StandstillTicks	INT			False	False	Standstill tick counter
LearnWinOk	BOOL			False	False	Learning window gave a valid result
ThickWin_mm	REAL			False	False	Paper thickness this window (mm)
LearnFailCnt	INT			False	False	Consecutive failed learning windows
StopLearnOk	BOOL			False	False	Stop measurement gave a valid result
StopDecelMeas_mm	REAL			False	False	Measured stop decel (mm/s2)
StopMeas_Pls	REAL			False	False	Measured stop length (pulses)
StopDecelMeas	REAL			False	False	Measured stop decel (pulses/s2)
StopLen_Pls	REAL			False	False	Stop length (pulses)
StopLenFinal_Pls	REAL			False	False	Stop length from slow speed (pulses)
RwSlowLen_Pls	REAL			False	False	Rewinder Slow-down length (pulses)
SlowDecel_Pps2	REAL			False	False	Slow-down decel (pulses/s2)
SpdFinal_Pps	REAL			False	False	Final speed (pulses/s)
RwRemain_Pls	REAL			False	False	Rewinder Remaining length to target (pulses)
LearnWinStartPos	DINT			False	False	Rewinder Position Window 0
LearnWinStartDiam	REAL			False	False	Rewinder Diameter Window 0
T_ByPassProtOff	TOF			False	False	Delay Off Timer
UwEnMem	BOOL			False	False	Edge memory UwStopEnable
RwEnMem	BOOL			False	False	Edge memory RwEnable
SpdRef_Pps	REAL			False	False	Reference speed MAX (actual,setpoint) (pulses/s)
SpdWork_Pps	REAL			False	False	Work speed setpoint (pulses/s)
SlowLenBase_Pls	REAL			False	False	Slow-down length without margin (pulses)
UwSlowLen_Pls	REAL			False	False	Unwinder slow-down length (pulses)
UwMarginUsed_m	REAL			False	False	Unwinder margin at slow speed (m)
SqrtArg	REAL			False	False	Square root argument
AutoSlowCmd	BOOL			False	False	Automatic Slow Command
AutoStopCmd	BOOL			False	False	Automatic Stop Command
RwLastValidDiam	REAL			False	False	Rewinder Last valid diameter (mm)

	RwRemainMax_Pls	REAL			False	False	Rewinder Remaining length to max diameter (pulses)
	UnWinder_Empty	BOOL			False	False	UnWinder Empty
	StopFailCnt	INT			False	False	Consecutive failed stop measurements
	UwTeachSum	REAL			False	False	Unwinder TEACH CORE sonar sum (mm)
	T_MatReset	TON			False	False	v6s: RwMatReset stays ON 3 s
	RwAutoMaxSpdAct	REAL			False	False	v6y: active rewinder AUTO max speed (taken over at standstill)
	UwEmptyMem	BOOL			False	False	v6x: UnWinder_Empty edge memory
	LearnWinLen_Pls	REAL			False	False	v6w: learning window length in encoder pulses
	WebRatio	REAL			False	False	V7: drive counts per m/min of web at the current diameter
	FFActive	BOOL			False	False	V7: feed-forward loop active
	FFWasActive	BOOL			False	False	V7: edge memory
	FFTarget	REAL			False	False	V7: reference for the set speed (mm/s of the 0-10 V scale)
	RampRef	REAL			False	False	V7: ramped reference (mm/s of the 0-10 V scale)
	StepAcc	REAL			False	False	V7: accel step per scan
	StepDec	REAL			False	False	V7: decel step per scan
	FFSteadyTicks	INT			False	False	V7: scans at target
	FFErr	REAL			False	False	V7: relative speed error
	Clock_100ms	bool			False	False	Clock100ms
	AlarmCnt	INT			False	False	
	AlarmCntMem	INT			False	False	
	RstEdge	BOOL			False	False	
	RstMem	BOOL			False	False	
	SilenceTicks	INT			False	False	
	WarnCnt	INT			False	False	
	WarnCntMem	INT			False	False	
	WarnBeepTicks	INT			False	False	
	TensIdeal_N	REAL			False	False	
	TensionSeen	BOOL	False		False	False	Tension reached once since START
	T_TensOn	TON			False	False	Tension ON debounce 0.5 s
	T_TensOff	TON			False	False	Tension OFF debounce 2 s
	T_UwSensFlt	TON			False	False	Unwinder sonar fault debounce 1 s
	AS_RunMem	BOOL			False	False	
	AutoStopSeen	BOOL			False	False	
	T_AutoStopDisp	TON			False	False	
	RwArmMem	BOOL			False	False	
	UwArmMem	BOOL			False	False	
	T_AutoEnRef	TON			False	False	
	TensRef	REAL			False	False	
	WebBreakNow	BOOL			False	False	
	TensSPLow	REAL			False	False	
VAR_EXTERNAL							
	VelSet	REAL				False	Set speed
	OverSpeed	BOOL				False	Speed too high
	SpdDelta_Percent	REAL				False	Speed difference percentage
	Encoder_Pos	DINT				False	Encoder counter
	FinalStopDecel	REAL				False	
	FinalStopRamp	BOOL				False	

Velocity_M_min	REAL				False	Speed in m/min
Current_Metre	REAL				False	Current metres
PaperKraft	BOOL				False	
TensFactorBleached	REAL				False	
TensFactorKraft	REAL				False	
TensAuto	BOOL				False	
TensAOEnable	BOOL				False	
TensManual_kg	REAL				False	
TensTaper	REAL				False	
TensCoreBoost	REAL				False	
TensRate_kg_s	REAL				False	
TensMin_kg	REAL				False	
TensMax_kg	REAL				False	
TensSet_kg	REAL				False	
TensAtCore	REAL				False	
TensAtMid	REAL				False	
TensAtMax	REAL				False	Metre counter reset button
TensDiaFrac	REAL				False	
SonarAskYes	BOOL				False	
WorkSpd_Max_Set	REAL				False	HMI request: work pot full scale (m/min), applied only with motor stopped
FinalSpd_Max_Set	REAL				False	HMI request: final pot full scale (m/min), applied only with motor stopped
SpdMax_Pending	BOOL				False	New pot range waiting for the motor to stop
RwAxisDist_Prev	REAL				False	Rewinder: RwAxisDist before the last TEACH (for UNDO)
UwCoreDist_Prev	REAL				False	Unwinder: UwCoreDistance before the last TEACH (for UNDO)
RwTeachUndo	BOOL				False	HMI: UNDO rewinder TEACH (PLC resets it)
UwTeachUndo	BOOL				False	HMI: UNDO unwinder TEACH (PLC resets it)
RwLearnReset	BOOL				False	HMI / Watch: clear learned decels RwStopDecelK, RwStopDecel, RwSlowDecelK (PLC resets it)
UnderSpeed	BOOL				False	Speed too low
I_ControlVoltage	BOOL				False	Auxiliaries ON
I_Side_Door_Closed	BOOL				False	Roll diameter photocell
RwCoreDiameter	REAL				False	Rewinder core outside diameter for TEACH CORE (mm)
UwTeachState	INT				False	Unwinder TEACH CORE: 0 idle 1 teaching 2 done 3 refused
SoftStop	BOOL				False	Soft stop in progress (HMI)
SoftStopEnable	BOOL				False	STOP button = soft stop (False = immediate as before)
SoftStopEndSpd	REAL				False	Soft stop: run command drops below this speed (m/min)
SoftStopDecel	REAL				False	Soft stop decel (mm/s2), 0 = DecelCoarse
RwTeachCore	BOOL				False	HMI: TEACH CORE request (PLC resets it)
RwTeachState	INT				False	TEACH CORE: 0 idle 1 teaching 2 done 3 refused

O_MotorStart	BOOL				False	Motor start
RwSlowDecelK	REAL				False	Learned slow-down decel per mm of rewinder diameter (mm/s2 per mm)
RwStopDecelK	REAL				False	Learned stop decel per mm of rewinder diameter (mm/s2 per mm)
I_OR_Position_Sens	BOOL				False	Rider roll position sensor
WorkSpd_Max	REAL				False	Work speed pot full scale (m/min), 50..VelMax
FinalSpd_Max	REAL				False	Final speed pot full scale (m/min), 5..WorkSpd_Max
I_OR_Hoover_Mode	BOOL				False	Rider roll pressure switch
I_OR_Auto_OS	BOOL				False	Rider roll in automatic
I_OR_Cont_OS	BOOL				False	Rider roll in continuous
I_Start_PB	BOOL				False	Start button
I_Motor_Thermal	BOOL				False	Motor thermal switch
I_FanTemp_Ok	BOOL				False	Motor (fan) thermal OK
I_Stop_PB	BOOL				False	Stop button
I_Hub_Position	BOOL				False	Hub/chuck in extraction position
I_FrontGuard_Closed	BOOL				False	Front guard closed
RwMaxSlowFallback	REAL				False	Rewinder max: slow-down diameter while K not learned (mm)
RwMaxSlowAtDiam	REAL				False	Rewinder max: slow-down starts at diameter (mm)
O_Alarm_Array	BOOL				False	Alarm lamp
O_RwUnload	BOOL				False	Unloaders enable - opening
I_Prot_ByPass	BOOL				False	Guards-bypassed selector switch
O_Prot_ByPass_Lamp	BOOL				False	Guards bypassed lamp
O_Brake	BOOL				False	Brake command
Fan_Timeout	REAL				False	Fans stop timeout (from panel)
O_CoolingFan	BOOL				False	Brake and motor fans
O_OR_Auto_Lamp	BOOL				False	Rider roll auto/continuous lamp
O_OverRider_Up	BOOL				False	Raise rider roll
O_OverRider_Hold	BOOL				False	Lock rider roll
AO_TOne_SetPoint	INT				False	Fixed 10 Vdc generator
InvRef	INT				False	Inverter speed reference
Encoder_On	BOOL				False	Encoder enable
Encoder_Reset	BOOL				False	Encoder reset
OverRider_TimeOFF	REAL				False	Rider roll photocell OFF timeout (panel)
OverRider_TimeON	REAL				False	Rider roll photocell ON timeout (panel)
VelMax	REAL				False	Maximum speed
I_Jog_PB	BOOL				False	Slow speed button
O_Motor_Lamp	BOOL				False	Motor ON lamp
AlarmOn	BOOL				False	Alarm active
BuzzerOn	BOOL				False	Panel buzzer command
I_VFD_Alarm	BOOL				False	Inverter in alarm
Msg	INT				False	Message

Index_B	INT				False	Active alarm index
MotStop	BOOL				False	Motor stopped
Clock1s	BOOL				False	1 s clock
ManMode	BOOL				False	Manual mode
Counter_Reset	BOOL				False	Panel counter reset button
I_FrontGuard_Open	BOOL				False	Front guard open
O_RwLoading_Enable	BOOL				False	Unloaders enable - closing
JogSpd_SP	REAL				False	Speed in manual
O_AccDec2	BOOL				False	Second acc/dec ramp
AccelTrim	REAL				False	Acceleration coefficient in run
DecelTrim	REAL				False	Deceleration coefficient in run
DecelCoarse	REAL				False	Acceleration coefficient at start (orig. comment; name suggests decel)
AccelCoarse	REAL				False	Deceleration coefficient at start (orig. comment; name suggests accel)
Speed_Display	REAL				False	Set speed (displayed)
O_AcousticSignal	BOOL				False	Horn / acoustic signal
Pot1_Work_Velocity	INT				False	Potentiometer 1 value - Work speed -
O_FrontGuard_ByPass	BOOL				False	Front guard bypass
Pot2_Slow_Velocity	INT				False	Potentiometer 2 value - Final speed -
LowSpd_Display	REAL				False	Slow-down speed (displayed)
Encoder_Pulse	INT				False	Encoder pulses
RwSlowDecelLrn	REAL				False	Learned real slow-down decel (mm/s2) - 0 = not learned
UwSlowMargin_m	REAL				False	Slow-down metres
UwStop_at_M_Val	REAL				False	Total metres
I_AutoStop_OS	BOOL				False	Stop on metre counter
I_Input_11_Spare_1255	BOOL				False	Slow down at end of roll
I_T_one_TensionOK	BOOL				False	Slow-down photocell
I_WebBreak_Monitor	BOOL				False	Paper break monitoring ON
I_WebBreak_PH	BOOL				False	Paper break photocell
I_Vacuum_OK	BOOL				False	Extractor fan thermal OK
CountReset_Req	BOOL				False	Metre counter reset request
Psw	INT				False	Password
I_RsetAlarm	BOOL				False	Alarm reset
EncoderWheel_Circumf	REAL				False	Encoder wheel diameter
AlarmCodeSV	ARRAY[0..32] OF BOOL				False	Panel alarms
SafeGuard_ByPass	BOOL				False	Guards bypassed
FrontGuard_Closed	BOOL				False	Front guard closed
Core_Measure	BOOL				False	(?) Stored core value
UwDiam_Measure	REAL				False	Measured distance
UwCoreDistance	REAL				False	(?) Core measurement

RwSlowFallbackMm	REAL				False	
AI_Diam_Unwinder	INT				False	Analog value - parent roll measurement
RwMatChangePct	REAL				False	
UwCoreDiameter	REAL				False	(?) Shaft diameter
Measure_ON	BOOL				False	(?) Measurement enabled
UwDiam_Act	REAL				False	(?) Measured roll diameter
UwStopDiam_Limit	REAL				False	(?) Stop diameter
UwSlow_at_D_Val	REAL				False	(?) Slow-down diameter
Alarm_Reset	BOOL				False	(?) Alarm reset from panel
HMI_Screen	ARRAY[0..1] OF DINT				False	HMI panel page
Psw22	BOOL				False	(?) Password 22
RwMatReset	BOOL				False	
Psw44	BOOL				False	(?) Password 44
AI_Diam_Rewinder	INT				False	Rewinder DT35 raw - I/O Map Unit2 Ch2
RwStopEnable	BOOL				False	Rewinder diameter stop enabled
RwTargetDiam	REAL				False	Rewinder final diameter (mm)
RwUseSlow	BOOL				False	Slow down before target
RwSlowMargin_m	REAL				False	Metres at slow speed before stop
UwRawMin	INT				False	
UwRawMax	INT				False	
RwSlowDecel	REAL				False	Slow-down decel mm/s2 - 0 = auto from DecStart
RwResetHyst	REAL				False	Reset requests below target - this (mm)
RwDist0V	REAL				False	DT35 distance at 0 V (mm)
RwDist10V	REAL				False	DT35 distance at 10 V (mm)
RwRawMin	INT				False	Raw below = sensor fault
RwRawMax	INT				False	Raw above = sensor fault
RwStopOnFault	BOOL				False	Stop on sensor fault
RwAxisDist	REAL				False	Sensor to shaft centre (mm) - teach
RwFilter_K	REAL				False	Distance filter per 100 ms (0..1)
RwMinPulses	DINT				False	Min pulses in window to learn
RwLearn_K	REAL				False	Material factor filter weight
RwMat_K	REAL				False	Learned material factor
RwLearnStop	BOOL				False	Learn stop decel on every stop
RwMinLearnSpd	REAL				False	Min speed to learn a stop (m/min)
RwStopDecel	REAL				False	Learned stop decel (pulses/s2)
RwTreact	REAL				False	Stop reaction time (s)
RwTstopDef	REAL				False	Stop time until decel learned (s)
RwTrim	REAL				False	Stop length trim (>1 = earlier)
RwDist	REAL				False	Filtered distance (mm)
RwDiam	REAL				False	Rewinder diameter (mm)
RwSpeed	REAL				False	Line speed from encoder (m/min)
RwRemain_m	REAL				False	Metres to target diameter
RwStopLen_m	REAL				False	Stopping length now (m)
RwSlowLen_m	REAL				False	Slow-down length now (m)
RwStop_at_D_Req	BOOL				False	Rewinder diameter STOP request
RwSlow_at_D_	BOOL				False	Rewinder diameter SLOW reque

Req						st
RwSensFault	BOOL				False	Rewinder DT35 sensor fault
UwStopEnable	BOOL				False	Unwinder metre-counter stop ON (HMI)
SlowAtDisp	REAL				False	HMI: slow-down starts at (m or mm, see SlowAtMode)
UwSlow_at_D_Req	BOOL				False	
UwSlowDiamAct	REAL				False	
RwJogMaxDiam	REAL				False	
JogInhibit	BOOL				False	
SCurveTime	REAL				False	
TOneDecel	BOOL				False	
HornStartBeep_s	REAL				False	
HornBeepLen_s	REAL				False	
HornBeepPeriod	REAL				False	
HornStopBeep_s	REAL				False	
HornTest	BOOL				False	
RwLockDiam	REAL				False	
UwSensTest	BOOL				False	
RwLockMinGrow_mm	REAL				False	
RwMatLocked	BOOL				False	
LongThick_mm	REAL				False	
RwDiamRev	REAL				False	
RwWave_mm	REAL				False	
UwStopLenShow	BOOL				False	
UwSlowDiamShow	BOOL				False	
SoftCancelAccel	REAL				False	
SpdHold	BOOL				False	
SlowAtMode	INT				False	HMI: 0=none 1=unwinder [m] 2=rewinder [mm]
SlowAtShow	BOOL				False	HMI: show slow-down display
UwSlow_at_M_Val	REAL				False	Unwinder: slow-down starts at (m)
UwSlow_at_M_Req	BOOL				False	Unwinder metre slow-down request (latched)
RwSlow_at_D_Val	REAL				False	Rewinder: slow-down starts at diameter (mm)
RwStop_at_D_Val	REAL				False	Rewinder: stop command at diameter (mm)
RwMaxDiam	REAL				False	Rewinder max diameter protection (mm)
RwMaxStop	BOOL				False	Rewinder max diameter STOP (latched)
RwMaxSlow	BOOL				False	Rewinder max diameter SLOW (latched)
UwEmptyHyst	REAL				False	
RwThickMin_mm	REAL				False	Learning: min plausible paper thickness (mm)
RwThickMax_mm	REAL				False	Learning: max plausible paper thickness (mm)
RwDecelMin_mm	REAL				False	Learning: min plausible stop decel (mm/s2)
RwDecelMax_mm	REAL				False	Learning: max plausible stop decel (mm/s2)
RwLearnFailMa	INT				False	Learning: failed windows before

x						alarm (12 = 2 min)
RwStopFailMax	INT				False	Learning: failed stops before alarm
GSM_Learned_OK	BOOL				False	Rewinder learning fault (alarm 16)
RwThick_mm	REAL				False	Learned paper thickness (mm)
Psw33	BOOL				False	
RwAutoMaxSpd	REAL				False	
RwWinLen_m	REAL				False	
RwWinGrow_mm	REAL				False	
GsmAsk	BOOL				False	
GsmAskYes	BOOL				False	
DecelWeb	REAL				False	
AccelWeb	REAL				False	
RwRatioD	REAL				False	
FFEnable	BOOL				False	
UwStopDiamShow	BOOL				False	
UwStop_at_D_Val	REAL				False	
Clock10ms	BOOL				False	
Sec_to_Min	REAL				False	
Psw48	BOOL				False	
JogRun	BOOL				False	
GuardStopOK	BOOL				False	V11: stopped for real -> guard bypass allowed (Output 13)
SlowDown	BOOL				False	Slow down
WarningOn	bool				False	
AlarmSilenced	BOOL				False	
PaperGSM	REAL				False	
WebWidth_mm	REAL				False	
TensFactor	REAL				False	
TensIdeal_Disp	REAL				False	
TensCheckEnable	BOOL				False	
TensLostWarn	BOOL				False	
TensWaitSpd	REAL				False	
TensWait	BOOL				False	
TensionOK	BOOL				False	
UwSensFault	BOOL				False	
Thickness_Fault	BOOL				False	
AutoStopDone	BOOL				False	
AutoStopWhy	INT				False	
AutoEnRefused	BOOL				False	
AutoEnWhy	INT				False	
TensWaitHz	REAL				False	
StartBlocked	BOOL				False	
GuardOpen	BOOL				False	
SideDoorOpen	BOOL				False	
SonarAsk	BOOL				False	
Psw40	BOOL				False	
TensJogBoost	REAL				False	
TensTrim_kg	REAL				False	
TensJogLatch	BOOL				False	
CntResetMode	INT				False	
CntResetPend	BOOL				False	
LastRoll_m	REAL				False	
RwSlowLearnMin_mm	REAL				False	

	TensDecelBoost	REAL				False	
	TensDecelActive	BOOL				False	
	TensDecelRate_kg_s	REAL				False	
	TensSPMin_kg	REAL				False	

1-8-1-1-2.General

0 Remote 10.146.233.11 ... 1s clock.



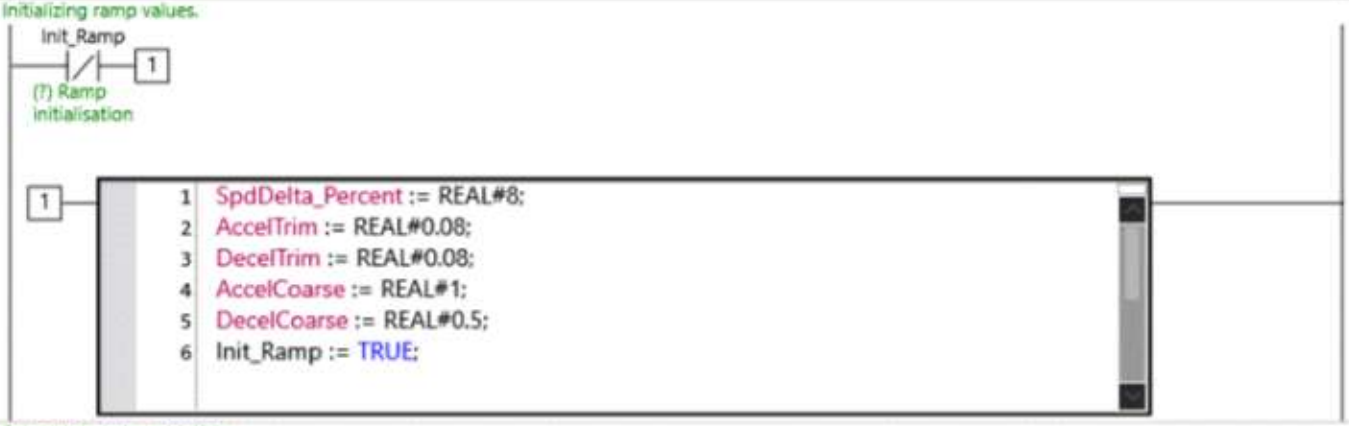
1 100ms clock.



2 10ms clock.



3 Initializing ramp values.



4 Set analog scale resolution.



5 Variable for converting m/min to mm/sec and vice versa.



6 10 V DC source.



7 START blocked by JOG

```
T_JogStartBlk(In := I_Jog_PB, PT := T#2s);      // TOF: ON while held + 2 s
StartHeldBlk := (StartHeldBlk OR (I_Start_PB AND T_JogStartBlk.Q)) AND I_Start_PB;
StartPB_OK := I_Start_PB AND NOT T_JogStartBlk.Q AND NOT StartHeldBlk;

// Side door open (no key bypass): no JOG, and stop any JOG / run in progress

SideDoorOpen := NOT I_Side_Door_Closed AND NOT I_Prot_ByPass;

JogRun := I_Jog_PB AND NOT Slow_Inhibit AND NOT SoftStop AND NOT
SideDoorOpen;

GuardStopOK := NOT StartMotor_Cmd;

// START refused: no paper (monitoring ON), front gate or side door open (no key
bypass)
// JOG: allowed with the front gate open (guard bypass), NOT with the side door
open

WebBreakNow := I_WebBreak_Monitor AND I_WebBreak_PH;
GuardOpen   := (NOT I_FrontGuard_Closed OR NOT I_Side_Door_Closed) AND NOT
I_Prot_ByPass;
StartBlocked := WebBreakNow OR GuardOpen;

```

8

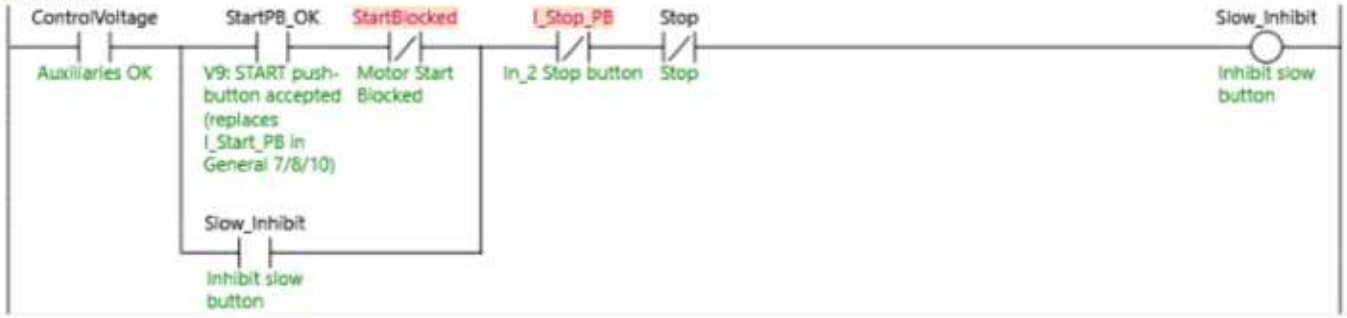
```
T_TensOn(In := I_T_one_TensionOK, PT := T#500ms);      // stable 0.5 s
T_TensOff(In := NOT I_T_one_TensionOK, PT := T#2s);    // ignore short dips

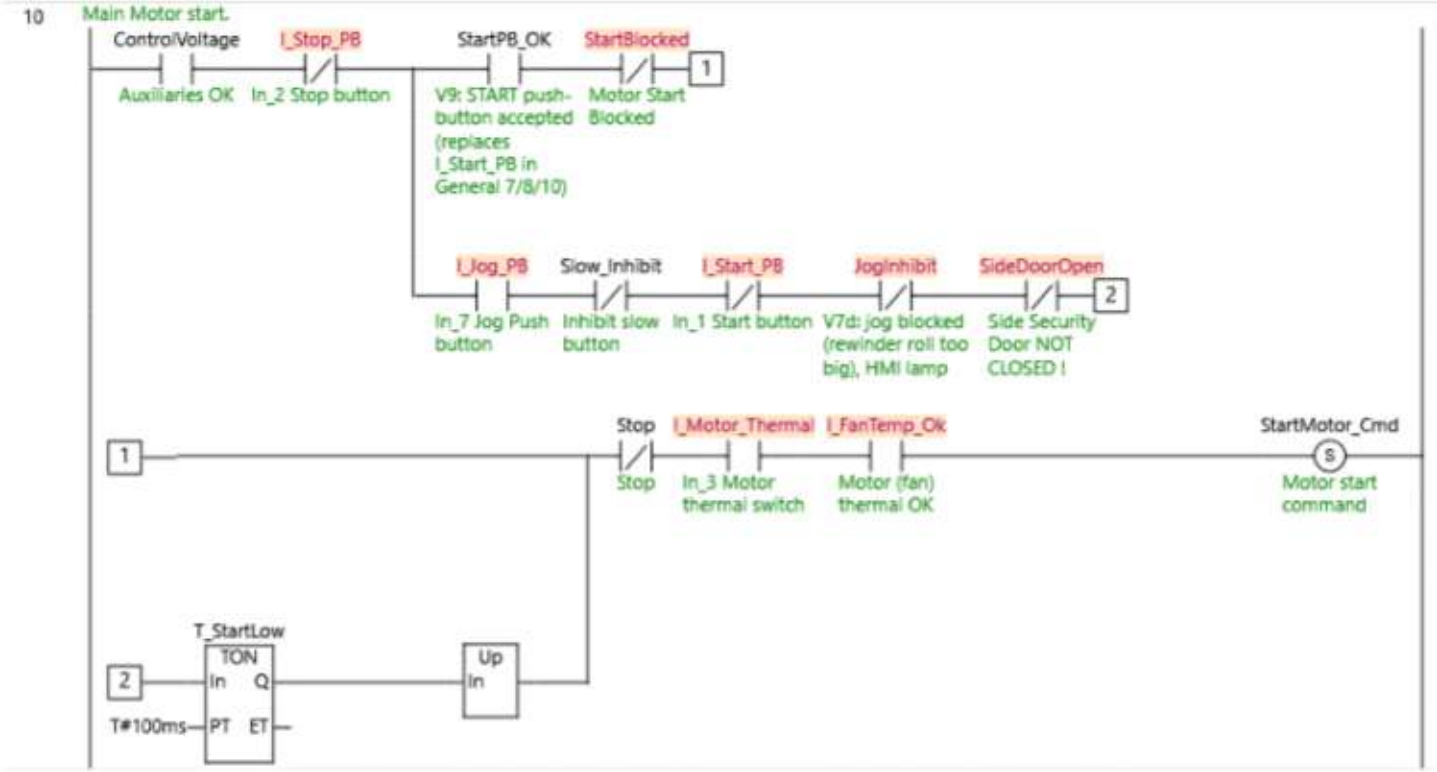
IF T_TensOn.Q OR NOT TensCheckEnable THEN
    TensionOK := TRUE;
ELSIF T_TensOff.Q THEN
    TensionOK := FALSE;
END_IF;

IF NOT StartMotor_Cmd THEN
    TensionSeen := FALSE;
ELSIF TensionOK THEN
    TensionSeen := TRUE;
END_IF;
TensWait := StartMotor_Cmd AND NOT TensionSeen AND NOT JogRun;
TensLostWarn := StartMotor_Cmd AND TensionSeen AND NOT TensionOK;

```

9 Slow Inhibit




10
Main Motor start.
ControlVoltage
Auxiliaries OK
I_Stop_PB
In_2 Stop button
StartPB_OK
V9: START push-button accepted (replaces I_Start_PB in General 7/8/10)
StartBlocked
Motor Start Blocked
1
I_Jog_PB
In_7 Jog Push button
Slow_Inhibit
Inhibit slow button
I_Start_PB
In_1 Start button
JogInhibit
V7d: jog blocked (rewinder roll too big), HMI lamp
SideDoorOpen
Side Security Door NOT CLOSED !
2
1
Stop
Stop
I_Motor_Thermal
In_3 Motor thermal switch
I_FanTemp_Ok
Motor (fan) thermal OK
StartMotor_Cmd
S
Motor start command
T_StartLow
TON
2
In
Q
T#100ms
PT
ET
Up
In

11 Soft Stop Process

```
// ==== V12: REPLACE the code of General rung 10 ("Soft Stop Process", printed V9).
// v6m: STOP button = soft stop. StopNow replaces I_Stop_PB in rung 9.
// V12: an end-of-roll stop (FinalStopRamp, AutoStop section 11b) also starts the
// soft stop, at FinalStopDecel (Motor_Manager rung 17): START cannot cancel it.

StopNow := FALSE;
T_StopLong(In := I_Stop_PB, PT := T#1s);

IF NOT StartMotor_Cmd THEN
    SoftStop := FALSE;
ELSIF I_Stop_PB AND NOT StopPB_Mem THEN          // STOP pressed (edge)

    IF SoftStop OR NOT SoftStopEnable OR I_Jog_PB
      OR (Velocity_M_min <= SoftStopEndSpd) THEN
        StopNow := TRUE;                         // 2nd press / jog / slow / disabled
    ELSE
        SoftStop := TRUE;                        // start soft stop
    END_IF;
END_IF;

StopPB_Mem := I_Stop_PB;

IF T_StopLong.Q THEN
    StopNow := TRUE;                             // held 1 s = immediate
END_IF;

// V12: end-of-roll stop ramp (FinalStopRamp set by AutoStop section 11b)

IF FinalStopRamp AND StartMotor_Cmd AND NOT SoftStop THEN
    SoftStop := TRUE;
END_IF;

// Side door opened while running (not JOG): soft stop. START cannot cancel it

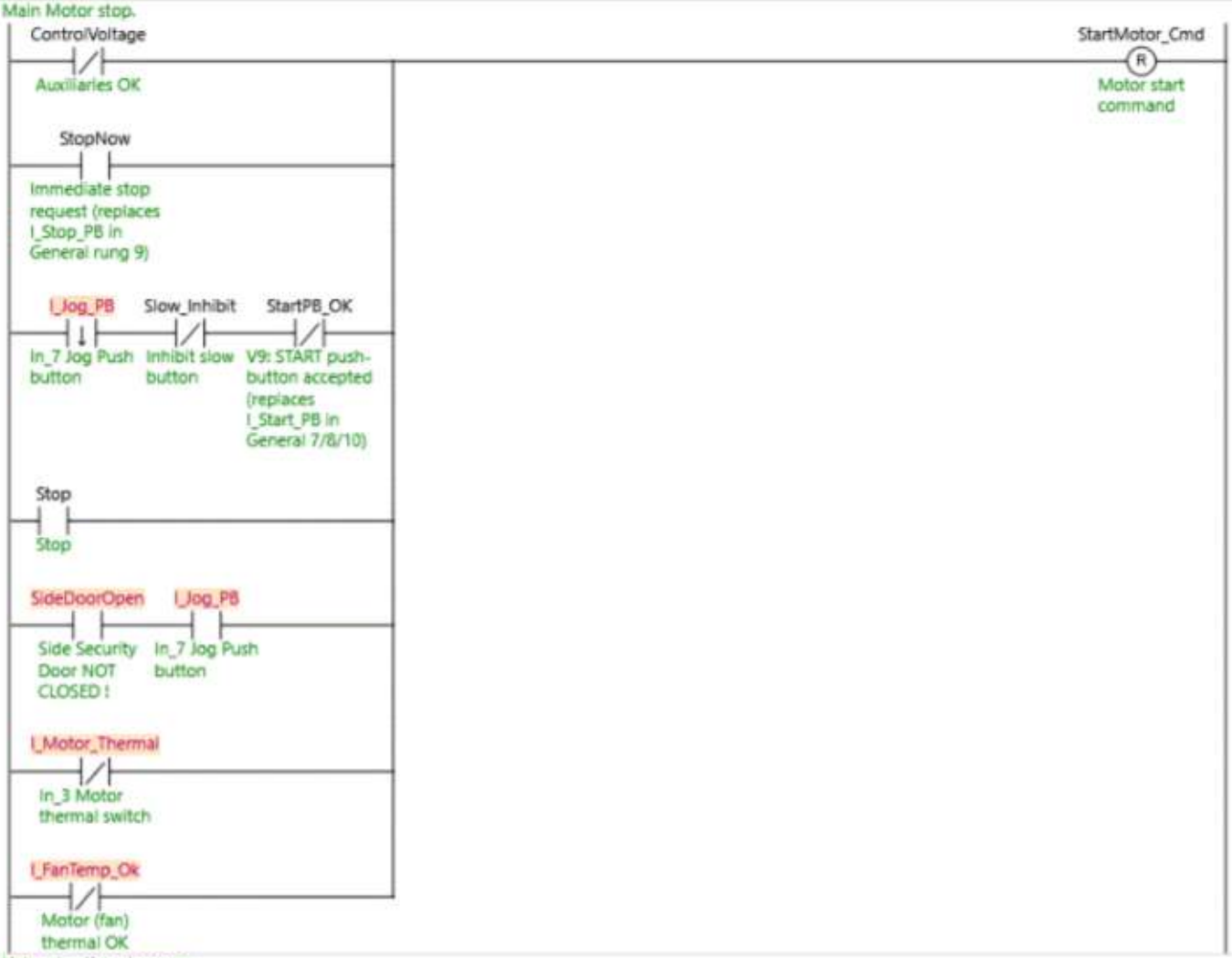
IF SideDoorOpen AND StartMotor_Cmd AND NOT I_Jog_PB AND NOT SoftStop THEN
    SoftStop := TRUE;
END_IF;

IF SoftStop AND I_Start_PB AND NOT FinalStopRamp AND NOT SideDoorOpen THEN
    SoftStop := FALSE;                           // START cancels the soft stop
END_IF;

T_SoftStop(In := SoftStop, PT := T#120s);

IF SoftStop AND ((Velocity_M_min <= SoftStopEndSpd) OR T_SoftStop.Q) THEN
    StopNow := TRUE;                             // almost stopped (or timeout)
END_IF;
```

12 Main Motor stop.



13 Hub extraction clearance.



14 Brake control.

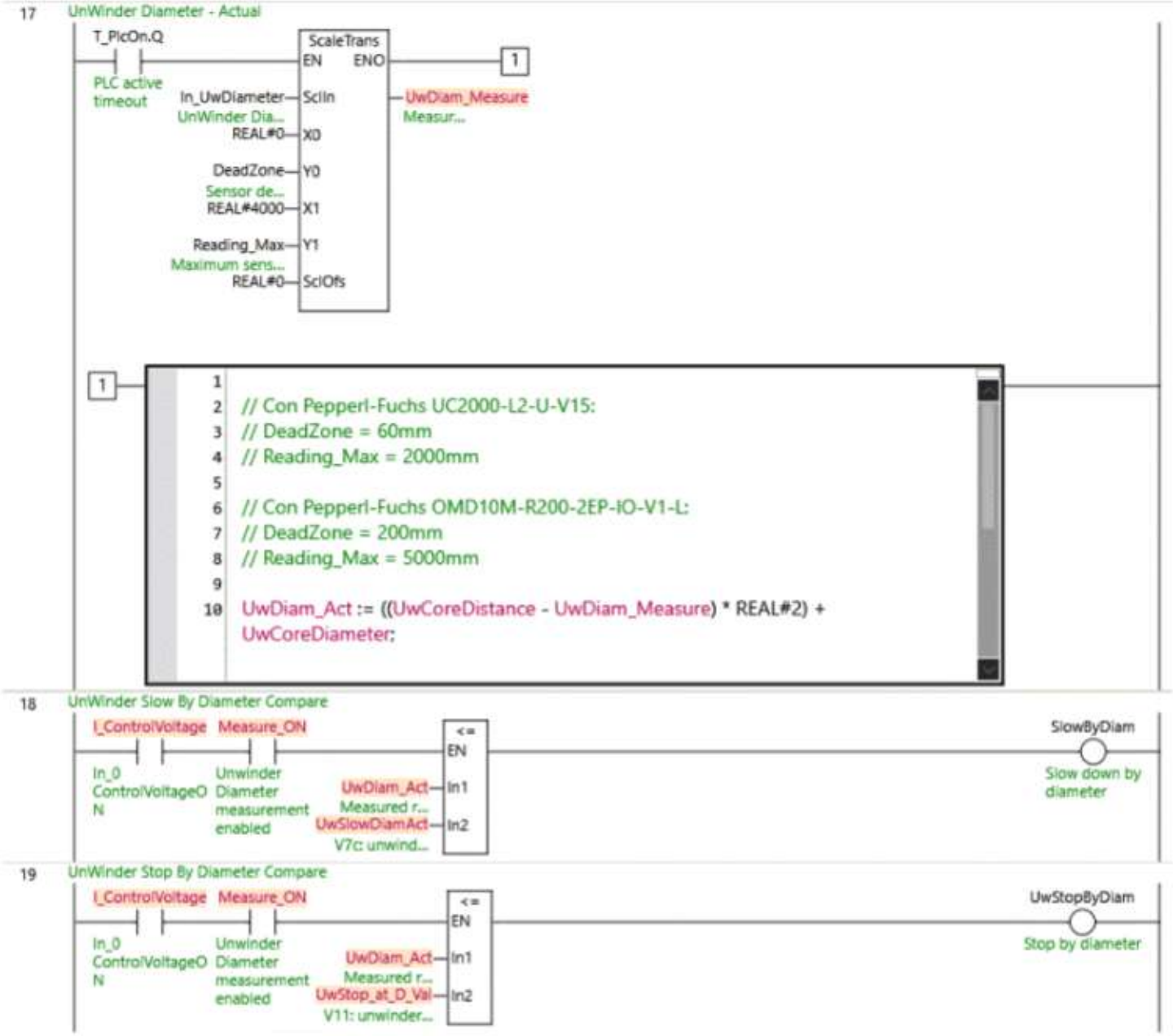


15 Cooling Fan control.



16 UnWinder Diameter - Input

```
1  In_UwDiameter := INT_TO_REAL(AI_Diam_Unwinder)/REAL#1;
```


17
UnWinder Diameter - Actual
T_PlcOn.Q
PLC active timeout
ScaleTrans
EN
ENO
In_UwDiameter
UnWinder Dia...
REAL#0
SclIn
X0
DeadZone
Sensor de...
REAL#4000
Y0
X1
Reading_Max
Maximum sens...
REAL#0
Y1
SclOfs
UwDiam_Measure
Measur...
1
// Con Pepperl-Fuchs UC2000-L2-U-V15:
// DeadZone = 60mm
// Reading_Max = 2000mm
// Con Pepperl-Fuchs OMD10M-R200-2EP-IO-V1-L:
// DeadZone = 200mm
// Reading_Max = 5000mm
UwDiam_Act := ((UwCoreDistance - UwDiam_Measure) * REAL#2) + UwCoreDiameter;
18
UnWinder Slow By Diameter Compare
I_ControlVoltage
In_0 ControlVoltageON
Measure_ON
Unwinder Diameter measurement enabled
<=
EN
UwDiam_Act
Measured r...
In1
UwSlowDiamAct
V7c: unwind...
In2
SlowByDiam
Slow down by diameter
19
UnWinder Stop By Diameter Compare
I_ControlVoltage
In_0 ControlVoltageON
Measure_ON
Unwinder Diameter measurement enabled
<=
EN
UwDiam_Act
Measured r...
In1
UwStop_at_D_Val
V11: unwinder...
In2
UwStopByDiam
Stop by diameter

1-8-1-1-3.Motor_Manager



1 Calculation of mm per encoder pulse.

```
IF Encoder_Pulse = INT#0 THEN
    Encoder_Pulse := INT#1;
END_IF;

// Encoder Standard 400mm

IF EncoderWheel_Circumf = REAL#0 THEN
    EncoderWheel_Circumf := REAL#400;
END_IF;

EncoderPulse_to_mm := EncoderWheel_Circumf / (INT_TO_REAL(Encoder_Pulse) *
REAL#4);

// ---- Metre counter auto reset at the next START
// CntResetMode (one bit per mode, HMI bit lamps .0 / .1 / .2):
//   1 = with new roll (new parent roll on the unwinder)
//   2 = after auto-stop
//   4 = manual only (eraser)
// The event arms CntResetPend: the reset happens with the next accepted START;
// the old length is kept in LastRoll_m.
// While CntResetPend: the metre AUTO STOP / slow-down / refusal ignore the old
metres.

IF (CntResetMode <> 1) AND (CntResetMode <> 2) AND (CntResetMode <> 4)
THEN
    CntResetMode := 4;                          // anything else -> manual
END_IF;

// unwinder roll was empty (at the stop diameter) since the last reset

IF Measure_ON AND UnWinder_Empty THEN
    CntUwWasEmpty := TRUE;
END_IF;

// new parent roll: stopped, sonar ON, roll no longer empty for 3 s, after an empty
roll

T_CntNewRoll(In := NOT StartMotor_Cmd AND Measure_ON AND CntUwWasEmpty
                AND NOT UnWinder_Empty AND (UwDiam_Act > 0.0), PT := T#3s);

// arm the reset (machine stopped)

IF NOT StartMotor_Cmd
   AND (((CntResetMode = 1) AND T_CntNewRoll.Q) OR ((CntResetMode = 2) AND
AutoStopDone)) THEN
    CntResetPend := TRUE;
    CntUwWasEmpty := FALSE;
END_IF;

IF CntResetMode = 4 THEN
    CntResetPend := FALSE;                      // manual: nothing automatic
END_IF;

// manual reset (HMI eraser) while a reset is pending: keep the length, nothing more
at START

IF CntResetPend AND Counter_Reset AND NOT StartMotor_Cmd THEN
    LastRoll_m := Current_Metre;
```

```
    CntResetPend := FALSE;
END_IF;

// START accepted with a reset pending -> keep the length, 50 ms reset pulse

CntPulseReq := CntResetPend AND StartPB_OK AND NOT CntStartMem AND NOT
StartBlocked;
CntStartMem := StartPB_OK;

IF CntPulseReq THEN
    LastRoll_m := Current_Metre;
    CntPulseDone := TRUE;
END_IF;

T_CntResetPulse(In := CntPulseReq, PT := T#50ms);   // TP
CntAutoReset := T_CntResetPulse.Q;

// pending cleared when the counter is back near 0 (or 2 s after the pulse, safety)

T_CntDone(In := CntPulseDone AND NOT T_CntResetPulse.Q, PT := T#2s);

IF CntPulseDone AND NOT T_CntResetPulse.Q
   AND ((Current_Metre < REAL#1.0) OR T_CntDone.Q) THEN
    CntResetPend := FALSE;
    CntPulseDone := FALSE;
END_IF;
```

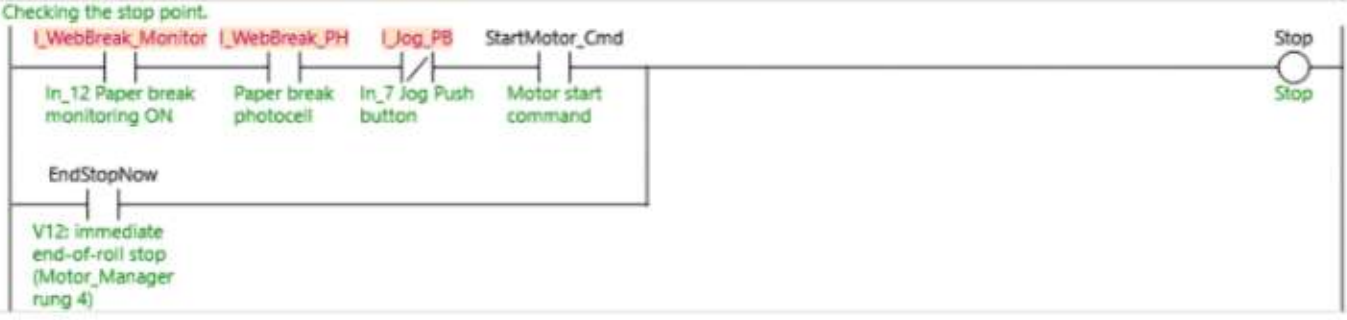
2 Reset encoder counter.



3 Checking the slowdown point.



4 Checking the stop point.



5 Check the maximum and minimum values for working speed and slow speed.

```
IF VelMax < REAL#600 THEN
    VelMax := REAL#600;
ELSIF VelMax > REAL#1000 THEN
    VelMax := REAL#1000;
END_IF;

Vel_Max_mms := VelMax * REAL#1000 / REAL#60;

IF JogSpd_SP < REAL#10 THEN
    JogSpd_SP := REAL#10;
ELSIF JogSpd_SP > REAL#70 THEN
    JogSpd_SP := REAL#70;
END_IF;

// v6y: rewinder AUTO max speed, 0 = no limit

IF RwAutoMaxSpd > VelMax THEN
    RwAutoMaxSpd := VelMax;
ELSIF (RwAutoMaxSpd > REAL#0) AND (RwAutoMaxSpd < REAL#50) THEN
    RwAutoMaxSpd := REAL#50;
END_IF;

IF NOT StartMotor_Cmd THEN        // taken over at standstill only
    RwAutoLimAct := RwStopEnable AND (RwAutoMaxSpd > REAL#0);
    RwAutoMaxSpdAct := RwAutoMaxSpd;
END_IF;

// V7: limits on settings that stop the machine or break the auto stop if 0 / absurd

IF SpdDelta_Percent < REAL#2 THEN
    SpdDelta_Percent := REAL#2;
ELSIF SpdDelta_Percent > REAL#15 THEN
    SpdDelta_Percent := REAL#15;
END_IF;

IF AccelTrim < REAL#0.02 THEN
    AccelTrim := REAL#0.02;
ELSIF AccelTrim > REAL#0.5 THEN
    AccelTrim := REAL#0.5;
END_IF;

IF DecelTrim < REAL#0.02 THEN
    DecelTrim := REAL#0.02;
ELSIF DecelTrim > REAL#0.5 THEN
    DecelTrim := REAL#0.5;
END_IF;

IF AccelCoarse < REAL#0.2 THEN
    AccelCoarse := REAL#0.2;
ELSIF AccelCoarse > REAL#4 THEN
    AccelCoarse := REAL#4;
END_IF;

IF DecelCoarse < REAL#0.2 THEN
    DecelCoarse := REAL#0.2;
ELSIF DecelCoarse > REAL#4 THEN
    DecelCoarse := REAL#4;
END_IF;
```

```
59
60  IF DecelWeb > REAL#0 THEN
61
62      IF DecelWeb < REAL#50 THEN
63          DecelWeb := REAL#50;
64      ELSIF DecelWeb > REAL#1000 THEN
65          DecelWeb := REAL#1000;
66      END_IF;
67  END_IF;
68
69  IF AccelWeb > REAL#0 THEN
70
71      IF AccelWeb < REAL#50 THEN
72          AccelWeb := REAL#50;
73      ELSIF AccelWeb > REAL#1000 THEN
74          AccelWeb := REAL#1000;
75      END_IF;
76  END_IF;
77
78  IF SoftStopDecel > REAL#1500 THEN
79      SoftStopDecel := REAL#1500;
80  END_IF;
81
82  IF RwMaxDiam > REAL#1450 THEN
83      RwMaxDiam := REAL#1450;
84  ELSIF RwMaxDiam < REAL#300 THEN
85      RwMaxDiam := REAL#300;
86  END_IF;
87
88  IF RwMaxSlowFallback >= RwMaxDiam THEN
89      RwMaxSlowFallback := RwMaxDiam - REAL#30;
90  END_IF;
91
92  // V7d: jog blocked on a big rewinder roll (0 = no limit, else min 150): DT35 fault ->
    jog allowed
93
94  IF (RwJogMaxDiam > REAL#0) AND (RwJogMaxDiam < REAL#150) THEN
95      RwJogMaxDiam := REAL#150;
96  END_IF;
97
98  JogInhibit := (RwJogMaxDiam > REAL#0) AND (RwDiam > RwJogMaxDiam) AND
    NOT RwSensFault;
99  Output_Ratio := (VelMax * Mt_To_mm) / UINT_TO_REAL(Analog_Res): //REAL#4000;
100 Input_Ratio := UINT_TO_REAL(Analog_Res) / VelMax ;
```

6 Reading the current value of the potentiometers. v6n: requested pot ranges, limited, applied only with the motor stopped !

```
IF WorkSpd_Max_Set < 50.0 THEN
    WorkSpd_Max_Set := 50.0;
ELSIF WorkSpd_Max_Set > VelMax THEN
    WorkSpd_Max_Set := VelMax;
END_IF;

IF FinalSpd_Max_Set < 5.0 THEN
    FinalSpd_Max_Set := 5.0;
ELSIF FinalSpd_Max_Set > WorkSpd_Max_Set THEN
    FinalSpd_Max_Set := WorkSpd_Max_Set;
END_IF;

IF NOT StartMotor_Cmd THEN
    WorkSpd_Max := WorkSpd_Max_Set;        // motor stopped: apply
    FinalSpd_Max := FinalSpd_Max_Set;
END_IF;

SpdMax_Pending := (WorkSpd_Max <> WorkSpd_Max_Set)
            OR (FinalSpd_Max <> FinalSpd_Max_Set)
            OR (RwAutoLimAct <> (RwStopEnable AND (RwAutoMaxSpd > REAL#0)))
            OR (RwAutoLimAct AND (RwAutoMaxSpdAct <> RwAutoMaxSpd));

// v6i: pots scaled to their own full scale (m/min)

IF Pot1_Work_Velocity > INT#5 THEN
    Pot1_Mem := REAL_TO_INT(INT_TO_REAL(Pot1_Work_Velocity) * WorkSpd_Max /
VelMax);
ELSE
    Pot1_Mem := INT#0;
END_IF;

IF Pot2_Slow_Velocity > INT#5 THEN
    Pot2_Mem := REAL_TO_INT(INT_TO_REAL(Pot2_Slow_Velocity) * FinalSpd_Max /
VelMax);
ELSE
    Pot2_Mem := INT#0;
END_IF;
```

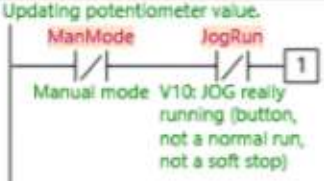
7 Updating the set speed.

```
IF ManMode = FALSE THEN
    Speed_Display := UINT_TO_REAL(Pot1_Mem) / Input_Ratio;
    LowSpd_Display := UINT_TO_REAL(Pot2_Mem) / Input_Ratio;

    // v6y: rewinder AUTO stop -> work speed limited

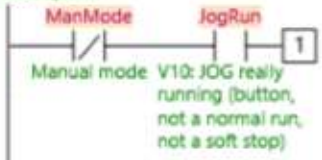
    IF RwAutoLimAct AND (Speed_Display > RwAutoMaxSpdAct) THEN
        Speed_Display := RwAutoMaxSpdAct;
    END_IF;
ELSE
    Speed_Display := Pot1_Work_Velocity / (Analog_Res / UINT#40); // v6i: raw pot. %
    LowSpd_Display := Pot2_Slow_Velocity / (Analog_Res / UINT#40);
END_IF;
```

8 Updating potentiometer value.




```
1  IF SoftStop THEN
2      VelSet := 0.0;                    // v6m: soft stop
3  ELSIF SlowDown = FALSE  THEN
4      VelSet := UINT_TO_REAL(Pot1_Mem) / Input_Ratio;
5
6      // v6y: rewinder AUTO stop -> work speed limited
7
8      IF RwAutoLimAct AND (VelSet > RwAutoMaxSpdAct) THEN
9          VelSet := RwAutoMaxSpdAct;
10     END_IF;
11
12 ELSE
13     VelSet := UINT_TO_REAL(Pot2_Mem) / Input_Ratio;
14 END_IF;
15
16 VelSet_mms := VelSet * Mt_To_mm;
```

9 Slow speed.




```
1  VelSet := JogSpd_SP;
2  VelSet_mms := VelSet * Mt_To_mm;
```

10 Check the difference between the set speed and the detected speed.

```
//SpdDelta := VelSet_mms * SpdDelta_Percent / REAL#100;

SpdDelta := SpdDelta_Percent;
SpdDeltaMax := VelSet + SpdDelta;
SpdDeltaMin := VelSet - SpdDelta;

// Too fast

IF Velocity_M_min > SpdDeltaMax THEN
    OverSpeed := TRUE;
ELSE
    OverSpeed := FALSE;
END_IF;

// Too slow

IF Velocity_M_min < SpdDeltaMin THEN
    UnderSpeed := TRUE;
ELSE
    UnderSpeed := FALSE;
END_IF;
```

11 Speed calculation.

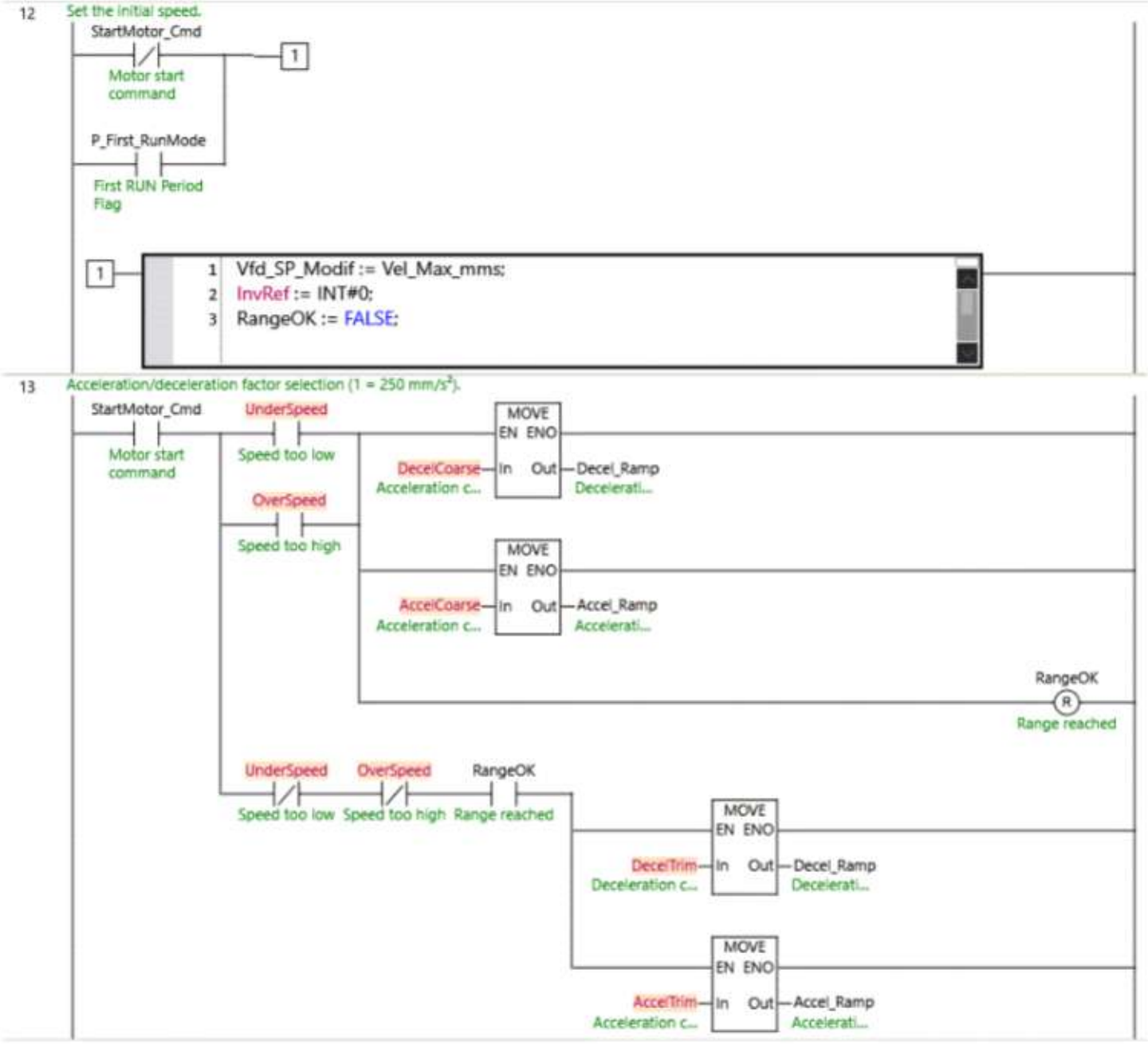
```
// Exactly 25 scans = 100 ms (same as AutoStop SpdAct_Pps), updated 10 times/s.

SpdScanCnt := SpdScanCnt + 1;

IF SpdScanCnt >= 25 THEN
    SpdScanCnt := 0;
    DeltaPulse := Encoder_Pos - MemPos;
    MemPos := Encoder_Pos;

    IF ABS(DeltaPulse) > 100000 THEN            // counter reset (Encoder_Reset)
        DeltaPulse := 0;
    END_IF;

    Velocity := DINT_TO_REAL(DeltaPulse) * EncoderPulse_to_mm * REAL#10;   // mm/s
    Velocity_M_min := Velocity / Mt_To_mm;
END_IF;
```


12
Set the initial speed.
StartMotor_Cmd
Motor start command
1
P_First_RunMode
First RUN Period Flag
1
1 Vfd_SP_Modif := Vel_Max_mms;
2 InvRef := INT#0;
3 RangeOK := FALSE;
13
Acceleration/deceleration factor selection (1 = 250 mm/s²).
StartMotor_Cmd
Motor start command
UnderSpeed
Speed too low
OverSpeed
Speed too high
MOVE
EN ENO
DecelCoarse
Acceleration c...
In Out
Decel_Ramp
Decelerati...
MOVE
EN ENO
AccelCoarse
Acceleration c...
In Out
Accel_Ramp
Accelerati...
RangeOK
R
Range reached
UnderSpeed
Speed too low
OverSpeed
Speed too high
RangeOK
Range reached
MOVE
EN ENO
DecelTrim
Deceleration c...
In Out
Decel_Ramp
Decelerati...
MOVE
EN ENO
AccelTrim
Acceleration c...
In Out
Accel_Ramp
Accelerati...

14 Ratio drive counts per m/min of web at the current rewinder diameter.

```
// ==== V10: REPLACE the code of Motor_Manager rung 14 (printed V9) ("Ratio drive counts per m/min of web
// ==== at the current rewinder diameter." = V7 rung A) with this file. (V7b jog line inside.)
// Ratio drive counts per m/min of web at the current rewinder diameter.
// The motor drives the rewinder: web speed per Hz grows with the diameter, so
// counts per m/min = RwRatioD / diameter. RwRatioD (Retain) is adapted by rung B.

IF (RwRatioD > 0.0) AND (DecelDiam >= 100.0) THEN
    WebRatio := RwRatioD / DecelDiam;
ELSE
    WebRatio := 0.0;
END_IF;

// Fallback only (FFEnable = FALSE): old integrating loop (rung 16) with the coarse
// ramps in real web mm/s2. With FFEnable = TRUE, rung B drives the reference.
// V7b: never in jog -> jog keeps the rung 13 ramps exactly as v6y. V10: JogRun (jog really running).

IF NOT FFEnable AND NOT JogRun AND StartMotor_Cmd AND (UnderSpeed OR OverSpeed)
   AND (WebRatio > 0.0) THEN

    IF DecelWeb > 0.0 THEN
        Decel_Ramp := DecelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;

        IF Decel_Ramp > 8.0 THEN
            Decel_Ramp := 8.0;
        END_IF;
    END_IF;

    IF AccelWeb > 0.0 THEN
        Accel_Ramp := AccelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;

        IF Accel_Ramp > 8.0 THEN
            Accel_Ramp := 8.0;
        END_IF;
    END_IF;
END_IF;
```

15 Soft Stop - SoftStopDecel (mm/s2), same for every roll.

```
IF SoftStop AND NOT SoftStopMem THEN

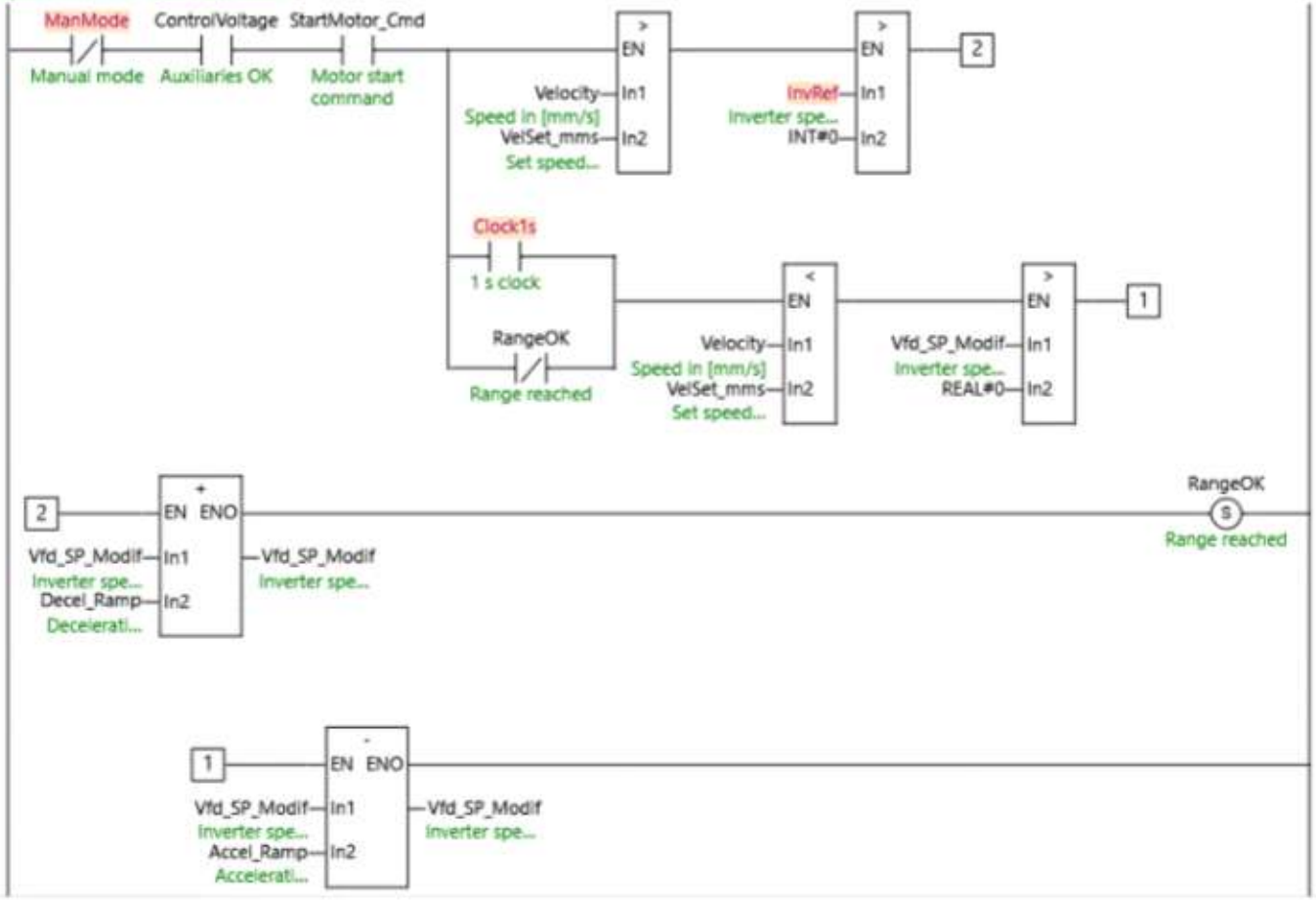
   IF Velocity_M_min > 1.0 THEN
      SoftStopRatio := INT_TO_REAL(InvRef) / Velocity_M_min;
   ELSE
      SoftStopRatio := 0.0;
   END_IF;
END_IF;
SoftStopMem := SoftStop;

IF SoftStop AND (SoftStopDecel > 0.0) THEN

   IF SoftStopRatio > 0.0 THEN
      Decel_Ramp := SoftStopDecel * 0.06 * SoftStopRatio * Vel_Max_mms /
1000000.0;
   ELSE
      Decel_Ramp := SoftStopDecel / 250.0;    // fallback (old v6m behaviour)
   END_IF;

   IF Decel_Ramp > 8.0 THEN
      Decel_Ramp := 8.0;                  // max 9 Hz/s asked from the PLC
   END_IF;
END_IF;
```

16 Speed correction factor calculation.



17 Slow adaptation of RwRatioD = feed-forward (ST, V12 rung B + tension hold)

```
// ==== V12: REPLACE the code of Motor_Manager rung 17 (printed V9) ("Slow adaptation of RwRatioD on the
// ==== relative speed error." = V7 rung B) with this file. (V7a + V7b + V7f + V9 + V10 + V12 inside.)
// The PLC sends the drive STRAIGHT to the frequency of the set speed
// (VelSet x WebRatio), along a ramp in REAL web mm/s2 that does not depend on the
// measured speed -> no hunting, whatever the roll inertia / web delay.
// The measured speed only adapts RwRatioD slowly (~10 s), once at target.
// It overwrites Vfd_SP_Modif after rung 16, so rung 16 has no effect in AUTO.
// FFEnable = FALSE -> old loop (rung 16) exactly as before.
// V7b: not in jog (V10: JogRun = jog really running): at 10 m/min the feed-forward gives
// only ~0.4 Hz, too little to move the web -> the old loop (rung 16) runs the jog.

FFActive := FFEnable AND NOT ManMode AND NOT JogRun AND StartMotor_Cmd
        AND ControlVoltage AND (WebRatio > 0.0) AND (Vel_Max_mms > 0.0);

IF NOT StartMotor_Cmd THEN
   RampRef := 0.0;
   FFRate := 0.0;
   FFSteadyTicks := 0;
   FFSlowAcc := FALSE;
END_IF;

// V9: START during a soft stop: the decel flattens and the speed is HELD
// (SpdHold, HMI lamp) until START is pushed AGAIN -> gentle ramp up at
// SoftCancelAccel (mm/s2) to the set speed. The hold only blocks speeding up:
// a lower target (pot down, auto slow-down, RwAutoMaxSpd) still slows down.
// STOP during the hold = new soft stop (General rung 9).

IF (SoftCancelAccel > 0.0) AND (SoftCancelAccel < 20.0) THEN
   SoftCancelAccel := 20.0;
ELSIF SoftCancelAccel > 1000.0 THEN
   SoftCancelAccel := 1000.0;
END_IF;

FFStartEdge := I_Start_PB AND NOT FFStartMem;
FFStartMem := I_Start_PB;

IF FFSoftMem AND NOT SoftStop AND StartMotor_Cmd AND FFActive THEN   // soft stop cancelled
   SpdHold := TRUE;
   FFSlowAcc := FALSE;
ELSIF SpdHold AND FFStartEdge THEN                    // 2nd START: resume
   SpdHold := FALSE;
   FFSlowAcc := TRUE;
END_IF;

IF SoftStop OR NOT StartMotor_Cmd OR NOT FFActive THEN    // FFEnable OFF: old loop
   SpdHold := FALSE;
   FFSlowAcc := FALSE;
END_IF;

FFSoftMem := SoftStop;

// V7f: S-curve time (s). 0 = off. max 2 s
```

```
53
54  IF SCurveTime < 0.0 THEN
55      SCurveTime := 0.0;
56  ELSIF SCurveTime > 2.0 THEN
57      SCurveTime := 2.0;
58  END_IF;
59
60  IF FFActive THEN
61
62      IF NOT FFWasActive THEN
63          RampRef := Vel_Max_mms - Vfd_SP_Modif;        // bumpless start / switch-
    over
64          FFRate := 0.0;
65      END_IF;
66
67      // Target reference (mm/s of the 0..10 V scale) for the set speed
68
69      FFTarget := VelSet * WebRatio * Output_Ratio;
70
71      IF FFTarget > Vel_Max_mms THEN
72          FFTarget := Vel_Max_mms;
73      ELSIF FFTarget < 0.0 THEN
74          FFTarget := 0.0;
75      END_IF;
76
77      // Tension: hold at a minimum DRIVE FREQUENCY until the T-one reports tension
78      // (independent of the rewinder diameter). TensWaitHz = 0 -> old m/min hold.
79
80      IF TensWait THEN
81
82          IF TensWaitHz > REAL#0.0 THEN
83              TensRef := TensWaitHz * REAL#4000.0 / REAL#60.0 * Output_Ratio;
84          ELSE
85              TensRef := TensWaitSpd * WebRatio * Output_Ratio;
86          END_IF;
87
88          IF FFTarget > TensRef THEN
89              FFTarget := TensRef;
90          END_IF;
91      END_IF;
92
93      // V9: hold after a cancelled soft stop (the S-curve flattens it smoothly):
94      // never above the present reference, lower targets still allowed
95
96      IF SpdHold THEN
97
98          IF FFTarget > RampRef THEN
99              FFTarget := RampRef;
100         END_IF;
101
102     ELSIF FFSlowAcc AND (ABS(FFTarget - RampRef) <= FFTarget * 0.005 + 2.0) THEN
103         FFSlowAcc := FALSE;                    // back at the set speed
104     END_IF;
105
106     // Ramp steps per 4 ms scan: web mm/s2 -> reference mm/s per scan
107
108     IF FFSlowAcc AND (SoftCancelAccel > 0.0) THEN
109         StepAcc := SoftCancelAccel * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;  //
    V9
```

```
110     ELSIF AccelWeb > 0.0 THEN
111         StepAcc := AccelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;
112     ELSE
113         StepAcc := AccelCoarse;
114     END_IF;
115
116     IF SoftStop AND FinalStopRamp AND (FinalStopDecel > 0.0) THEN
117         StepDec := FinalStopDecel * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;   //
    V12 end-of-roll
118     ELSIF SoftStop AND (SoftStopDecel > 0.0) THEN
119         StepDec := SoftStopDecel * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;
120     ELSIF DecelWeb > 0.0 THEN
121         StepDec := DecelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;
122     ELSE
123         StepDec := DecelCoarse;
124     END_IF;
125
126     IF StepAcc > 8.0 THEN
127         StepAcc := 8.0;
128     END_IF;
129
130     IF StepDec > 8.0 THEN
131         StepDec := 8.0;
132     END_IF;
133
134     // Ramp generator
135     // V7f: S-curve. FFRate (reference per scan) changes at most FFJerk per scan,
136     // so the full ramp is reached after SCurveTime (s) and is rounded off the
137     // same way before the target -> the T-one / brake air can follow.
138     // SCurveTime = 0 -> plain linear ramp as V7a.
139
140     IF SCurveTime > 0.0 THEN
141
142         IF (FFRate > 0.0) OR ((FFRate = 0.0) AND (FFTarget > RampRef)) THEN
143             FFJerk := StepAcc * 0.004 / SCurveTime;
144         ELSE
145             FFJerk := StepDec * 0.004 / SCurveTime;
146         END_IF;
147
148         // rate allowed so that it can still fall to 0 at the target
149         // (no MIN(), argument in a REAL variable first: Sysmac data type rules,
150         //  same pattern as SqrtArg / SQRT(SqrtArg) in AutoStop)
151
152         IF FFTarget >= RampRef THEN
153             FFSqrtArg := REAL#2.0 * FFJerk * (FFTarget - RampRef);
154             FFRateSet := SQRT(FFSqrtArg);
155
156             IF FFRateSet > StepAcc THEN
157                 FFRateSet := StepAcc;
158             END_IF;
159         ELSE
160             FFSqrtArg := REAL#2.0 * FFJerk * (RampRef - FFTarget);
161             FFRateSet := SQRT(FFSqrtArg);
162
163             IF FFRateSet > StepDec THEN
164                 FFRateSet := StepDec;
165             END_IF;
166
167             FFRateSet := -FFRateSet;
```

```
        END_IF;

        IF FFRateSet > FFRate + FFJerk THEN
            FFRate := FFRate + FFJerk;
        ELSIF FFRateSet < FFRate - FFJerk THEN
            FFRate := FFRate - FFJerk;
        ELSE
            FFRate := FFRateSet;
        END_IF;

        RampRef := RampRef + FFRate;

        IF (ABS(FFTarget - RampRef) <= 3.0) AND (ABS(FFRate) <= 2.0 * FFJerk) THEN
            RampRef := FFTarget;                    // at target (pot noise)
            FFRate := 0.0;
        END_IF;
    ELSE
        FFRate := 0.0;

        IF RampRef < FFTarget - StepAcc THEN
            RampRef := RampRef + StepAcc;
        ELSIF RampRef > FFTarget + StepDec THEN
            RampRef := RampRef - StepDec;
        ELSE
            RampRef := FFTarget;                    // at target
        END_IF;
    END_IF;

    // V7a: "at target" = within 0.5 % of the target (the pot and RwDiam noise
    // move FFTarget by a few counts every scan -> the counter never reached 500)

    IF (ABS(FFTarget - RampRef) <= FFTarget * 0.005 + 2.0)
       AND NOT SpdHold AND NOT FFSlowAcc AND NOT TensWait THEN          // V9:
not during the hold

        IF FFSteadyTicks < 30000 THEN
            FFSteadyTicks := FFSteadyTicks + 1;
        END_IF;
    ELSE
        FFSteadyTicks := 0;
    END_IF;

    // Slow adaptation of RwRatioD on the relative speed error (2 s after the
    // target is reached, not at the 60 Hz limit, not during soft stop / jog)

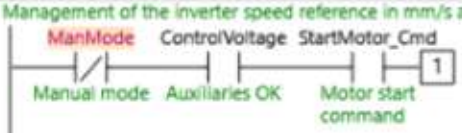
    IF (FFSteadyTicks >= 500) AND NOT SoftStop AND NOT JogRun
      AND (VelSet > 15.0) AND (Velocity_M_min > 10.0) AND NOT RwSensFault
      AND (FFTarget < Vel_Max_mms * 0.98) THEN
        FFErr := (VelSet - Velocity_M_min) / VelSet;

        IF FFErr > 0.05 THEN
            FFErr := 0.05;
        ELSIF FFErr < -0.05 THEN
            FFErr := -0.05;
        END_IF;

        IF ABS(FFErr) > 0.003 THEN                  // dead band 0.3 % (encoder noise)
            RwRatioD := RwRatioD * (1.0 + 0.0004 * FFErr);  // ~10 s time constant
```

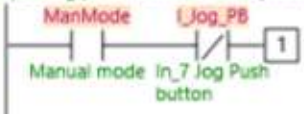
```
225     END_IF;
226
227     IF RwRatioD < 1000.0 THEN
228       RwRatioD := 1000.0;
229     ELSIF RwRatioD > 8000.0 THEN
230       RwRatioD := 8000.0;
231     END_IF;
232   END_IF;
233
234   Vfd_SP_Modif := Vel_Max_mms - RampRef;          // rung 17 -> InvRef
235 END_IF;
236
237 FFWasActive := FFActive;
238
239 // V7f: T-one decel signal (for a T-one "stop / boost" input, spare PLC output):
240 // ON while the feed-forward ramps down (more than 1 % above the target, so the
241 // slow drift of the target with the diameter does not pulse it) or soft stop.
242
243 TOneDecel := (FFActive AND (RampRef - FFTarget > FFTarget * 0.01 + 5.0))
244         OR (SoftStop AND StartMotor_Cmd);
245
```

18 Management of the inverter speed reference in mm/s and conversion to analog points (0–4000 points = 0–10 VDC = 0–10,000 mm/s).



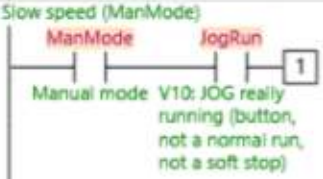

```
1
2 InvRef := REAL_TO_INT((Vel_Max_mms - Vfd_SP_Modif ) / Output_Ratio);
```

19 Updating potentiometer value (ManMode)



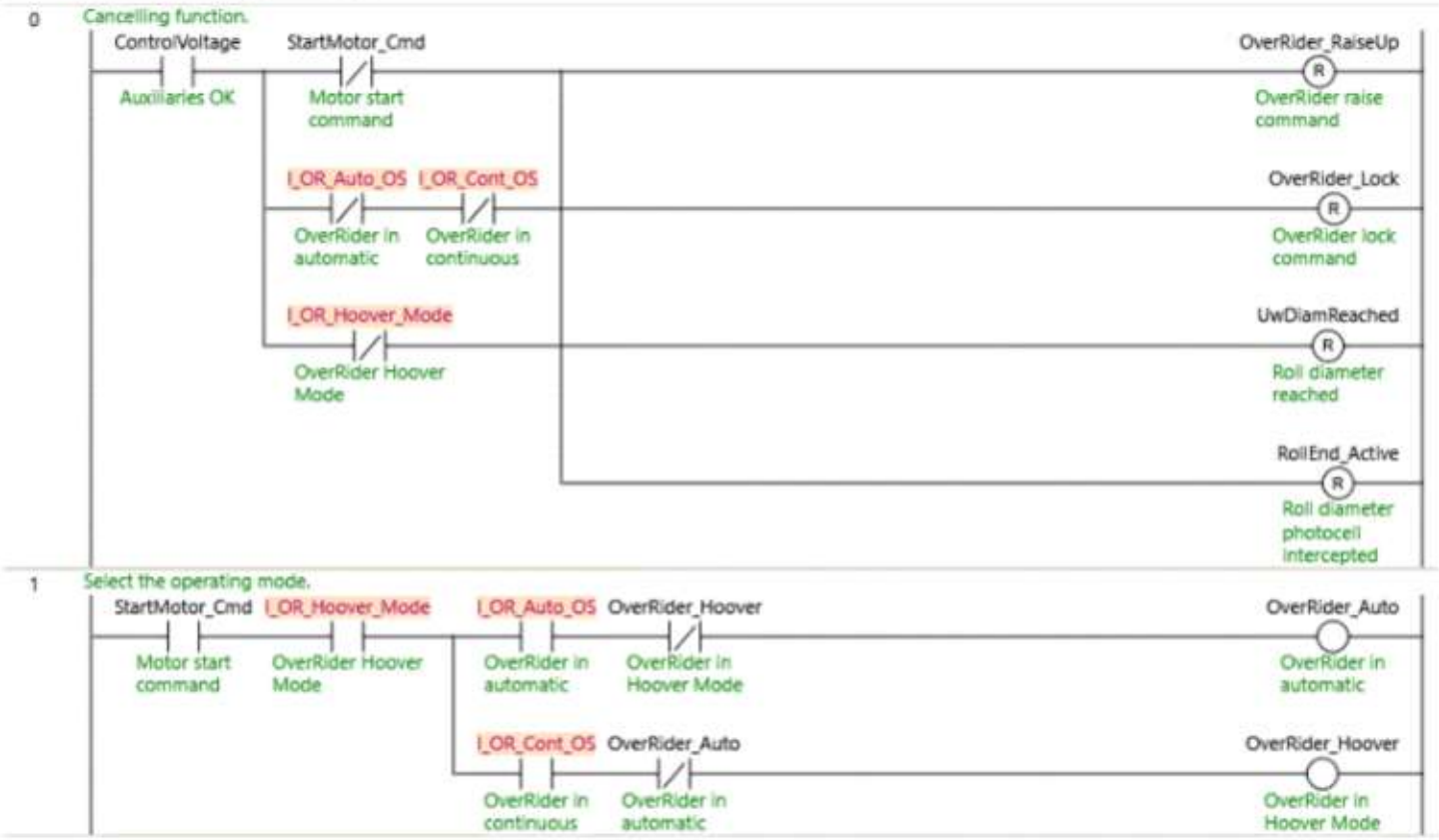

```
1 IF SoftStop THEN
2   InvRef := INT#0;                    // v6m: drive ramps down on its own
3 ELSIF SlowDown = FALSE THEN
4   InvRef := Pot1_Work_Velocity;
5 ELSE
6   InvRef := Pot2_Slow_Velocity;
7 END_IF;
```

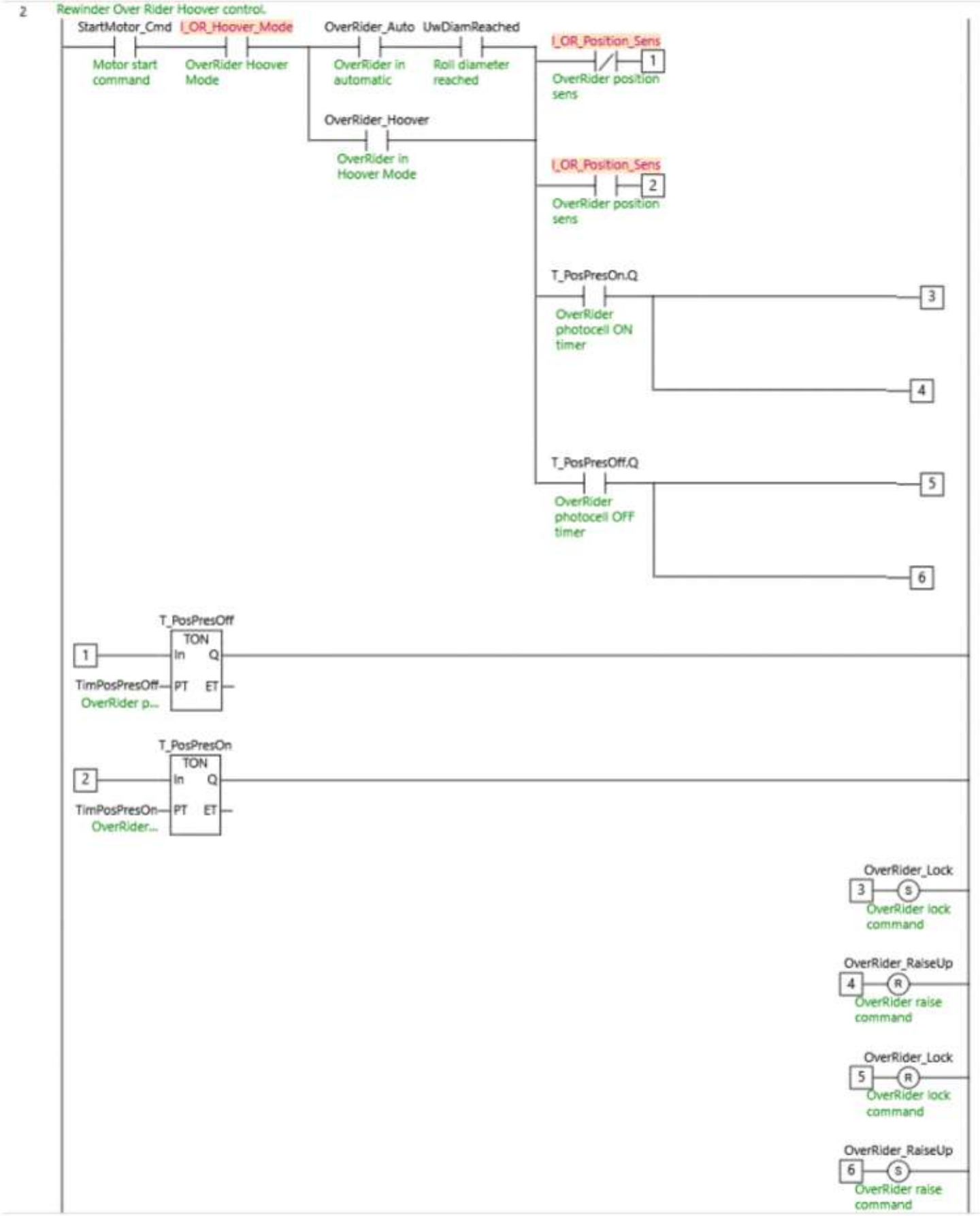
20 Slow speed (ManMode)




```
1 VelSet := JogSpd_SP;
2 VelSet_mms := VelSet * Mt_To_mm;
3
4 InvRef := REAL_TO_INT(VelSet_mms / Output_Ratio);
```

1-8-1-1-4.OverRider_Manager




2
Rewinder Over Rider Hoover control.
StartMotor_Cmd
Motor start command
I_OR_Hoover_Mode
OverRider Hoover Mode
OverRider_Auto
OverRider in automatic
UwDiamReached
Roll diameter reached
I_OR_Position_Sens
1
OverRider position sens
OverRider_Hoover
OverRider in Hoover Mode
I_OR_Position_Sens
2
OverRider position sens
T_PosPresOn.Q
OverRider photocell ON timer
3
4
T_PosPresOff.Q
OverRider photocell OFF timer
5
6
T_PosPresOff
TON
In
Q
PT
ET
TimPosPresOff
OverRider p...
T_PosPresOn
TON
In
Q
PT
ET
TimPosPresOn
OverRider...
OverRider_Lock
3
S
OverRider lock command
OverRider_RaiseUp
4
R
OverRider raise command
OverRider_Lock
5
R
OverRider lock command
OverRider_RaiseUp
6
S
OverRider raise command

3 Rider roll raise command.

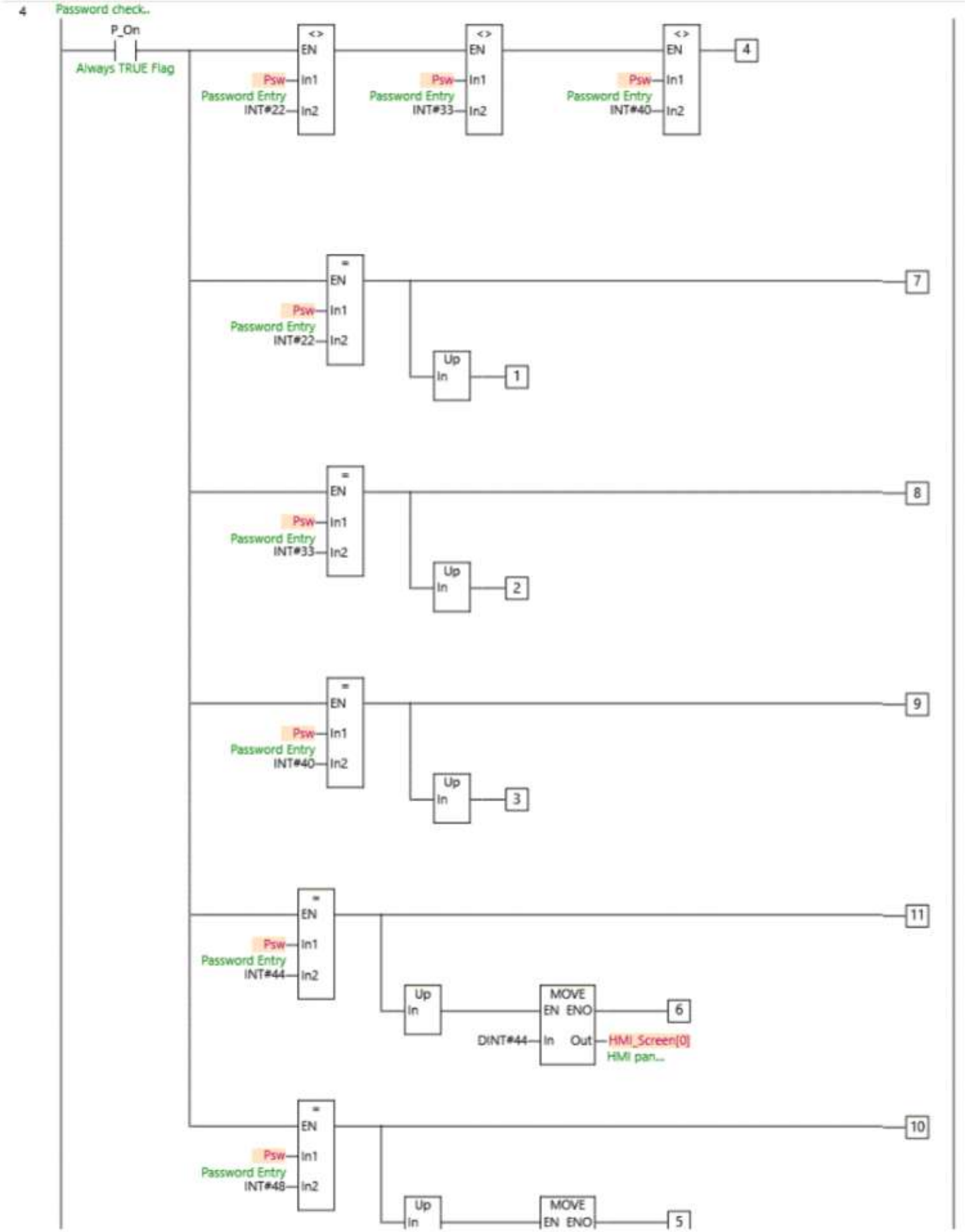


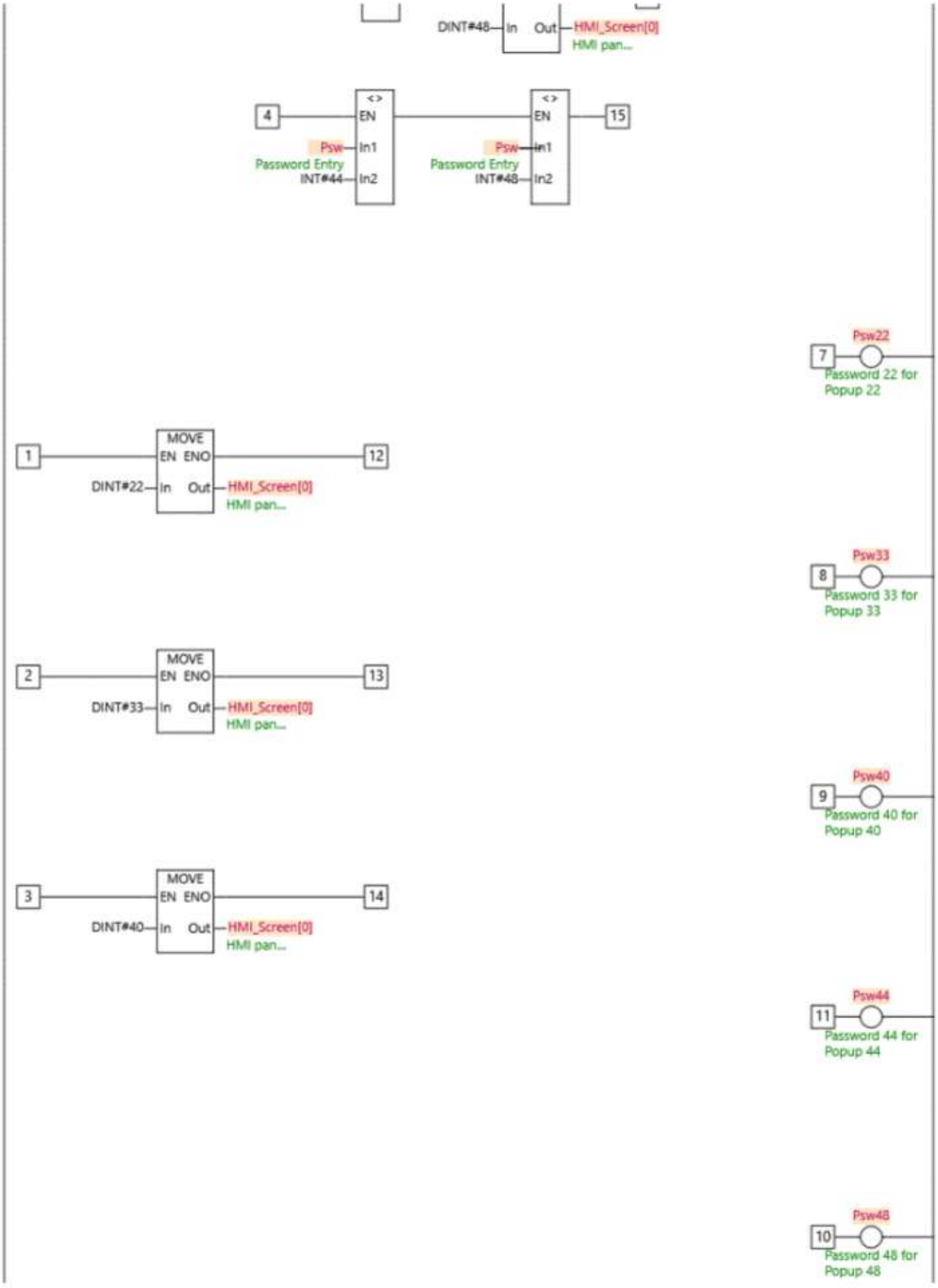
4 Rider roll raise command Reset

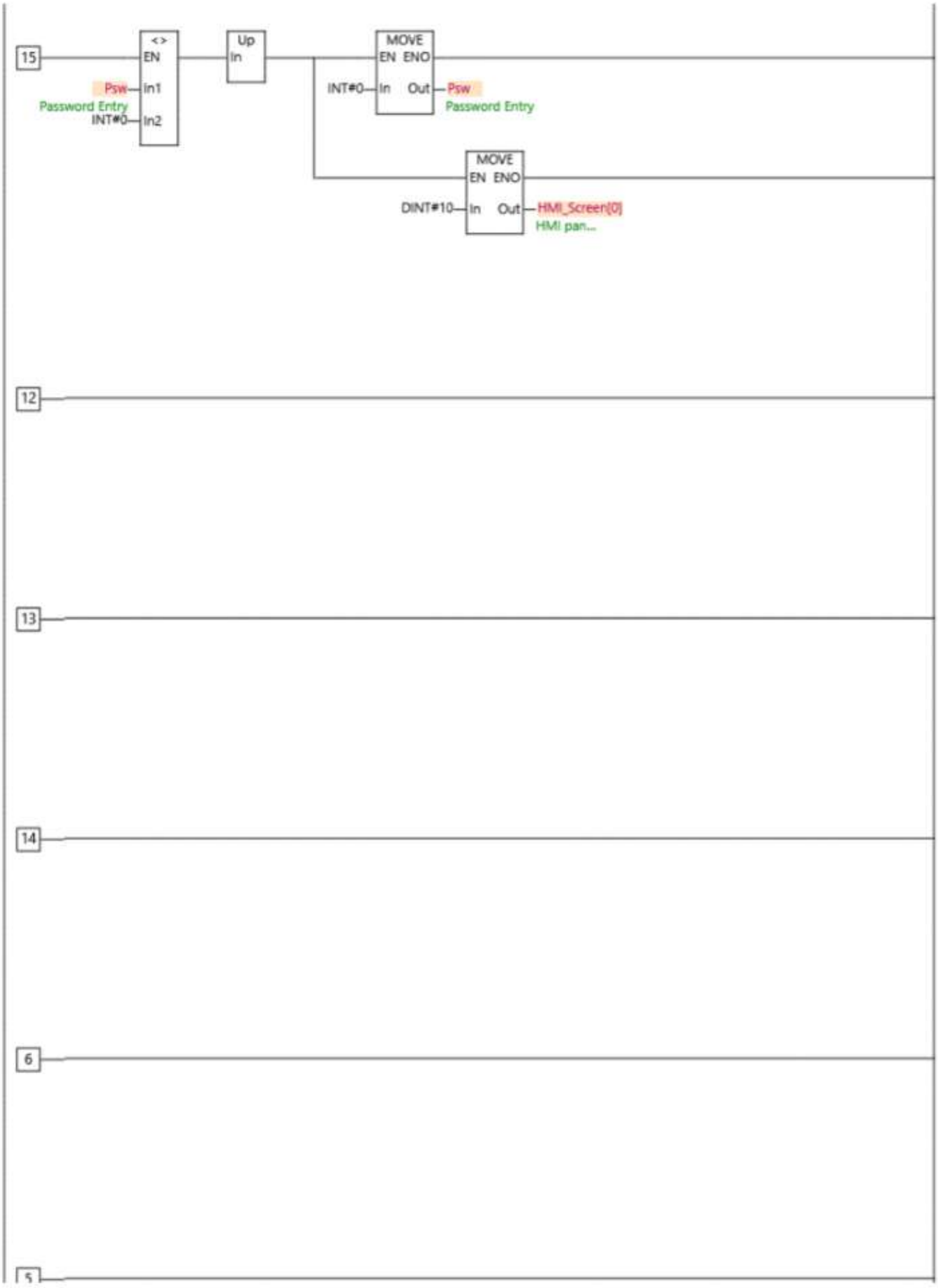


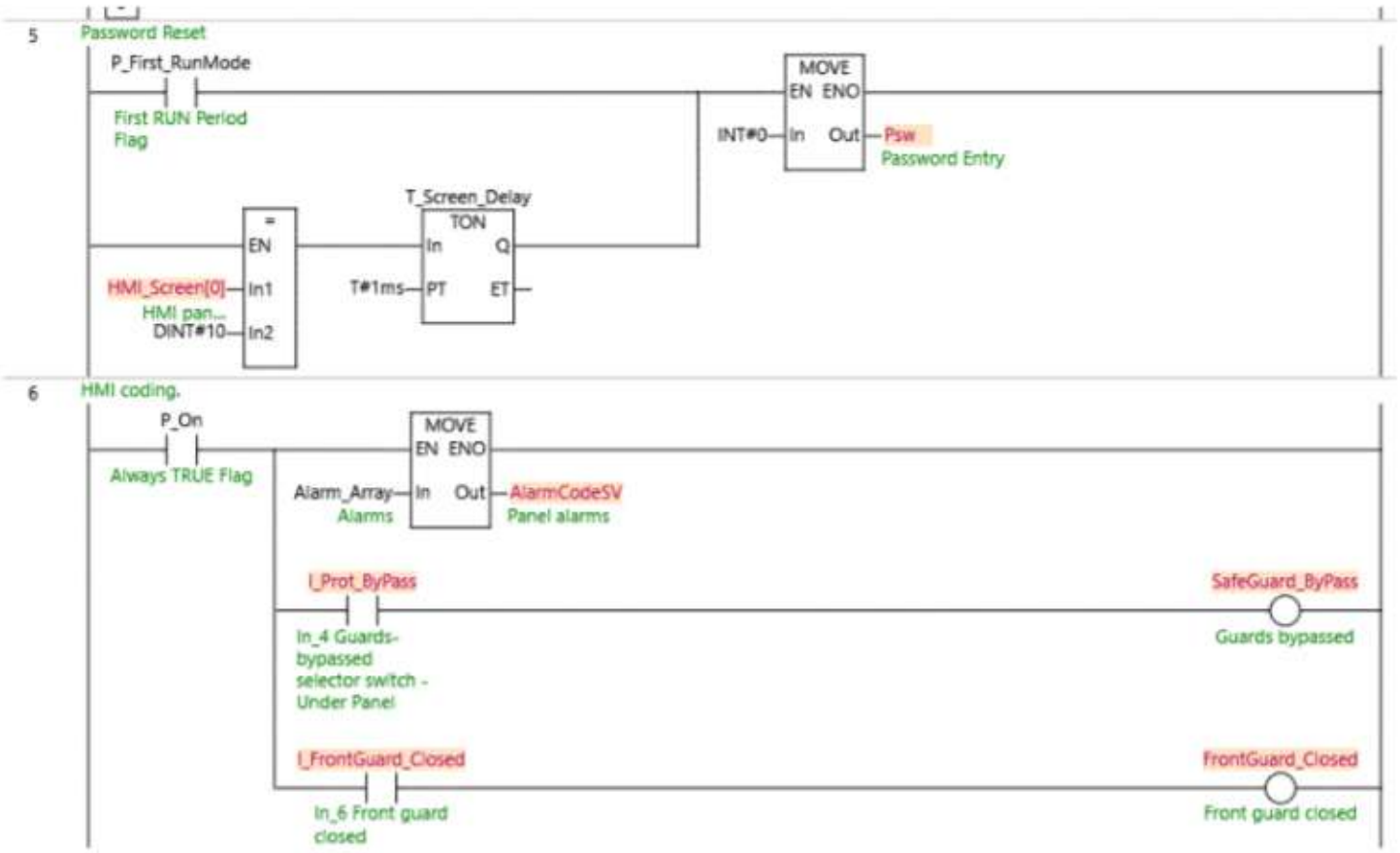
1-8-1-1-5.Console




4
Password check..
P_On
Always TRUE Flag
<>
EN
Psw
Password Entry
In1
In2
INT#22
INT#33
INT#40
4
=
7
1
8
2
9
3
11
6
DINT#44
HMI_Screen[0]
HMI pan...
10
5
Up
In
MOVE
EN ENO
Out


DINT#48
In
Out
HMI_Screen[0]
HMI pan...
4
EN
Psw
Password Entry
INT#44
In1
In2
15
INT#48
Psw22
7
Password 22 for Popup 22
MOVE
EN ENO
1
12
DINT#22
Psw33
8
Password 33 for Popup 33
2
13
DINT#33
Psw40
9
Password 40 for Popup 40
3
14
DINT#40
Psw44
11
Password 44 for Popup 44
Psw48
10
Password 48 for Popup 48


15
<>
EN
Psw
Password Entry
In1
INT#0
In2
Up
In
MOVE
EN ENO
INT#0
In
Out
Psw
Password Entry
MOVE
EN ENO
DINT#10
In
Out
HMI_Screen[0]
HMI pan...
12
13
14
6
5


5
Password Reset
P_First_RunMode
First RUN Period Flag
MOVE
EN ENO
INT#0
In
Out
Psw
Password Entry
T_Screen_Delay
=
EN
In1
In2
HMI_Screen[0]
HMI pan...
DINT#10
TON
In
Q
T#1ms
PT
ET
6
HMI coding.
P_On
Always TRUE Flag
MOVE
EN ENO
Alarm_Array
Alarms
In
Out
AlarmCodeSV
Panel alarms
I_Prot_ByPass
In_4 Guards-bypassed selector switch - Under Panel
SafeGuard_ByPass
Guards bypassed
I_FrontGuard_Closed
In_6 Front guard closed
FrontGuard_Closed
Front guard closed

1-8-1-1-6.Alarms

0 Check Present alarms.

```
// Alarm / warning handling + panel buzzer (BuzzerOn).
// Alarms: buzzer pulses (Clock1s). ALARM RESET (panel or HMI) silences it 1 min;
//         a NEW alarm during the silence sounds again at once.
// Warnings: ONE 1 s beep each time a new warning appears.
// PrimaryTask 4 ms: 250 scans = 1 s, 15000 scans = 60 s.

AlarmOn := FALSE;
WarningOn := FALSE;
AlarmCnt := 0;
WarnCnt := 0;

FOR Index_A := 0 TO 32 DO

   IF Alarm_Array[Index_A] THEN

      CASE Index_A OF
         5, 13, 15, 16, 17, 18, 19, 20, 21:        // warnings
            WarningOn := TRUE;
            WarnCnt := WarnCnt + 1;

            IF NOT AlarmMem[Index_A] THEN
               WarnBeepTicks := 250;              // NEW warning -> 1 s beep
            END_IF;
      ELSE                                        // alarms
         AlarmOn := TRUE;
         AlarmCnt := AlarmCnt + 1;
      END_CASE;

   END_IF;
   AlarmMem[Index_A] := Alarm_Array[Index_A];
END_FOR;

// ALARM RESET button (panel I_RsetAlarm or HMI Alarm_Reset): silence 1 min

RstEdge := (I_RsetAlarm OR Alarm_Reset) AND NOT RstMem;
RstMem := I_RsetAlarm OR Alarm_Reset;

IF RstEdge AND AlarmOn THEN
   SilenceTicks := 15000;
END_IF;

IF (AlarmCnt > AlarmCntMem) OR NOT AlarmOn THEN
   SilenceTicks := 0;                             // new alarm / no alarm: not silenced
END_IF;

IF SilenceTicks > 0 THEN
   SilenceTicks := SilenceTicks - 1;
END_IF;

IF WarnBeepTicks > 0 THEN
   WarnBeepTicks := WarnBeepTicks - 1;
END_IF;

AlarmCntMem := AlarmCnt;
WarnCntMem := WarnCnt;

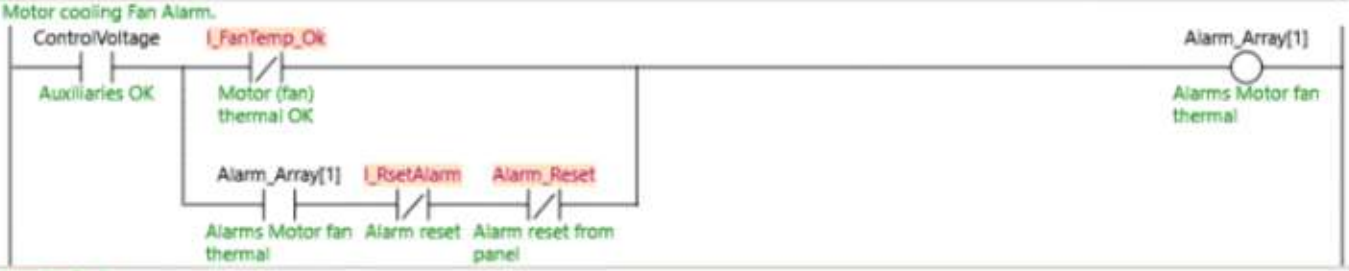
```

```
57  AlarmSilenced := AlarmOn AND (SilenceTicks > 0);
58
59  BuzzerOn := ControlVoltage
60          AND ((AlarmOn AND NOT AlarmSilenced AND Clock1s) OR (WarnBeepTicks
    > 0));
```

1

```
1   // ---- Alarm 14 (ALARM): unwinder sonar (NX-AD2603 Ch1, P+F UC2000) fault ----
2   // Raw value out of range for 1 s (no echo / broken wire / sensor off).
3   // Set when the measurement is used (unwinder AutoStop ON or motor running).
4   // latched. RESET only once the sensor is OK again (same as alarm 10, DT35).
5
6   T_UwSensFlt(In := (AI_Diam_Unwinder < UwRawMin) OR (AI_Diam_Unwinder >
    UwRawMax), PT := T#1s);
7   UwSensTest := T_UwSensFlt.Q;
8
9   IF ControlVoltage AND Measure_ON AND UwSensTest AND (UwStopEnable OR
    StartMotor_Cmd) THEN
10      UwSensFault := TRUE;
11  ELSIF NOT UwSensTest AND (I_RsetAlarm OR Alarm_Reset) THEN
12      UwSensFault := FALSE;
13  END_IF;
14
15  // ---- Alarm 17 (WARNING): Paper thickness not learned ----
16  // RwMat_K = 0 -> diameter / stop-length targets use the fixed fallback.
17  // Only above 300 mm rewinder diameter (learning needs ~20 mm growth after
    NEW PAPER),
18  // so it does not beep at every new roll. Not latched.
19
20  Thickness_Fault := ControlVoltage AND StartMotor_Cmd
21              AND (RwMat_K <= 0.0) AND (RwDiam > 300.0);
```

2 Motor cooling Fan Alarm.



3 Inverter fault.



4 Machine in emergency mode.



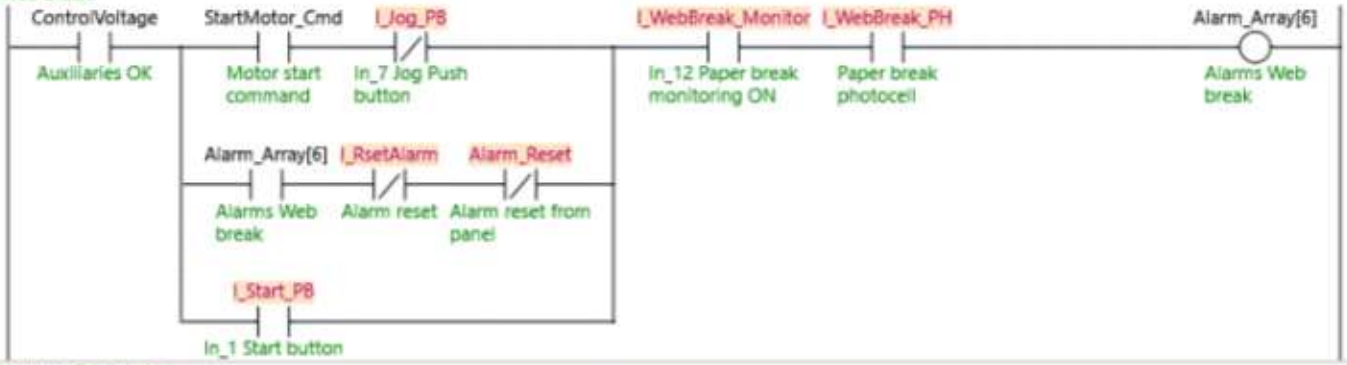
5 Main Motor thermal cutout.



6 Front Guard - Warning



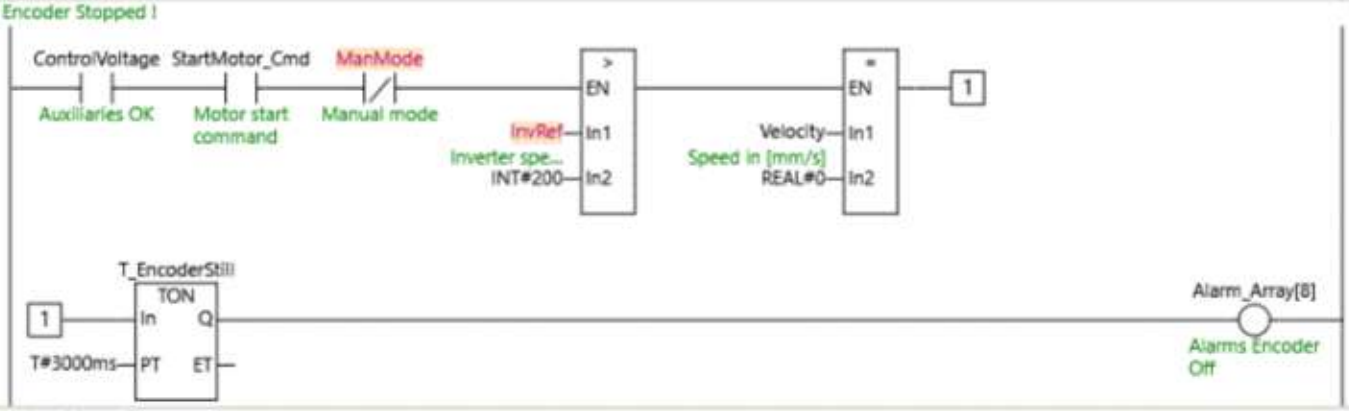
7 Web Break.



8 Vacuum Cooling fan.



9 Encoder Stopped !



10 Hub not in position.



11 Rewinder Diameter Sensor Fault.



12 Rewinder Diameter Sensor Fault - Reset



13 Rewinder max diameter Machine protection.



14 Rewinder max diameter Machine protection - Reset



15 Emergency Stop by Holding Stop PB 2s



16 Web Tension Monitor - Warning



17 Unwinder Diameter Sensor Fault.



18 Unwinder Roll is Empty - Warning



19 Unwinder Roll is Empty - Reset



20 Rewinder GSM learning fault



21 Rewinder learning fault - Reset



22 Paper Thickness Learn Process - Not Done ! - Warning



23 Side Door Opened - Warning

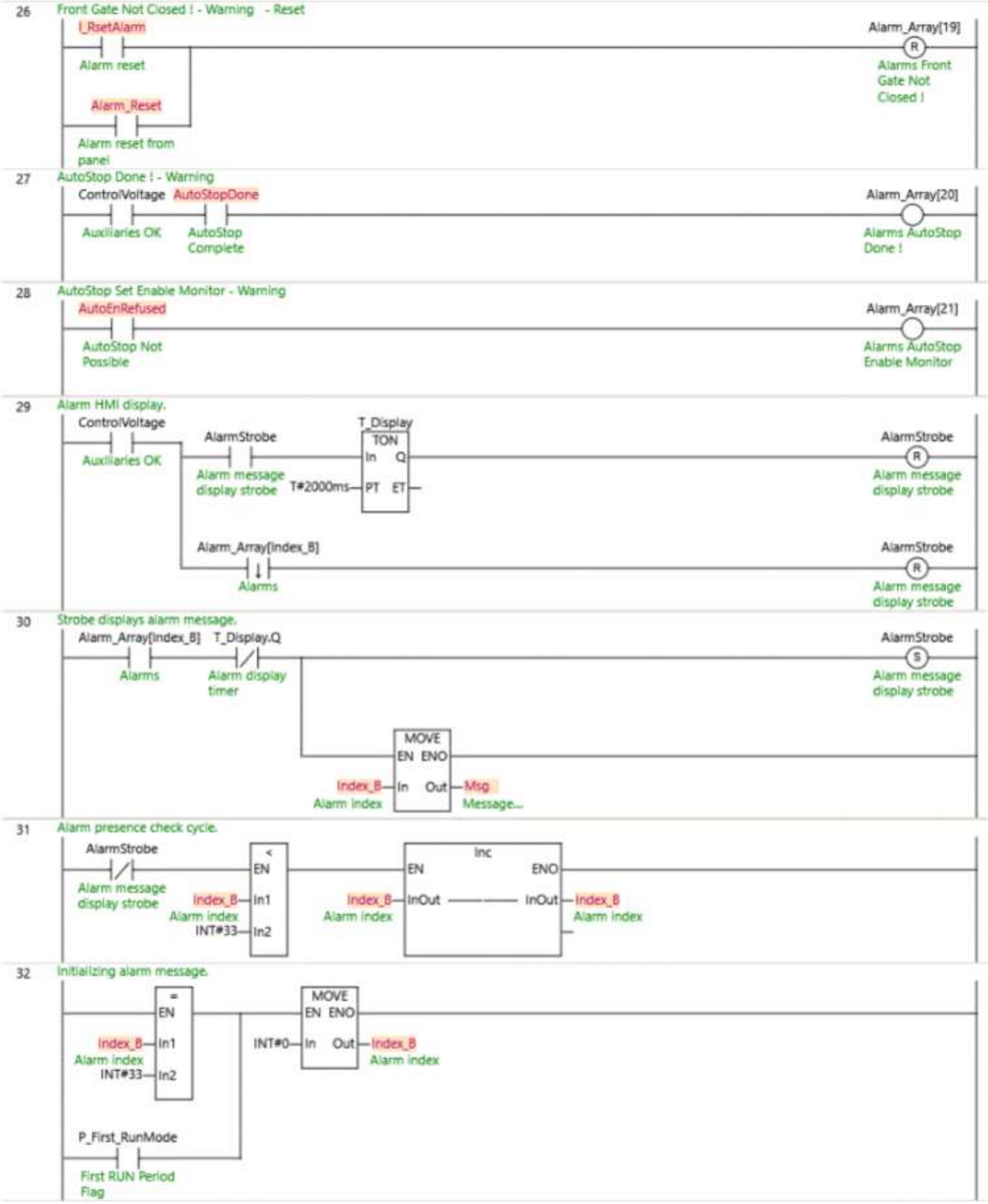


24 Side Door Opened - Warning - Reset



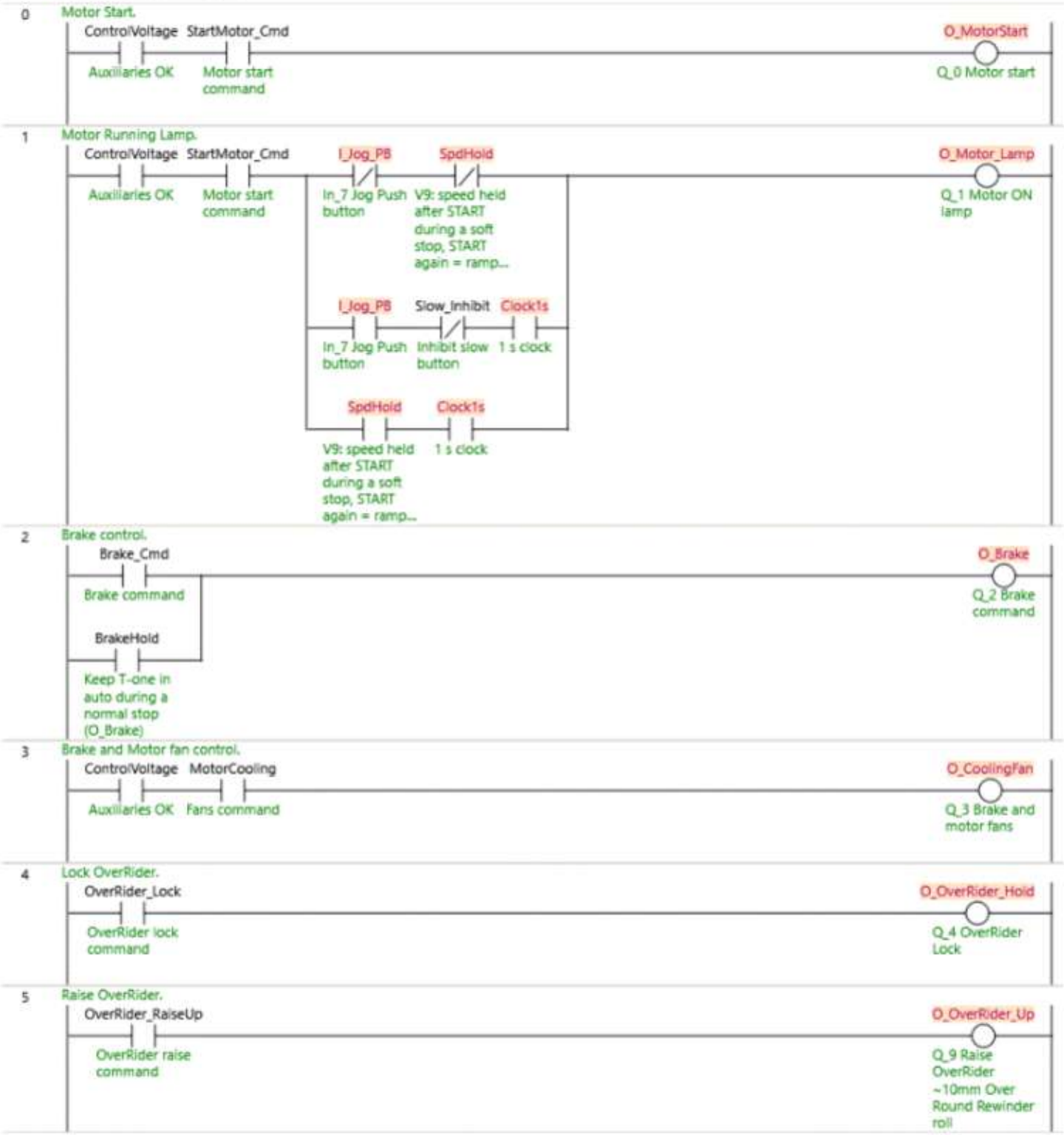
25 Front Gate Not Closed ! - Warning

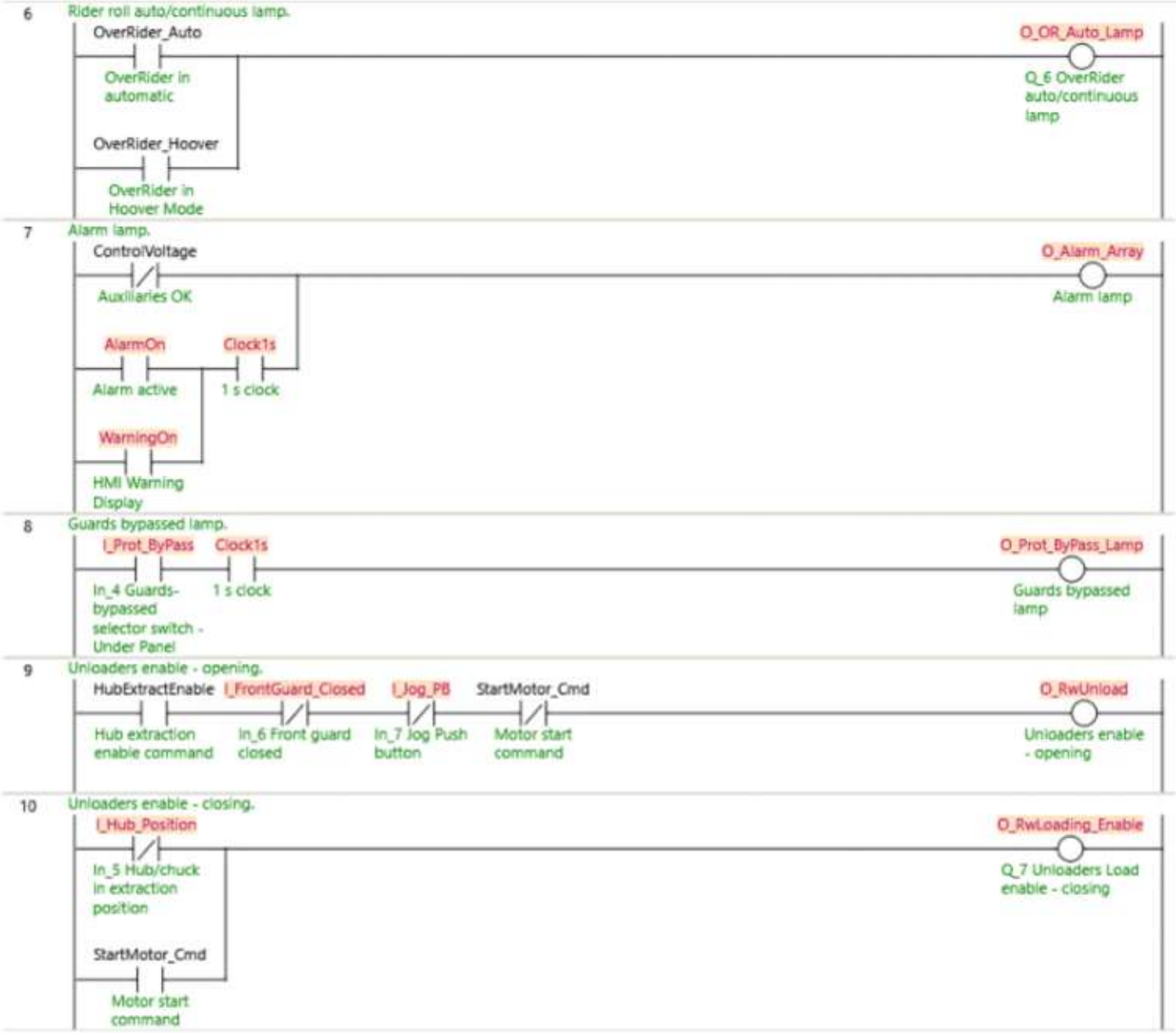



26
Front Gate Not Closed ! - Warning - Reset
I_RsetAlarm
Alarm reset
Alarm_Reset
Alarm reset from panel
Alarm_Array[19]
R
Alarms Front Gate Not Closed !
27
AutoStop Done ! - Warning
ControlVoltage
Auxiliaries OK
AutoStopDone
AutoStop Complete
Alarm_Array[20]
Alarms AutoStop Done !
28
AutoStop Set Enable Monitor - Warning
AutoEnRefused
AutoStop Not Possible
Alarm_Array[21]
Alarms AutoStop Enable Monitor
29
Alarm HMI display.
ControlVoltage
Auxiliaries OK
AlarmStrobe
Alarm message display strobe
T_Display
TON
In Q
T#2000ms PT ET
AlarmStrobe
R
Alarm message display strobe
Alarm_Array[Index_B]
Alarms
AlarmStrobe
R
Alarm message display strobe
30
Strobe displays alarm message.
Alarm_Array[Index_B]
Alarms
T_Display.Q
Alarm display timer
AlarmStrobe
S
Alarm message display strobe
MOVE
EN ENO
Index_B
Alarm index
In Out
Msg
Message...
31
Alarm presence check cycle.
AlarmStrobe
Alarm message display strobe
<
EN
Index_B
Alarm index
In1
INT#33
In2
Inc
EN ENO
Index_B
Alarm index
InOut InOut
Index_B
Alarm index
32
Initializing alarm message.
=
EN
Index_B
Alarm index
In1
INT#33
In2
MOVE
EN ENO
INT#0
In Out
Index_B
Alarm index
P_First_RunMode
First RUN Period Flag


33
Alarm Reset
AlarmOn
Alarm active
WarningOn
HMI Warning Display
MOVE
EN ENO
INT#33
In
Out
Msg
Message...

1-8-1-1-7.Output_Manager




6
Rider roll auto/continuous lamp.
OverRider_Auto
OverRider in automatic
OverRider_Hoover
OverRider in Hoover Mode
O_OR_Auto_Lamp
Q_6 OverRider auto/continuous lamp
7
Alarm lamp.
ControlVoltage
Auxiliaries OK
AlarmOn
Alarm active
Clock1s
1 s clock
WarningOn
HMI Warning Display
O_Alarm_Array
Alarm lamp
8
Guards bypassed lamp.
I_Prot_ByPass
In_4 Guards-bypassed selector switch - Under Panel
Clock1s
1 s clock
O_Prot_ByPass_Lamp
Guards bypassed lamp
9
Unloaders enable - opening.
HubExtractEnable
Hub extraction enable command
I_FrontGuard_Closed
In_6 Front guard closed
I_Jog_PB
In_7 Jog Push button
StartMotor_Cmd
Motor start command
O_RwUnload
Unloaders enable - opening
10
Unloaders enable - closing.
I_Hub_Position
In_5 Hub/chuck in extraction position
StartMotor_Cmd
Motor start command
O_RwLoading_Enable
Q_7 Unloaders Load enable - closing

11 Horn / acoustic signal.

```
// ==== V9: REPLACE Output_Manager rung 11 ("Horn / acoustic signal.", ladder) by ONE ST rung
// ==== with this code (no contact in front). Comment: "V9: horn, beeps set on the HMI".
// Old: horn blinking with Clock1s (0.5 s ON / 0.5 s OFF) during the whole slow-down
//      and the crawl at final speed -> very annoying.
// New, slow-down / final speed (StartMotor_Cmd AND SlowDown AND NOT Stop):
//      HornStartBeep_s beep when it starts (1 s).
//      then a HornBeepLen_s beep (0.2 s) every HornBeepPeriod (5 s).
//      then ONE HornStopBeep_s beep (3 s) when the machine has stopped (< 2 m/min).
//      Each setting 0 = that beep OFF. Front guard moving: unchanged (Clock1s blinking).
// PrimaryTask 4 ms: 250 scans = 1 s.

// HMI limits

IF HornStartBeep_s < REAL#0 THEN
    HornStartBeep_s := REAL#0;
ELSIF HornStartBeep_s > REAL#5 THEN
    HornStartBeep_s := REAL#5;
END_IF;

IF HornBeepLen_s < REAL#0 THEN
    HornBeepLen_s := REAL#0;
ELSIF HornBeepLen_s > REAL#2 THEN
    HornBeepLen_s := REAL#2;
END_IF;

IF HornBeepPeriod < REAL#0 THEN
    HornBeepPeriod := REAL#0;
ELSIF (HornBeepPeriod > REAL#0) AND (HornBeepPeriod < HornBeepLen_s + REAL#0.5) THEN
    HornBeepPeriod := HornBeepLen_s + REAL#0.5;    // always a pause between beeps
ELSIF HornBeepPeriod > REAL#30 THEN
    HornBeepPeriod := REAL#30;
END_IF;

IF HornStopBeep_s < REAL#0 THEN
    HornStopBeep_s := REAL#0;
ELSIF HornStopBeep_s > REAL#10 THEN
    HornStopBeep_s := REAL#10;
END_IF;

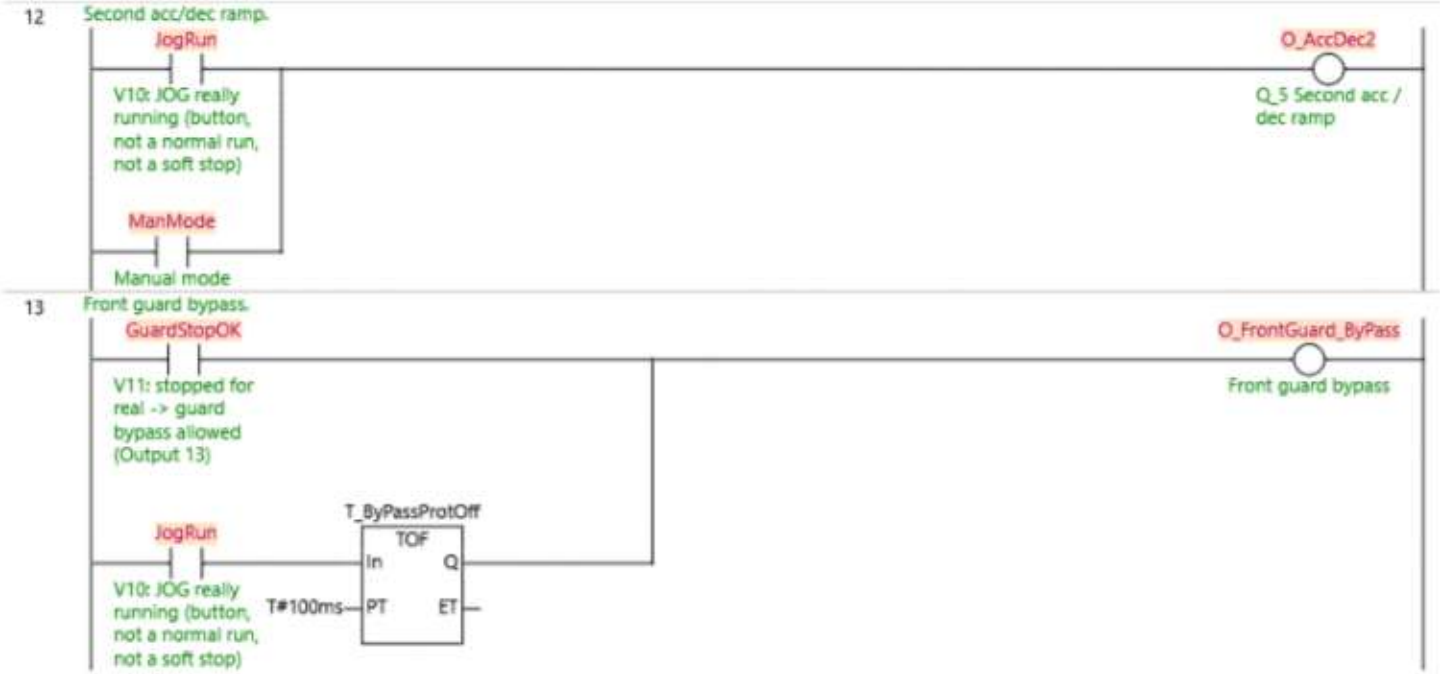
HornStartScans := REAL_TO_INT(HornStartBeep_s * REAL#250);
HornLenScans   := REAL_TO_INT(HornBeepLen_s * REAL#250);
HornPer        := REAL_TO_INT(HornBeepPeriod * REAL#250);

// Slow-down / final speed beeps

HornRun := StartMotor_Cmd AND SlowDown AND NOT Stop AND NOT JogRun;  // no horn while jogging

IF HornRun THEN
    IF NOT HornRunMem THEN
        HornTicks := 0;                    // slow-down starts
```

```
51      END_IF;
52
53      IF HornTicks < 30000 THEN
54          HornTicks := HornTicks + 1;
55      END_IF;
56
57      IF HornTicks <= HornStartScans THEN
58          HornSlowBeep := TRUE;                    // start beep
59          HornCnt := 0;
60      ELSIF (HornPer > 0) AND (HornLenScans > 0) THEN
61          HornCnt := HornCnt + 1;
62          IF HornCnt >= HornPer THEN
63              HornCnt := 0;
64          END_IF;
65
66          HornSlowBeep := (HornCnt >= HornPer - HornLenScans);   // beep at the end
    of each period
67      ELSE
68          HornSlowBeep := FALSE;                   // periodic beeps OFF
69      END_IF;
70  ELSE
71      HornTicks := 0;
72      HornCnt := 0;
73      HornSlowBeep := FALSE;
74  END_IF;
75
76  // End of a slow-down by a stop (auto stop, STOP button): one beep at standstill
77  // (Stop is set in Motor_Manager before StartMotor_Cmd drops on the next scan)
78
79  IF HornRunMem AND NOT HornRun AND (Stop OR NOT StartMotor_Cmd) THEN
80      HornStopPend := TRUE;
81  END_IF;
82
83  IF StartMotor_Cmd AND NOT HornStartMem THEN
84      HornStopPend := FALSE;                       // restarted: no stop beep
85      HornStopTicks := 0;
86  END_IF;
87
88  HornStartMem := StartMotor_Cmd;
89
90  IF HornStopPend AND (Velocity_M_min < REAL#2) THEN
91      HornStopPend := FALSE;
92      HornStopTicks := REAL_TO_INT(HornStopBeep_s * REAL#250);
93  END_IF;
94
95  IF HornStopTicks > 0 THEN
96      HornStopTicks := HornStopTicks - 1;
97  END_IF;
98
99  HornRunMem := HornRun;
100
101 // HMI test button: horn while pressed (machine stopped only)
102
103 O_AcousticSignal := (NOT I_FrontGuard_Closed AND NOT I_FrontGuard_Open AND
    Clock1s)
104                 OR HornSlowBeep OR (HornStopTicks > 0)
105                 OR (HornTest AND NOT StartMotor_Cmd);
```


12
Second acc/dec ramp.
JogRun
V10: JOG really running (button, not a normal run, not a soft stop)
ManMode
Manual mode
O_AccDec2
Q_5 Second acc / dec ramp
13
Front guard bypass.
GuardStopOK
V11: stopped for real -> guard bypass allowed (Output 13)
JogRun
V10: JOG really running (button, not a normal run, not a soft stop)
T_ByPassProtOff
TOF
In
Q
T#100ms
PT
ET
O_FrontGuard_ByPass
Front guard bypass

1-8-1-1-8.AutoStop_Control

0 Auto Stop Process 1-5

```
(*
=============================================================
====================
  AutoStop_Control  V10  -  PART 1 of 2 (rung 0): sections 1 - 5c
  Project : 04820KA001 Mono Canada II  (NX1P2, PrimaryTask 4 ms)
  Rewinder (Rw) : SICK DT35 0-10 V -> AI_Diam_Rewinder (AD2603 Ch2)
  Unwinder (Uw) : meter counter mode + sonar diameter (UwDiam_Act)

  NAMING  Uw_ = unwinder   Rw_ = rewinder
        _mm = millimeters   _m = meters
        _Pls = encoder pulses (counts)   _Pps = pulses per second
        _Pps2 = pulses per second squared

  V10: JogRun (jog really running) instead of the raw jog button (5, 5b):
       long baseline also restarted if the encoder goes 2 m back (4b).
  V9 : RwDiamRev = DT35 averaged over exactly one revolution (2b), used by
       the thickness learning (removes the out-of-round wave), RwWave_mm.
     GSM popup: once per real new roll (empty 3 s, then > stop + 100 mm
       for 3 s): NEW PAPER closes it (section 10).
     paper thickness: long-baseline learning (D^2 growth / length) from the
       start point, LOCKED at RwLockDiam (300 mm): no more change, no
       paper-change detection, until NEW PAPER / GSM popup YES (4b).
  V8 : = V7 + V7c unwinder DIAMETER slow-down follows the speed (6c).
       V7e ManMode slow-down model as before V7 (section 6),
       V7f S-curve extra length in the slow-down (section 6)
  V7 : FEED-FORWARD speed loop (Motor_Manager rung 8): reference = set speed
     x RwRatioD / diameter along a ramp in real web mm/s2 (DecelWeb /
     AccelWeb), RwRatioD adapted slowly -> no hunting on big rolls.
     Section 6: slow-down length = one ramp at DecelWeb (FFEnable), else
     web coarse ramp + trim tail; learning only as fallback.
  v6y: rewinder AUTO stop (RwStopEnable) limits the work speed to
     RwAutoMaxSpd (500 m/min), taken over at standstill only; section 6
     uses the limited speed.
  v6x: new parent roll on the unwinder -> GsmAsk (HMI popup "did you change
     the paper?"); GsmAskYes forgets the learned thickness.
  v6w: paper thickness window is LENGTH-based instead of 10 s, modulated by
     the diameter (RwWinGrow_mm 2.5 mm of growth, 4 m .. RwWinLen_m 30 m):
     same accuracy at every speed and roll size; stopped window = dropped.
     ALARM RESET also clears LearnFailCnt / StopFailCnt (alarm 16).
  v6v: BrakeHold released on the SIGNED web speed (Velocity_M_min < 5),
     so a web pulled back after the stop releases the T-one at once.
  v6u: stop learning: a stop shorter than the reaction distance or faster
     than RwDecelMax_mm is ignored (not a fault). Only a too slow stop
     (< RwDecelMin_mm) counts toward alarm 16.
  v6t: rewinder slow-down fallback while the paper thickness is not
     learned: RwSlowReq at RwTargetDiam - RwSlowFallbackMm (30 mm).
  v6s: paper change: a learning window more than RwMatChangePct (25 %) away
     from RwMat_K is ignored once, taken directly the 2nd time in a row
     (automatic). RwMatReset (HMI "NEW PAPER") forgets RwMat_K.
  v6p: section 5c: RwLearnReset clears RwStopDecelK, RwStopDecel and
     RwSlowDecelK (the v6f seed that refilled RwStopDecelK is removed).
  v6l: previous value saved before every TEACH CORE (RwAxisDist_Prev,
       UwCoreDist_Prev, Retain) + UNDO buttons (RwTeachUndo,
       UwTeachUndo): swap current <-> previous. State 4 = RESTORED.
  v6k: unwinder TEACH CORE (Core_Measure) works like the rewinder one:
       same checks, 2 s stable with sonar AVERAGE, status UwTeachState.
```

```
55          Replaces General rung 15 (one immediate sonar reading).
56      v6j: TEACH CORE (section 14): HMI button RwTeachCore, empty core on the
57          rewinder, machine stopped -> after 2 s stable:
58          RwAxisDist := RwDist + RwCoreDiameter / 2. Status in RwTeachState.
59      v6i: auto: pots scaled to WorkSpd_Max / FinalSpd_Max (Motor_Manager rung 6,
60          Pot1_Mem / Pot2_Mem). ManMode keeps the raw pots (percent),
61          so section 6 uses the raw pots in ManMode.
62      v6h: BrakeHold keeps O_Brake (T-one "reel brake automatic") ON during a
63          normal stop until the web is almost stopped (< 5 m/min, max 15 s).
64          Before: T-one switched to "stop" with the run command, while the
65          rewinder was decelerating hard -> unwinder overran, web on the
66          floor (field 10/20/2026). Used in Output_Manager rung 2.
67      v6g: slow-down AND stop decel proportional to the rewinder diameter.
68          The ATV930 ramp is in Hz/s (motor = rewind shaft), so the web
69          decel [mm/s2] = shaft decel x radius. Field data 10/20/2026:
70          24 mm/s2 at 120 mm -> 90 mm/s2 at 600 mm.
71          Learned: RwSlowDecelK, RwStopDecelK [mm/s2 per mm of diameter].
72          Live:    RwSlowDecelLrn [mm/s2], RwStopDecel [pulses/s2] = K x DecelDiam.
73      v6f: learn the REAL slow-down deceleration (section 5b) -> RwSlowDecelLrn
74          (mm/s2). The ATV930 stretches its ramp with a heavy roll (no
75          braking resistor), so the PLC ramp (DecelCoarse) was too
76          optimistic: the stop came before final speed was reached.
77          Priority in section 6: RwSlowDecel (manual) > RwSlowDecelLrn
78          (learned) > DecelCoarse/DecelTrim model.
79      v6e: GSM_Learned_OK (replaces global RwLearnFault): TRUE only when
80          RwMat_K is learned, RwStopDecel is learned (if RwLearnStop) and
81          there is no learning fault. Alarm 16 uses GSM_LearnFault.
82      v6d: exact 100 ms tick (25 x 4 ms scan counter, the self-resetting
83          TON ran ~104-108 ms -> speed ~4-8 % too high)
84         standstill = <= 2 counts / 100 ms (encoder jitter)
85         stop learning skipped after jog / manual / slow-button runs and
86          when the auxiliaries drop (E-stop): RunWasJog, StopLearnValid
87         work / final speed from the potentiometers in BOTH modes
88          (Speed_Display / LowSpd_Display are not m/min in ManMode)
89         slow-down length includes the DecelTrim tail inside the
90          SpdDelta_Percent band (Motor_Manager rungs 14/15)
91      v6c: learning supervision -> RwLearnFault (alarm 16), learned paper
92          thickness RwThick_mm, plausibility limits for K and stop decel
93      v6b: RwMaxSlowAtDiam display + RwMaxSlowFallback (slow-down warning
94          while RwMat_K is not learned yet)
95      v6 : consistent internal names, units in names.
96         Globals renamed: RwRemain_m, RwStopLen_m, RwSlowLen_m,
97                     RwSlowMargin_m, UwSlowMargin_m
98      v5 : max diameter protection (always active), sensor plausibility,
99         12SA5 master (section 9), alarms in section Alarms (ladder)
100     v4b: AUTO interlock + single HMI display (SlowAtDisp / SlowAtMode)
101
    ================================================================
    ==================== *)
102
103 (* ---- 1. 100 ms tick, mm per encoder pulse ------------------------------ *)
104 (* v6d: 25 scans x 4 ms PrimaryTask = exactly 100 ms.                    *)
105 (*      Change 25 if the PrimaryTask period is changed.                  *)
106 TickCnt := TickCnt + 1;
107
108 IF TickCnt >= 25 THEN
109     TickCnt := 0;
110     Tick := TRUE;
111 ELSE
```

```
112     Tick := FALSE;
113  END_IF;
114
115  IF (Encoder_Pulse > 0) AND (EncoderWheel_Circumf > 0.0) THEN
116     MmPerPulse := EncoderWheel_Circumf / (INT_TO_REAL(Encoder_Pulse) * 4.0);
117  ELSE
118     MmPerPulse := 0.0;
119  END_IF;
120
121  (* ---- 2. Rewinder DT35 -> filtered distance -> diameter ---------- *)
122  RwDistRaw_mm := RwDist0V + (INT_TO_REAL(AI_Diam_Rewinder) / 4000.0)
123                * (RwDist10V - RwDist0V);
124
125  IF Tick THEN
126
127     IF NOT RwFiltInit THEN
128        RwDist := RwDistRaw_mm;
129        RwFiltInit := TRUE;
130     ELSE
131        RwDist := RwDist + RwFilter_K * (RwDistRaw_mm - RwDist);
132     END_IF;
133  END_IF;
134
135  RwSensFault := (AI_Diam_Rewinder < RwRawMin) OR (AI_Diam_Rewinder >
     RwRawMax)
136               OR (RwDist > RwAxisDist);
137
138  RwDiam := 2.0 * (RwAxisDist - RwDist);
139
140  IF RwDiam < 0.0 THEN
141     RwDiam := 0.0;
142  END_IF;
143
144  (* ---- 2b. V9: diameter averaged over exactly ONE revolution ---------- *)
145  (* The out-of-round roll gives a wave at the rotation frequency (seen in   *)
146  (* the 10/29 traces: 2.4 Hz at 444 m/min / 973 mm). Averaging the raw     *)
147  (* DT35 distance over exactly one revolution cancels it (and all its      *)
148  (* harmonics), whatever the phase -> no shaft position sensor needed:     *)
149  (* one revolution = PI x D of web on the encoder. Each scan is weighted   *)
150  (* by its encoder pulses = by roll ANGLE, so ramps and standstill do not  *)
151  (* bias it (stopped: nothing added). Used only by the thickness learning. *)
152  (* RwWave_mm = peak-peak diameter wave in the last revolution (HMI).      *)
153  RevDP := ABS(Encoder_Pos - RevPrevPos);
154  RevPrevPos := Encoder_Pos;
155
156  IF RwSensFault OR (MmPerPulse <= 0.0) OR (RwDiam < 50.0) OR (RevDP > 100000)
     THEN
157     RevW := 0.0;
158     RevSum := 0.0;
159     RwDiamRevOk := RwDiamRevOk AND NOT RwSensFault;
160  ELSIF RevDP > 0 THEN
161
162     IF RevW <= 0.0 THEN
163        RevMin := RwDistRaw_mm;
164        RevMax := RwDistRaw_mm;
165     END_IF;
166     RevSum := RevSum + RwDistRaw_mm * DINT_TO_REAL(RevDP);
167     RevW := RevW + DINT_TO_REAL(RevDP);
168
```

```
169     IF RwDistRaw_mm < RevMin THEN
170         RevMin := RwDistRaw_mm;
171     END_IF;
172
173     IF RwDistRaw_mm > RevMax THEN
174         RevMax := RwDistRaw_mm;
175     END_IF;
176
177     IF RevW >= 3.14159 * RwDiam / MmPerPulse THEN
178         RwDiamRev := 2.0 * (RwAxisDist - RevSum / RevW);
179         RwWave_mm := 2.0 * (RevMax - RevMin);
180         RwDiamRevOk := TRUE;
181         RevW := 0.0;
182         RevSum := 0.0;
183     END_IF;
184 END_IF;
185
186 (* v6g: diameter used by the decel model (last valid on sensor fault)      *)
187
188 IF NOT RwSensFault THEN
189     DecelDiam := RwDiam;
190 ELSE
191     DecelDiam := RwLastValidDiam;
192 END_IF;
193
194 IF DecelDiam < 100.0 THEN
195     DecelDiam := 100.0;                    (* core *)
196 END_IF;
197
198 (* ---- 3. Actual speed from encoder, 100 ms window ---------------------- *)
199
200 IF Tick THEN
201
202     IF NOT SpdInit THEN
203         EncPrev := Encoder_Pos;
204         SpdInit := TRUE;
205     END_IF;
206     SpdAct_Pps := DINT_TO_REAL(ABS(Encoder_Pos - EncPrev)) * 10.0;
207     EncPrev := Encoder_Pos;
208 END_IF;
209 RwSpeed := SpdAct_Pps * MmPerPulse * 0.06;        (* m/min *)
210
211 (* ---- 4a. v6s: NEW PAPER button: forget the learned thickness ---------- *)
212 (* RwMat_K = 0 -> the first valid 10 s window sets it directly.           *)
213 (* HMI momentary button: must be HELD 3 s, then the reset is done once and *)
214 (* the PLC clears the bit (HMI lamp goes off = done).                     *)
215 T_MatReset(In := RwMatReset, PT := T#3s);
216
217 IF T_MatReset.Q THEN
218     RwMat_K := 0.0;
219     LearnFailCnt := 0;
220     MatChangeCnt := 0;
221     RwMatLocked := FALSE;                  (* V9 *)
222     LongActive := FALSE;
223     GsmAsk := FALSE;                       (* V9: NEW PAPER answers the popup *)
224     GsmArmed := FALSE;
225     RwMatReset := FALSE;
226 END_IF;
227
```

```
(* ---- 4b. V9: long-baseline thickness learning + LOCK ---------- *)
(* From the start point (first run after NEW PAPER / GSM YES / power-on    *)
(* while not locked): K = (D^2 - D0^2) / (2 x pulses). The waving of the   *)
(* roll (+/-0.4 mm) only matters at the two ends, so the error falls with  *)
(* the growth (50 mm growth: ~1 %). After 20 mm growth it replaces the     *)
(* 30 m windows. At RwLockDiam (and >= RwLockMinGrow_mm growth) the value
 *)
(* is LOCKED (Retain): windows and paper-change detection stop.           *)
(* RwLockDiam = 0 -> never locked (windows + long baseline as learning).   *)

IF Tick AND NOT RwMatLocked THEN

    IF LongActive AND (RwDiamRev < LongStartD - 30.0) AND RwDiamRevOk THEN
        LongActive := FALSE;                    (* new core: restart *)
    END_IF;

    IF Encoder_Reset OR (Encoder_Pos < LongStartPos - 20000) THEN
        LongActive := FALSE;                    (* metre counter reset / 2 m back *)
    END_IF;

    IF NOT LongActive AND O_MotorStart AND RwDiamRevOk
      AND (RwDiamRev > 0.0) THEN
        LongStartPos := Encoder_Pos;
        LongStartD := RwDiamRev;
        LongActive := TRUE;
    END_IF;
    LongValid := FALSE;

    IF LongActive AND RwDiamRevOk AND (MmPerPulse > 0.0) THEN
        LongGrow_mm := RwDiamRev - LongStartD;
        LongPls := DINT_TO_REAL(ABS(Encoder_Pos - LongStartPos));

        IF (LongGrow_mm >= 20.0) AND (LongPls > 0.0) THEN
            LongK := (RwDiamRev * RwDiamRev - LongStartD * LongStartD) / (2.0 *
LongPls);
            LongThick_mm := LongK * 3.14159 / (2.0 * MmPerPulse);

            IF (LongThick_mm >= RwThickMin_mm) AND (LongThick_mm <=
RwThickMax_mm) THEN
                LongValid := TRUE;
                RwMat_K := LongK;
                MatChangeCnt := 0;
                LearnFailCnt := 0;

                IF (RwLockDiam > 0.0) AND (RwDiamRev >= RwLockDiam)
                  AND (LongGrow_mm >= RwLockMinGrow_mm) THEN
                    RwMatLocked := TRUE;        (* solid from now on *)
                    LongActive := FALSE;
                END_IF;
            END_IF;
        END_IF;
    END_IF;
END_IF;

IF RwMatLocked THEN
    LongValid := FALSE;
    LearnFailCnt := 0;                  (* no thickness alarm *)
END_IF;

```

```
(* ---- 4. Learn material factor  K = dDiam/dPulse * Diam ---------- *)

IF Tick THEN
   LearnWinTicks := LearnWinTicks + 1;

   IF NOT LearnInit THEN
      LearnWinStartPos := Encoder_Pos;  LearnWinStartDiam := RwDiamRev;  (* V9 *)
      LearnWinTicks := 0;
      LearnWinRunning := O_MotorStart;
      LearnInit := TRUE;
   ELSE
      LearnWinRunning := LearnWinRunning AND O_MotorStart;
      (* v6w: LENGTH-based window, modulated by the diameter: closes    *)
      (* when the roll should have grown RwWinGrow_mm (2.5 mm), so the  *)
      (* accuracy vs DT35 noise is the same at every speed and size.    *)
      (* Pulses = growth x diameter / K (K = learned RwMat_K).          *)
      (* Limits: min 4 m, max RwWinLen_m (30 m). K not learned: max.    *)
      (* Motor stopped during the window -> window dropped (no fault).  *)

      IF MmPerPulse > 0.0 THEN

         IF (RwMat_K > 0.0) AND (RwWinGrow_mm > 0.0) THEN
            LearnWinLen_Pls := RwWinGrow_mm * MAX(RwDiam, REAL#100.0) /
RwMat_K;
         ELSE
            LearnWinLen_Pls := RwWinLen_m * 1000.0 / MmPerPulse;
         END_IF;

         IF LearnWinLen_Pls < 4000.0 / MmPerPulse THEN
            LearnWinLen_Pls := 4000.0 / MmPerPulse;            (* 4 m *)
         END_IF;

         IF (RwWinLen_m > 0.0)
           AND (LearnWinLen_Pls > RwWinLen_m * 1000.0 / MmPerPulse) THEN
            LearnWinLen_Pls := RwWinLen_m * 1000.0 / MmPerPulse;
         END_IF;
      ELSE
         LearnWinLen_Pls := 300000.0;
      END_IF;

      IF (DINT_TO_REAL(ABS(Encoder_Pos - LearnWinStartPos)) >= LearnWinLen_Pls)
        OR NOT LearnWinRunning THEN
         LearnWin_Pls := ABS(Encoder_Pos - LearnWinStartPos);
         LearnWinDiamDelta := RwDiamRev - LearnWinStartDiam;  (* V9 *)
         (* A window SHOULD learn: ran all the time, enough length,  *)
         (* DT35 OK. Then it must give a plausible paper thickness. *)

         IF LearnWinRunning AND (LearnWin_Pls >= RwMinPulses)
           AND NOT RwSensFault
           AND NOT RwMatLocked AND NOT LongValid     (* V9 *)
           AND RwDiamRevOk AND (LearnWinStartDiam > 0.0) THEN
            LearnWinOk := FALSE;

            IF (LearnWinDiamDelta > 0.0) AND (MmPerPulse > 0.0) THEN
               MatK_Window := (LearnWinDiamDelta / DINT_TO_REAL(LearnWin_Pls))
                           * ((RwDiamRev + LearnWinStartDiam) / 2.0);
               (* K = 2 * thickness * mm-per-pulse / PI         *)
               ThickWin_mm := MatK_Window * 3.14159 / (2.0 * MmPerPulse);
```

```
342                 IF (ThickWin_mm >= RwThickMin_mm)
343                   AND (ThickWin_mm <= RwThickMax_mm) THEN
344                     LearnWinOk := TRUE;
345
346                     IF RwMat_K <= 0.0 THEN
347                        RwMat_K := MatK_Window;
348                        MatChangeCnt := 0;
349                     ELSIF (RwMatChangePct > 0.0)
350                         AND (ABS(MatK_Window - RwMat_K)
351                             > RwMat_K * RwMatChangePct / 100.0) THEN
352                        (* v6s: far from the learned value: 1 window = ignored *)
353                        (* (outlier). 2 in a row = paper changed -> take it.  *)
354                        MatChangeCnt := MatChangeCnt + 1;
355
356                        IF MatChangeCnt >= 2 THEN
357                           RwMat_K := MatK_Window;
358                           MatChangeCnt := 0;
359                        END_IF;
360                     ELSE
361                        MatChangeCnt := 0;
362                        RwMat_K := RwMat_K + RwLearn_K * (MatK_Window - RwMat_K);
363                     END_IF;
364                 END_IF;
365               END_IF;
366
367               IF LearnWinOk THEN
368                  LearnFailCnt := 0;
369               ELSIF LearnFailCnt < 1000 THEN
370                  LearnFailCnt := LearnFailCnt + 1;  (* no growth / implausible *)
371               END_IF;
372             END_IF;
373             LearnWinStartPos := Encoder_Pos;  LearnWinStartDiam := RwDiamRev;  (* V9
    *)
374             LearnWinTicks := 0;
375             LearnWinRunning := O_MotorStart;
376           END_IF;
377         END_IF;
378       END_IF;
379
380     (* ---- 5. Learn real stop deceleration on every stop ---------------------- *)
381     (* v6d: a run with jog / manual / slow button uses the 2nd ramp (O_AccDec2) *)
382     (*      -> its stop must not be learned. Latched for the whole run because  *)
383     (*      the jog button is already released on the stop edge.                *)
384
385     IF O_MotorStart AND NOT MotorRunMem THEN          (* start edge *)
386        RunWasJog := FALSE;
387     END_IF;
388
389     IF O_MotorStart AND (ManMode OR JogRun) THEN
390        RunWasJog := TRUE;
391     END_IF;
392
393     IF MotorRunMem AND NOT O_MotorStart THEN          (* stop command edge *)
394        StopMeasActive := TRUE;
395        StopStartPos := Encoder_Pos;
396        StopStartSpd_Pps := SpdAct_Pps;
397        StopStartDiam := DecelDiam;
398        StandstillTicks := 0;
399        (* STOP button and automatic stop take the same path (drive DI1 and  *)
```

```
    (* O_Brake -> T-one drop together, electrical plan sheets 1/9/15),    *)
    (* so both are learned.                                               *)
    StopLearnValid := ControlVoltage AND NOT RunWasJog AND NOT ManMode;
END_IF;
MotorRunMem := O_MotorStart;

IF O_MotorStart THEN
    StopMeasActive := FALSE;                    (* restarted: abort *)
ELSIF StopMeasActive AND Tick THEN

    IF NOT ControlVoltage THEN
        StopLearnValid := FALSE;                (* E-stop during the stop *)
    END_IF;

    IF SpdAct_Pps <= 20.0 THEN                  (* <= 2 counts / 100 ms *)
        StandstillTicks := StandstillTicks + 1;
    ELSE
        StandstillTicks := 0;
    END_IF;

    IF StandstillTicks >= 3 THEN                (* standstill 300 ms *)
        StopMeasActive := FALSE;
        StopMeas_Pls := DINT_TO_REAL(ABS(Encoder_Pos - StopStartPos));

        IF RwLearnStop AND StopLearnValid
          AND (StopStartSpd_Pps * MmPerPulse * 0.06 >= RwMinLearnSpd) THEN
            (* This stop SHOULD learn: result must be plausible       *)
            (* v6u: a stop shorter than the reaction distance, or     *)
            (* faster than RwDecelMax_mm, is NOT a fault: the web     *)
            (* stopped within a few mm, nothing to learn (trace       *)
            (* 10/28/2026: 23 m/min -> stopped in 102 pulses = 10 mm). *)
            (* Only a too SLOW stop (< RwDecelMin_mm) counts as fail.  *)
            StopLearnOk := FALSE;

            IF StopMeas_Pls > StopStartSpd_Pps * RwTreact THEN
                StopDecelMeas := (StopStartSpd_Pps * StopStartSpd_Pps)
                              / (2.0 * (StopMeas_Pls - StopStartSpd_Pps * RwTreact));
                StopDecelMeas_mm := StopDecelMeas * MmPerPulse;  (* mm/s2 *)

                IF (StopDecelMeas_mm >= RwDecelMin_mm)
                  AND (StopDecelMeas_mm <= RwDecelMax_mm) THEN
                    StopLearnOk := TRUE;
                    (* v6g: learn per mm of diameter *)
                    DecelK_Meas := StopDecelMeas_mm / ((StopStartDiam + DecelDiam) /
2.0);

                    IF RwStopDecelK <= 0.0 THEN
                        RwStopDecelK := DecelK_Meas;
                    ELSE
                        RwStopDecelK := RwStopDecelK + 0.3 * (DecelK_Meas -
RwStopDecelK);
                    END_IF;
                    StopFailCnt := 0;
                ELSIF StopDecelMeas_mm < RwDecelMin_mm THEN

                    IF StopFailCnt < 1000 THEN
                        StopFailCnt := StopFailCnt + 1;   (* stop too long *)
                    END_IF;
                END_IF;                                   (* too fast: ignored *)
```

```
            END_IF;                          (* too fast: ignored *)
          END_IF;                            (* too short: ignored *)
        END_IF;
      END_IF;
    END_IF;

    (* ---- 5b. v6f: learn real slow-down deceleration ------------------------ *)
    (* From the SlowDown edge (at speed) until final speed is reached, or     *)
    (* until the stop comes first. a = (v0^2 - v1^2) / (2 * distance).        *)
    (* Not learned in jog / manual / slow button (2nd ramp).                  *)

    IF SlowDown AND NOT SlowMem AND O_MotorStart
       AND NOT ManMode AND NOT JogRun
       AND (SpdAct_Pps > SpdFinal_Pps + SpdBand_Pps) THEN
      SlowMeasActive := TRUE;
      SlowStartPos := Encoder_Pos;
      SlowStartSpd_Pps := SpdAct_Pps;
      SlowStartDiam := DecelDiam;
    END_IF;
    SlowMem := SlowDown;

    IF SlowMeasActive THEN

      IF NOT SlowDown OR ManMode OR JogRun THEN
        SlowMeasActive := FALSE;                (* aborted *)
      ELSIF Tick AND ((SpdAct_Pps <= SpdFinal_Pps + 0.25 * SpdBand_Pps)
                 OR NOT O_MotorStart) THEN      (* final reached or stopped *)
        SlowMeasActive := FALSE;
        SlowMeas_Pls := DINT_TO_REAL(ABS(Encoder_Pos - SlowStartPos));

        IF (SlowMeas_Pls > 0.0) AND (MmPerPulse > 0.0)
          AND ((SlowStartSpd_Pps - SpdAct_Pps) * MmPerPulse * 0.06 >= 10.0) THEN
          SlowDecelMeas_mm := (SlowStartSpd_Pps * SlowStartSpd_Pps
                             - SpdAct_Pps * SpdAct_Pps)
                            / (2.0 * SlowMeas_Pls) * MmPerPulse;  (* mm/s2 *)

          IF (SlowDecelMeas_mm >= RwDecelMin_mm)
            AND (SlowDecelMeas_mm >= RwSlowLearnMin_mm)    (* Fix 11/11 *)
            AND (SlowDecelMeas_mm <= RwDecelMax_mm) THEN
            (* v6g: learn per mm of diameter *)
            DecelK_Meas := SlowDecelMeas_mm / ((SlowStartDiam + DecelDiam) /
2.0);

            IF RwSlowDecelK <= 0.0 THEN
              RwSlowDecelK := DecelK_Meas;
            ELSE
              RwSlowDecelK := RwSlowDecelK + 0.5 * (DecelK_Meas -
RwSlowDecelK);
            END_IF;
          END_IF;
        END_IF;
      END_IF;
    END_IF;

    (* ---- 5c. v6g: live decels = learned K x current diameter --------------- *)
    (* v6p: the one-time v6f seed is removed (it re-filled RwStopDecelK from   *)
    (* RwStopDecel, so the learning could never be reset).                    *)
    (* RwLearnReset (HMI / Watch tab, PLC resets it): clear the learned decels. *)
    (* Both are re-learned at the next auto slow-down / stop >= RwMinLearnSpd. *)
```

```
513
514 IF RwLearnReset THEN
515     RwStopDecelK := 0.0;
516     RwStopDecel := 0.0;
517     RwSlowDecelK := 0.0;
518     StopFailCnt := 0;
519     RwLearnReset := FALSE;
520 END_IF;
521
522 IF (RwStopDecelK > 0.0) AND (MmPerPulse > 0.0) THEN
523     RwStopDecel := RwStopDecelK * DecelDiam / MmPerPulse;     (* pulses/s2 *)
524 END_IF;
525
526 IF RwSlowDecelK > 0.0 THEN
527     RwSlowDecelLrn := RwSlowDecelK * DecelDiam;               (* mm/s2 *)
528 ELSE
529     RwSlowDecelLrn := 0.0;
530 END_IF;
```

1 Auto Stop Process 6-14

```
(*
 ================================================================
 ====================
   AutoStop_Control  V12  -  PART 2 of 2 (rung 1): sections 6 - 14c
   V12: END-OF-ROLL STOP RAMP: auto / diameter stops ramp down at
 FinalStopDecel
       (soft stop mechanism) instead of the drive DEC ramp (3 s = 20 Hz/s,
       up to ~8 m/s2 of web on a big roll): stop lengths use it (6, 11b).
   V11: 3-state unwinder display + stop command at UwStop_at_D_Val (section 8).
   (Sysmac: an inline ST rung holds max 1000 rows -> split in 2 rungs.
    Rung 0 = part 1 runs first, then this rung, every scan.)

 ================================================================
 ==================== *)

(* ---- 6. Lengths in pulses: stop, slow-down, remaining ---------- *)
(* Stop length uses the ACTUAL speed (precision)                    *)

IF RwStopDecel > 0.0 THEN
   StopLen_Pls := SpdAct_Pps * RwTreact
               + (SpdAct_Pps * SpdAct_Pps) / (2.0 * RwStopDecel);
ELSE
   StopLen_Pls := SpdAct_Pps * (RwTreact + RwTstopDef / 2.0);
END_IF;

(* V12: end-of-roll stop ramp at FinalStopDecel (web mm/s2) + S-curve    *)

IF (FinalStopDecel > 0.0) AND FFEnable AND NOT ManMode AND (MmPerPulse >
 0.0) THEN
   StopLen_Pls := SpdAct_Pps * RwTreact
               + (SpdAct_Pps * SpdAct_Pps) / (2.0 * FinalStopDecel / MmPerPulse)
               + SpdAct_Pps * 0.5 * SCurveTime;
END_IF;

StopLen_Pls := StopLen_Pls * RwTrim;

(* Slow-down uses SpdRef = MAX(actual, work-speed setpoint):            *)
(* stable value, shown to the operator even before start                *)

SlowLenBase_Pls := 0.0;  RwSlowLen_Pls := 0.0;  UwSlowLen_Pls := 0.0;

IF MmPerPulse > 0.0 THEN
   (* v6f: manual > learned > PLC ramp model                           *)
   (* V7: manual > feed-forward (one ramp at DecelWeb, real web mm/s2)  *)
   (*     > old loop with web ramps (coarse DecelWeb + trim tail)      *)
   (*     > learned > old PLC ramp model                               *)

   IF RwSlowDecel > 0.0 THEN
      SlowDecel_Pps2 := RwSlowDecel / MmPerPulse;
      SlowOneRamp := TRUE;
   ELSIF FFEnable AND (DecelWeb > 0.0) AND (WebRatio > 0.0) AND NOT
 ManMode THEN
      SlowDecel_Pps2 := DecelWeb / MmPerPulse;
      SlowOneRamp := TRUE;
   ELSIF (DecelWeb > 0.0) AND (WebRatio > 0.0) AND NOT ManMode THEN  (* V7e
 *)
      SlowDecel_Pps2 := DecelWeb / MmPerPulse;
      SlowOneRamp := FALSE;
```

```
51    ELSIF RwSlowDecelLrn > 0.0 THEN
52       SlowDecel_Pps2 := RwSlowDecelLrn / MmPerPulse;  (* real, incl. trim tail *)
53       SlowOneRamp := TRUE;
54    ELSE
55       SlowDecel_Pps2 := DecelCoarse * 250.0 / MmPerPulse; (* 1 = 250 mm/s2 *)
56       SlowOneRamp := FALSE;
57    END_IF;
58
59    (* v6d: DecelTrim ramp used inside the +/- SpdDelta_Percent band *)
60
61    IF (DecelTrim > 0.0) AND (WebRatio > 0.0) AND (Vel_Max_mms > 0.0) THEN
62       (* V7: trim ramp (reference units) converted to the real web decel *)
63       TrimDecel_Pps2 := DecelTrim * 1000000.0 / (0.06 * WebRatio * Vel_Max_mms)
64                         / MmPerPulse;
65    ELSIF DecelTrim > 0.0 THEN
66       TrimDecel_Pps2 := DecelTrim * 250.0 / MmPerPulse;
67    ELSE
68       TrimDecel_Pps2 := SlowDecel_Pps2;
69    END_IF;
70
71    SpdBand_Pps := SpdDelta_Percent * 1000.0 / 60.0 / MmPerPulse;
72
73    (* v6d: speeds from the potentiometers, valid in auto AND ManMode     *)
74    (*      (m/min = Pot / Input_Ratio, same as Motor_Manager rung 8)     *)
75
76    IF (Input_Ratio > 0.0) AND ManMode THEN
77       (* v6i: ManMode drives the inverter with the raw pots (rung 17) *)
78       SpdFinal_Pps := INT_TO_REAL(Pot2_Slow_Velocity) / Input_Ratio * 1000.0 /
   60.0 / MmPerPulse;
79       SpdWork_Pps  := INT_TO_REAL(Pot1_Work_Velocity) / Input_Ratio * 1000.0 /
   60.0 / MmPerPulse;
80    ELSIF Input_Ratio > 0.0 THEN
81       SpdFinal_Pps := INT_TO_REAL(Pot2_Mem) / Input_Ratio * 1000.0 / 60.0 /
   MmPerPulse;
82       SpdWork_Pps  := INT_TO_REAL(Pot1_Mem) / Input_Ratio * 1000.0 / 60.0 /
   MmPerPulse;
83    ELSE
84       SpdFinal_Pps := LowSpd_Display * 1000.0 / 60.0 / MmPerPulse;
85       SpdWork_Pps  := Speed_Display  * 1000.0 / 60.0 / MmPerPulse;
86    END_IF;
87
88    (* v6y: rewinder AUTO stop -> work speed limited to RwAutoMaxSpdAct *)
89    (* (limit + value taken over only at standstill, Motor_Manager rung 5) *)
90
91    IF RwAutoLimAct AND NOT ManMode
92       AND (SpdWork_Pps > RwAutoMaxSpdAct * 1000.0 / 60.0 / MmPerPulse) THEN
93       SpdWork_Pps := RwAutoMaxSpdAct * 1000.0 / 60.0 / MmPerPulse;
94    END_IF;
95
96    SpdRef_Pps := MAX(SpdAct_Pps, SpdWork_Pps);
97
98    IF RwStopDecel > 0.0 THEN
99       StopLenFinal_Pls := SpdFinal_Pps * RwTreact
100                         + (SpdFinal_Pps * SpdFinal_Pps) / (2.0 * RwStopDecel);
101   ELSE
102      StopLenFinal_Pls := SpdFinal_Pps * (RwTreact + RwTstopDef / 2.0);
103   END_IF;
104
105   (* V12: final stop from the final speed on the end-of-roll ramp *)
```

```
106
107   IF (FinalStopDecel > 0.0) AND FFEnable AND NOT ManMode THEN
108      StopLenFinal_Pls := SpdFinal_Pps * RwTreact
109                        + (SpdFinal_Pps * SpdFinal_Pps) / (2.0 * FinalStopDecel /
      MmPerPulse)
110                        + SpdFinal_Pps * 0.5 * SCurveTime;
111   END_IF;
112
113   SlowLenBase_Pls := StopLenFinal_Pls * RwTrim;
114
115   IF (SpdRef_Pps > SpdFinal_Pps) AND (SlowDecel_Pps2 > 0.0) THEN
116
117      IF SlowOneRamp OR (TrimDecel_Pps2 <= 0.0) THEN
118         (* manual or learned decel: one constant ramp *)
119         SlowLenBase_Pls := SlowLenBase_Pls
120            + (SpdRef_Pps * SpdRef_Pps - SpdFinal_Pps * SpdFinal_Pps)
121             / (2.0 * SlowDecel_Pps2);
122      ELSE
123         (* auto: coarse ramp down to the band, trim ramp inside it *)
124         SpdKnee_Pps := SpdFinal_Pps + SpdBand_Pps;
125
126         IF SpdRef_Pps > SpdKnee_Pps THEN
127            SlowLenBase_Pls := SlowLenBase_Pls
128               + (SpdRef_Pps * SpdRef_Pps - SpdKnee_Pps * SpdKnee_Pps)
129                / (2.0 * SlowDecel_Pps2)
130               + (SpdKnee_Pps * SpdKnee_Pps - SpdFinal_Pps * SpdFinal_Pps)
131                / (2.0 * TrimDecel_Pps2);
132         ELSE
133            SlowLenBase_Pls := SlowLenBase_Pls
134               + (SpdRef_Pps * SpdRef_Pps - SpdFinal_Pps * SpdFinal_Pps)
135                / (2.0 * TrimDecel_Pps2);
136         END_IF;
137      END_IF;
138   END_IF;
139
140   (* V7f: S-curve rounding: the slow-down ends SCurveTime later ->      *)
141   (*      extra length = (SpdRef - SpdFinal) / 2 x SCurveTime          *)
142
143   IF FFEnable AND NOT ManMode AND (RwSlowDecel <= 0.0) AND (DecelWeb >
      0.0)
144      AND (WebRatio > 0.0) AND (SCurveTime > 0.0)
145      AND (SpdRef_Pps > SpdFinal_Pps) THEN
146      SlowLenBase_Pls := SlowLenBase_Pls
147                        + (SpdRef_Pps - SpdFinal_Pps) * 0.5 * SCurveTime;
148   END_IF;
149
150   RwSlowLen_Pls := SlowLenBase_Pls + RwSlowMargin_m * 1000.0 / MmPerPulse;
151
152   IF UwSlowMargin_m > 0.0 THEN
153      UwMarginUsed_m := UwSlowMargin_m;
154   ELSE
155      UwMarginUsed_m := RwSlowMargin_m;
156   END_IF;
157
158   UwSlowLen_Pls := SlowLenBase_Pls + UwMarginUsed_m * 1000.0 / MmPerPulse;
159 END_IF;
160
161 IF RwDiam >= RwTargetDiam THEN
```

```
ELSIF RwMat_K > 0.0 THEN
    RwRemain_Pls := (RwTargetDiam * RwTargetDiam - RwDiam * RwDiam)
                  / (2.0 * RwMat_K);
ELSE
    RwRemain_Pls := 1.0E9;                    (* K not learned: plain target *)
END_IF;

RwRemain_m  := RwRemain_Pls  * MmPerPulse / 1000.0;
RwStopLen_m := StopLen_Pls   * MmPerPulse / 1000.0;
RwSlowLen_m := RwSlowLen_Pls * MmPerPulse / 1000.0;

(* ---- 6b. Live operator displays ---------------------------------------- *)
(* Rewinder: diameter where slow-down / stop command will happen          *)

IF RwMat_K > 0.0 THEN
    SqrtArg := RwTargetDiam * RwTargetDiam - 2.0 * RwMat_K * RwSlowLen_Pls;

    IF SqrtArg > 0.0 THEN
        RwSlow_at_D_Val := SQRT(SqrtArg);
    ELSE
        RwSlow_at_D_Val := 0.0;
    END_IF;
    SqrtArg := RwTargetDiam * RwTargetDiam - 2.0 * RwMat_K * StopLen_Pls;

    IF SqrtArg > 0.0 THEN
        RwStop_at_D_Val := SQRT(SqrtArg);
    ELSE
        RwStop_at_D_Val := 0.0;
    END_IF;
ELSE
    (* v6t: thickness not learned: fixed slow-down point before the target *)

    IF RwUseSlow AND (RwSlowFallbackMm > 0.0) THEN
        RwSlow_at_D_Val := RwTargetDiam - RwSlowFallbackMm;
    ELSE
        RwSlow_at_D_Val := RwTargetDiam;
    END_IF;
    RwStop_at_D_Val := RwTargetDiam;
END_IF;

(* Unwinder: meter count where the slow-down will start                   *)

IF MmPerPulse > 0.0 THEN
    UwSlow_at_M_Val := UwStop_at_M_Val - UwSlowLen_Pls * MmPerPulse / 1000.0;
ELSE
    UwSlow_at_M_Val := UwStop_at_M_Val;
END_IF;

IF UwSlow_at_M_Val < 0.0 THEN
    UwSlow_at_M_Val := 0.0;
END_IF;

(* Unwinder slow-down request, latched                                    *)

IF (Current_Metre < UwSlow_at_M_Val - 50.0) OR (UwStop_at_M_Val <= 0.0) OR
CntResetPend THEN
    UwSlow_at_M_Req := FALSE;                 (* counter reset / longer roll *)
END_IF;
```

```
220
221 IF (UwStop_at_M_Val > 0.0) AND (Current_Metre >= UwSlow_at_M_Val) AND NOT
    CntResetPend THEN
222     UwSlow_at_M_Req := TRUE;
223 END_IF;
224
225 (* ---- 6c. V7c: unwinder DIAMETER slow-down follows the speed ---------- *)
226 (* Works without AutoStop (only Measure_ON, like General rung 16).       *)
227 (* Paper left above the stop diameter, in pulses:                         *)
228 (*     (UwDiam_Act^2 - UwStopDiam_Limit^2) / (2 x RwMat_K)                *)
229 (* (same paper as the rewinder -> same learned K). Slow down when it is   *)
230 (* shorter than UwSlowLen_Pls = decel from the speed (DecelWeb) + final   *)
231 (* stop + UwSlowMargin_m. Solved for the diameter:                        *)
232 (*     UwSlowDiamCalc = SQRT(UwStopDiam_Limit^2 + 2 x RwMat_K x
    UwSlowLen_Pls)   *)
233 (* K not learned / UwSlow_at_D_Req OFF -> fixed UwSlow_at_D_Val as before. *)
234 (* Latched: once slowed down, the point is frozen (+10 mm hysteresis), so  *)
235 (* the lower speed does not move it back and speed up again. Released by   *)
236 (* a new roll (diameter 30 mm above the frozen point).                     *)
237
238 IF UwSlow_at_D_Req AND (RwMat_K > 0.0) AND (UwStopDiam_Limit > 0.0)
239    AND (UwSlowLen_Pls > 0.0) THEN
240     SqrtArg := UwStopDiam_Limit * UwStopDiam_Limit + 2.0 * RwMat_K *
    UwSlowLen_Pls;
241     UwSlowDiamCalc := SQRT(SqrtArg);            (* same pattern as 6b *)
242 ELSE
243     UwSlowDiamCalc := UwSlow_at_D_Val;
244 END_IF;
245
246 IF UwSlowDiamCalc < UwStopDiam_Limit + 10.0 THEN
247     UwSlowDiamCalc := UwStopDiam_Limit + 10.0;
248 ELSIF UwSlowDiamCalc > 1500.0 THEN
249     UwSlowDiamCalc := 1500.0;
250 END_IF;
251
252 IF UwSlowDiamLatch AND ((UwDiam_Act > UwDiamAtLatch + 30.0)
253    OR (NOT StartMotor_Cmd AND (UwDiam_Act > UwSlowDiamCalc + 30.0))) THEN
254     UwSlowDiamLatch := FALSE;                   (* new roll / point moved. Fix 11/11 *)
255 END_IF;
256
257 IF NOT UwSlowDiamLatch THEN
258     UwSlowDiamAct := UwSlowDiamCalc;            (* live point, HMI *)
259
260     IF Measure_ON AND StartMotor_Cmd AND (UwDiam_Act > 0.0)
261       AND (UwDiam_Act <= UwSlowDiamCalc) THEN
262         UwSlowDiamLatch := TRUE;
263         UwDiamAtLatch := UwDiam_Act;            (* diameter when latched *)
264         UwSlowDiamAct := UwSlowDiamCalc + 10.0;    (* frozen + hysteresis *)
265     END_IF;
266 END_IF;
267
268 (* ---- 7. Rewinder requests: latched, reset when a new core is loaded ---- *)
269
270 IF NOT RwStopEnable
271    OR ((RwDiam < RwTargetDiam - RwResetHyst) AND NOT RwSensFault) THEN
272     RwStop_at_D_Req := FALSE;
273     RwSlow_at_D_Req := FALSE;
274 END_IF;
275
```

```
IF RwStopEnable THEN

    IF (RwRemain_Pls <= StopLen_Pls) OR (RwSensFault AND RwStopOnFault) THEN
        RwStop_at_D_Req := TRUE;
    END_IF;

    IF RwUseSlow AND (RwRemain_Pls <= RwSlowLen_Pls) THEN
        RwSlow_at_D_Req := TRUE;
    END_IF;

    (* v6t: thickness not learned (RwMat_K = 0): slow down at a fixed     *)
    (* distance before the target instead of stopping from full speed.    *)

    IF RwUseSlow AND (RwMat_K <= 0.0) AND (RwSlowFallbackMm > 0.0)
       AND NOT RwSensFault
       AND (RwDiam >= RwTargetDiam - RwSlowFallbackMm) THEN
        RwSlow_at_D_Req := TRUE;
    END_IF;
END_IF;

(* ---- 8. One AUTO at a time + single "SLOW DOWN AT" display ------------- *)
// ---- Refuse AUTO STOP when its target is already reached (HMI warning 21) ----
// AutoEnWhy: 1 = unwinder metres already reached, 2 = rewinder roll >= target

IF I_AutoStop_OS AND RwStopEnable AND NOT RwArmMem AND NOT RwSensFault
   AND (RwTargetDiam > REAL#0.0) AND (RwDiam >= RwTargetDiam -
RwResetHyst) THEN
    RwStopEnable := FALSE;                  // refused
    AutoEnRefused := TRUE;
    AutoEnWhy := 2;
END_IF;

IF I_AutoStop_OS AND UwStopEnable AND NOT UwArmMem AND NOT
CntResetPend
   AND (UwStop_at_M_Val > REAL#0.0) AND (Current_Metre >= UwStop_at_M_Val)
THEN
    UwStopEnable := FALSE;                  // refused
    AutoEnRefused := TRUE;
    AutoEnWhy := 1;
END_IF;

IF UwStopEnable AND NOT UwEnMem THEN            (* unwinder AUTO switched
ON *)
    RwStopEnable := FALSE;
END_IF;

IF RwStopEnable AND NOT RwEnMem THEN            (* rewinder AUTO switched
ON *)
    UwStopEnable := FALSE;
END_IF;

T_AutoEnRef(In := AutoEnRefused, PT := T#5s);     // warning shown 5 s

IF T_AutoEnRef.Q OR I_RsetAlarm OR Alarm_Reset THEN
    AutoEnRefused := FALSE;
END_IF;

RwArmMem := I_AutoStop_OS AND RwStopEnable;       // armed = selector 12SA5
AND enable
```

```
329  UwArmMem := I_AutoStop_OS AND UwStopEnable;
330  UwEnMem := UwStopEnable;
331  RwEnMem := RwStopEnable;
332
333  IF RwStopEnable THEN
334      SlowAtDisp := RwSlow_at_D_Val;                (* mm *)
335      SlowAtMode := 2;
336  ELSIF UwStopEnable THEN
337      SlowAtDisp := UwSlow_at_M_Val;                (* m  *)
338      SlowAtMode := 1;
339  ELSE
340      SlowAtDisp := 0.0;
341      SlowAtMode := 0;                          (* hide display *)
342  END_IF;
343
344  SlowAtShow := (SlowAtMode <> 0);
345
346  (* V11: unwinder column, same place, 3 states:                          *)
347  (*   AUTO STOP UNWINDER (12SA5 + unwinder AUTO) -> STOP LENGTH [m]      *)
348  (*   normal run, before the slow-down          -> SLOW AT Ø [mm]        *)
349  (*   slow-down reached (decel / final speed)   -> STOP AT Ø [mm]        *)
350  (* V11: stop by diameter EARLIER, so the web is stopped AT UwStopDiam_Limit:  *)
351  (* the stop command comes at UwStop_at_D_Val = UwStopDiam_Limit + the
     diameter   *)
352  (* of the paper unwound during the stop from the ACTUAL speed           *)
353  (* (StopLen_Pls, section 6, learned stop decel):                        *)
354  (*   UwStop_at_D_Val = SQRT(UwStopDiam_Limit^2 + 2 x RwMat_K x StopLen_Pls)
     *)
355  (* General rung 18 (UwStopByDiam) compares with UwStop_at_D_Val.        *)
356  (* K not learned -> UwStopDiam_Limit (as before). Max UwStopDiam_Limit + 100
     mm.    *)
357
358  UwStopLenShow := I_AutoStop_OS AND (UwStopEnable OR (AutoEnRefused AND
     (AutoEnWhy = 1)));
359  UwSlowDiamShow := NOT UwStopLenShow AND Measure_ON AND NOT
     UwSlowDiamLatch;
360  UwStopDiamShow := NOT UwStopLenShow AND Measure_ON AND
     UwSlowDiamLatch;
361
362  IF (RwMat_K > 0.0) AND (StopLen_Pls > 0.0) THEN
363      SqrtArg := UwStopDiam_Limit * UwStopDiam_Limit + 2.0 * RwMat_K *
     StopLen_Pls;
364      UwStop_at_D_Val := SQRT(SqrtArg);
365  ELSE
366      UwStop_at_D_Val := UwStopDiam_Limit;
367  END_IF;
368
369  IF UwStop_at_D_Val > UwStopDiam_Limit + 100.0 THEN
370      UwStop_at_D_Val := UwStopDiam_Limit + 100.0;
371  ELSIF UwStop_at_D_Val < UwStopDiam_Limit THEN
372      UwStop_at_D_Val := UwStopDiam_Limit;
373  END_IF;
374
375  (* ---- 9. Automatic stop master = selector 12SA5 (I_AutoStop_OS) -------- *)
376
377  AutoSlowCmd := I_AutoStop_OS AND
378              ((UwStopEnable AND UwSlow_at_M_Req) OR (RwStopEnable AND
     RwSlow_at_D_Req));
379
```

```
380  AutoStopCmd := I_AutoStop_OS AND
381              ((UwStopEnable AND (UwStop_at_M_Val > 0.0)
382               AND (Current_Metre >= UwStop_at_M_Val) AND NOT CntResetPend)
383              OR (RwStopEnable AND RwStop_at_D_Req));
384
385 (* ---- 10. Unwinder roll empty (for alarm 15) ------------------------------ *)
386
387 IF UwDiam_Act > (UwStopDiam_Limit + UwEmptyHyst) THEN
388     UnWinder_Empty := FALSE;                    (* new roll loaded *)
389 END_IF;
390
391 IF (UwDiam_Act > 0.0) AND (UwDiam_Act <= UwStopDiam_Limit) THEN
392     UnWinder_Empty := TRUE;                     (* roll empty *)
393 END_IF;
394
395 (* v6x: new parent roll loaded (empty -> not empty) -> HMI popup        *)
396 (* "DID YOU CHANGE THE PAPER (GSM)?". YES = forget the learned thickness *)
397 (* (same as NEW PAPER). NO = the HMI resets GsmAsk.                     *)
398 (* V9: ask only ONCE per REAL new roll: the roll must have been empty for *)
399 (* 3 s (arms the question), then the new roll must read more than       *)
400 (* UwStopDiam_Limit + 100 mm for 3 s. The sonar bouncing around
    UwStopDiam_Limit    *)
401 (* (empty core, hands, arms) no longer re-opens the popup.              *)
402
403 IF Tick THEN
404
405     IF UnWinder_Empty THEN
406
407         IF UwEmptyTicks < 1000 THEN
408             UwEmptyTicks := UwEmptyTicks + 1;
409         END_IF;
410
411         IF UwEmptyTicks >= 30 THEN
412             GsmArmed := TRUE;
413         END_IF;
414         UwFullTicks := 0;
415     ELSE
416         UwEmptyTicks := 0;
417
418         IF GsmArmed AND Measure_ON AND (UwDiam_Act > UwStopDiam_Limit +
    100.0) THEN
419             UwFullTicks := UwFullTicks + 1;
420
421             IF UwFullTicks >= 30 THEN
422                 GsmAsk := TRUE;
423                 GsmArmed := FALSE;
424                 UwFullTicks := 0;
425             END_IF;
426         ELSE
427             UwFullTicks := 0;
428         END_IF;
429     END_IF;
430 END_IF;
431
432 UwEmptyMem := UnWinder_Empty;
433
434 IF GsmAskYes THEN
435     RwMat_K := 0.0;
436     RwMatLocked := FALSE;                       (* V9 *)
```

```
437     LongActive := FALSE;
438     LearnFailCnt := 0;
439     MatChangeCnt := 0;
440     GsmAskYes := FALSE;
441     GsmAsk := FALSE;
442   END_IF;
443
444   (* V10: HMI popup "UNWINDER DIAMETER SENSOR SONAR IS OFF ! DO YOU WANT
        TO  *)
445   (* TURN IT ON ?" - only at standstill: at power-up and after every stop    *)
446   (* while Measure_ON = FALSE. YES (SonarAskYes, PLC resets it) = sonar ON:  *)
447   (* NO = the HMI resets SonarAsk (asked again after the next stop).        *)
448
449   IF Measure_ON THEN
450     SonarAsk := FALSE;                    (* sonar ON: no popup *)
451   ELSIF NOT StartMotor_Cmd AND (SonarRunMem OR NOT SonarInit) THEN
452     SonarAsk := TRUE;                     (* stopped / power-up *)
453   END_IF;
454
455   SonarInit := TRUE;
456   SonarRunMem := StartMotor_Cmd;
457
458   IF StartMotor_Cmd THEN
459     SonarAsk := FALSE;                    (* never while running *)
460   END_IF;
461
462   IF SonarAskYes THEN
463     Measure_ON := TRUE;
464     SonarAsk := FALSE;
465     SonarAskYes := FALSE;
466   END_IF;
467
468   (* ---- 11. Rewinder max diameter protection - ALWAYS active ---------- *)
469   (* Independent of selector 12SA5 and of the AUTO buttons.                 *)
470   (* Mechanical limit 680 mm radius = 1360 mm                               *)
471
472   IF NOT RwSensFault THEN
473     RwLastValidDiam := RwDiam;
474   END_IF;
475
476   IF RwDiam >= RwMaxDiam THEN
477     RwRemainMax_Pls := 0.0;
478   ELSIF RwMat_K > 0.0 THEN
479     RwRemainMax_Pls := (RwMaxDiam * RwMaxDiam - RwDiam * RwDiam)
480                        / (2.0 * RwMat_K);
481   ELSE
482     RwRemainMax_Pls := 1.0E9;             (* K not learned: plain limit *)
483   END_IF;
484
485   (* Display: diameter where the max-protection slow-down will start        *)
486
487   IF RwMat_K > 0.0 THEN
488     SqrtArg := RwMaxDiam * RwMaxDiam - 2.0 * RwMat_K * RwSlowLen_Pls;
489
490     IF SqrtArg > 0.0 THEN
491       RwMaxSlowAtDiam := SQRT(SqrtArg);
492     ELSE
493       RwMaxSlowAtDiam := 0.0;
494     END_IF;
```

```
495  ELSIF RwMaxSlowFallback > 0.0 THEN
496      RwMaxSlowAtDiam := RwMaxSlowFallback;        (* fallback in use *)
497  ELSE
498      RwMaxSlowAtDiam := RwMaxDiam;
499  END_IF;
500
501  (* Reset only after the roll is removed (diameter well below the limit)  *)
502
503  IF (RwDiam < RwMaxDiam - RwResetHyst) AND NOT RwSensFault THEN
504      RwMaxStop := FALSE;
505      RwMaxSlow := FALSE;
506  END_IF;
507
508  IF NOT RwSensFault THEN
509
510      IF RwRemainMax_Pls <= StopLen_Pls THEN
511          RwMaxStop := TRUE;                    (* speed-compensated stop *)
512      END_IF;
513
514      IF RwRemainMax_Pls <= RwSlowLen_Pls THEN
515          RwMaxSlow := TRUE;                    (* warning slow-down + horn *)
516      END_IF;
517
518      IF (RwMat_K <= 0.0) AND (RwMaxSlowFallback > 0.0)
519        AND (RwDiam >= RwMaxSlowFallback) THEN
520          RwMaxSlow := TRUE;                    (* fallback while K not learned *)
521      END_IF;
522
523  ELSIF (AI_Diam_Rewinder < RwRawMin) AND (RwLastValidDiam > RwMaxDiam -
     100.0) THEN
524      RwMaxStop := TRUE;                        (* roll grew into the DT35 near zone *)
525  END_IF;
526
527  (* Max diameter protection added OUTSIDE the 12SA5 master (section 9)     *)
528
529  AutoSlowCmd := AutoSlowCmd OR RwMaxSlow;
530  AutoStopCmd := AutoStopCmd OR RwMaxStop;
531
532  (* ---- 11b. V12: END-OF-ROLL STOP RAMP ---------------------------------- *)
533  (* Normal end-of-roll stops (AutoStopCmd: unwinder metres, rewinder target,*)
534  (* rewinder max diameter; UwStopByDiam: unwinder diameter) no longer drop  *)
535  (* the run command at once (drive DEC ramp 3 s = 20 Hz/s -> up to 8 m/s2  *)
536  (* of web on a big roll, hard on the rewinder gearbox). With the feed-    *)
537  (* forward in AUTO they start the SOFT STOP at FinalStopDecel (General    *)
538  (* rung 10): the run command drops below SoftStopEndSpd. START cannot     *)
539  (* cancel it. The stop lengths (section 6) use the same ramp, so the web  *)
540  (* still stops at the target / UwStopDiam_Limit.                          *)
541  (* Immediate stop (EndStopNow -> Motor_Manager rung 4 Stop):              *)
542  (*   ramp not usable (FinalStopDecel 0, FFEnable OFF, ManMode, JOG, the   *)
543  (*   motor stopped = start blocked as before), or safety net: unwinder    *)
544  (*   10 mm below UwStopDiam_Limit / rewinder at RwMaxDiam during the ramp.*)
545
546  IF FinalStopDecel < 0.0 THEN
547      FinalStopDecel := 0.0;
548  ELSIF (FinalStopDecel > 0.0) AND (FinalStopDecel < 50.0) THEN
549      FinalStopDecel := 50.0;
550  ELSIF FinalStopDecel > 2000.0 THEN
551      FinalStopDecel := 2000.0;
552  END_IF;
```

```
553
554  EndStopReq := AutoStopCmd OR (Measure_ON AND UwStopByDiam);
555  EndRampOK := (FinalStopDecel > 0.0) AND FFActive AND StartMotor_Cmd;
556
557  IF EndStopReq AND EndRampOK THEN
558      FinalStopRamp := TRUE;                    (* -> soft stop, General 10 *)
559  END_IF;
560
561  IF NOT StartMotor_Cmd THEN
562      FinalStopRamp := FALSE;
563  END_IF;
564
565  EndRampHard := FinalStopRamp
566              AND ((Measure_ON AND (UwDiam_Act > 0.0)
567                  AND (UwDiam_Act < UwStopDiam_Limit - 10.0))
568                  OR ((RwDiam >= RwMaxDiam) AND NOT RwSensFault));
569
570  EndStopNow := (EndStopReq AND NOT FinalStopRamp) OR EndRampHard;
571
572  // ---- 11c. Auto stop DONE (HMI warning 20) ----
573  // AutoStopWhy: 1 unwinder metres, 2 rewinder diameter, 4 rewinder MAX, 8
     unwinder diameter
574
575  IF StartMotor_Cmd AND NOT AS_RunMem THEN          // new START: clear
576      AutoStopDone := FALSE;
577      AutoStopSeen := FALSE;
578      AutoStopWhy := 0;
579  END_IF;
580
581  IF StartMotor_Cmd AND EndStopReq AND NOT AutoStopSeen AND NOT JogRun
     THEN
582      AutoStopSeen := TRUE;                     // auto stop started while running
583
584      IF RwMaxStop THEN
585          AutoStopWhy := 4;                     // rewinder MAX (protection, highest)
586      ELSIF I_AutoStop_OS AND RwStopEnable AND RwStop_at_D_Req THEN
587          AutoStopWhy := 2;                     // rewinder diameter reached
588      ELSIF Measure_ON AND UwStopByDiam THEN
589          AutoStopWhy := 8;                     // unwinder diameter (sonar)
590      ELSE
591          AutoStopWhy := 1;                     // unwinder metres
592      END_IF;
593  END_IF;
594
595  IF AS_RunMem AND NOT StartMotor_Cmd AND AutoStopSeen THEN
596      AutoStopDone := TRUE;                     // stopped by the auto stop
597  END_IF;
598
599  IF I_RsetAlarm OR Alarm_Reset THEN
600      AutoStopDone := FALSE;
601  END_IF;
602
603  T_AutoStopDisp(In := AutoStopDone, PT := T#8s);   // show the warning 8 s
604
605  IF T_AutoStopDisp.Q THEN
606      AutoStopDone := FALSE;
607  END_IF;
608
609  AS_RunMem := StartMotor_Cmd;
```

```
610
611 (* ---- 12. Learning supervision -> alarm 16 (section Alarms) ------------ *)
612 (* Learned paper thickness, for display / plausibility check              *)
613
614 IF MmPerPulse > 0.0 THEN
615     RwThick_mm := RwMat_K * 3.14159 / (2.0 * MmPerPulse);
616 ELSE
617     RwThick_mm := 0.0;
618 END_IF;
619
620 (* Fault: too many consecutive windows / stops that should have          *)
621 (* learned but did not. Clears by itself on the next good result.       *)
622 (* v6w: ALARM RESET (panel button or HMI) also clears the fail counters. *)
623 (* so alarm 16 resets in one press. It comes back only if the learning  *)
624 (* fails again RwLearnFailMax / RwStopFailMax times in a row.           *)
625
626 IF I_RsetAlarm OR Alarm_Reset THEN
627     LearnFailCnt := 0;
628     StopFailCnt := 0;
629 END_IF;
630
631 GSM_LearnFault := (LearnFailCnt >= RwLearnFailMax)
632             OR (StopFailCnt >= RwStopFailMax);      (* -> alarm 16 *)
633
634 (* v6e: learning status for the HMI. FALSE until learned.                *)
635
636 GSM_Learned_OK := (RwMat_K > 0.0)
637               AND ((RwStopDecel > 0.0) OR NOT RwLearnStop)
638               AND NOT GSM_LearnFault;
639
640 (* ---- 13. v6h: keep T-one in automatic tension control during the stop - *)
641 (* While running: BrakeHold = Brake_Cmd (no change in running behaviour).  *)
642 (* After the stop: stays ON until the web is below 5 m/min, after 15 s.    *)
643 (* or on E-stop. Already TRUE on the stop scan (Output_Manager runs before *)
644 (* this section). so O_Brake never drops for one scan.                     *)
645 (* Output_Manager rung 2:  O_Brake := Brake_Cmd OR BrakeHold               *)
646
647 IF O_MotorStart THEN
648     BrakeHold := Brake_Cmd AND ControlVoltage;
649     BrakeHoldTicks := 0;
650 ELSIF BrakeHold THEN
651
652     IF Tick THEN
653         BrakeHoldTicks := BrakeHoldTicks + 1;
654     END_IF;
655
656     (* v6v: signed web speed: a web pulled BACK (negative speed) also     *)
657     (* releases the T-one (trace 10/28 15:00: -6.6 m/min, about 20 cm).  *)
658
659     IF NOT ControlVoltage OR (Velocity_M_min < 5.0) OR (BrakeHoldTicks >= 150)
    THEN
660         BrakeHold := FALSE;
661     END_IF;
662 END_IF;
663
664 (* ---- 14. v6j: TEACH CORE - automatic offset with an empty core -------- *)
665 (* HMI: set RwTeachCore (button, Set ON). Conditions: motor stopped, web  *)
666 (* not moving, DT35 OK. After 2 s stable (filter settled):                 *)
```

```
    *)
(* RwTeachState: 0 idle, 1 teaching (wait 2 s), 2 done OK,              *)
(*               3 refused (motor running / web moving / DT35 fault).   *)
(* Only the offset is taught: the scale is RwDist10V - RwDist0V         *)
(* (= real distance between the two DT35 taught points).                *)

IF RwTeachCore THEN

   IF O_MotorStart OR (RwSpeed > 1.0) OR RwSensFault
     OR NOT RwFiltInit OR (RwCoreDiameter <= 0.0) THEN
      RwTeachState := 3;                    (* refused *)
      RwTeachCore := FALSE;
      TeachTicks := 0;
   ELSE
      RwTeachState := 1;                    (* teaching *)

      IF Tick THEN
         TeachTicks := TeachTicks + 1;
      END_IF;

      IF TeachTicks >= 20 THEN              (* 2 s stable *)
         RwAxisDist_Prev := RwAxisDist;     (* v6l: keep previous *)
         RwAxisDist := RwDist + RwCoreDiameter / 2.0;
         RwTeachState := 2;                 (* done *)
         RwTeachCore := FALSE;
         TeachTicks := 0;
      END_IF;
   END_IF;
ELSE
   TeachTicks := 0;

   IF O_MotorStart THEN
      RwTeachState := 0;                    (* clear status on next run *)
   END_IF;
END_IF;

(* ---- 14b. v6k: unwinder TEACH CORE (HMI button Core_Measure) ---------- *)
(* Same behaviour as section 14. Sonar is not filtered, so the 2 s are   *)
(* averaged:  UwCoreDistance := average(UwDiam_Measure)  (sonar -> core  *)
(* surface). UwDiam_Act = (UwCoreDistance - UwDiam_Measure) * 2          *)
(*                      + UwCoreDiameter   (General rung 14, unchanged). *)
(* UwTeachState: 0 idle, 1 teaching, 2 done OK, 3 refused.              *)

IF Core_Measure THEN

   IF O_MotorStart OR (RwSpeed > 1.0) OR (UwCoreDiameter <= 0.0)
     OR (UwDiam_Measure <= 0.0) THEN
      UwTeachState := 3;                    (* refused *)
      Core_Measure := FALSE;
      UwTeachTicks := 0;
      UwTeachSum := 0.0;
   ELSE
      UwTeachState := 1;                    (* teaching *)

      IF Tick THEN
         UwTeachTicks := UwTeachTicks + 1;
         UwTeachSum := UwTeachSum + UwDiam_Measure;
      END_IF;
```

```
725
726         IF UwTeachTicks >= 20 THEN              (* 2 s, 20 samples *)
727             UwCoreDist_Prev := UwCoreDistance;     (* v6l: keep previous *)
728             UwCoreDistance := UwTeachSum / INT_TO_REAL(UwTeachTicks);
729             UwTeachState := 2;                  (* done *)
730             Core_Measure := FALSE;
731             UwTeachTicks := 0;
732             UwTeachSum := 0.0;
733         END_IF;
734     END_IF;
735 ELSE
736     UwTeachTicks := 0;
737     UwTeachSum := 0.0;
738
739     IF O_MotorStart THEN
740         UwTeachState := 0;                      (* clear status on next run *)
741     END_IF;
742 END_IF;
743
744 (* ---- 14c. v6l: UNDO teach = swap current <-> previous (machine stopped) *)
745 (* Pressing UNDO twice goes back to the taught value. State 4 = RESTORED. *)
746
747 IF RwTeachUndo THEN
748
749     IF NOT O_MotorStart AND (RwAxisDist_Prev > 0.0) THEN
750         TeachSwap := RwAxisDist;
751         RwAxisDist := RwAxisDist_Prev;
752         RwAxisDist_Prev := TeachSwap;
753         RwTeachState := 4;                      (* restored *)
754     ELSE
755         RwTeachState := 3;                      (* refused *)
756     END_IF;
757     RwTeachUndo := FALSE;
758 END_IF;
759
760 IF UwTeachUndo THEN
761
762     IF NOT O_MotorStart AND (UwCoreDist_Prev > 0.0) THEN
763         TeachSwap := UwCoreDistance;
764         UwCoreDistance := UwCoreDist_Prev;
765         UwCoreDist_Prev := TeachSwap;
766         UwTeachState := 4;                      (* restored *)
767     ELSE
768         UwTeachState := 3;                      (* refused *)
769     END_IF;
770     UwTeachUndo := FALSE;
771 END_IF;
```

1-8-1-1-9.Tension_Manager

0

```
// ========== TENSION MANAGER (kg only) ==========
// Section "Tension_Manager" (was Tension_Helper), rung 0, ST, no contact.
// 1. Tension Helper: recommended tension (kg) from paper GSM, roll width, paper type
// 2. T-one remote set point (AO Ch1 -> isolator -> T-one pins 20/19, F.37 = 1):
//    helper x core boost x taper on the rewinder diameter + operator trim (+/-10 kg),
//    x JOG boost while threading (JOG ... START), rate-limited,
//    1..10 V = TensMin_kg..TensMax_kg (T-one F.38 / F.39)
// TensAOEnable = FALSE -> AO Ch1 = fixed 10 V (old Fix10Vdc, pots supply)

// ---- 1. Paper type: KRAFT / BLEACHED selects the factor (N/mm width per 100 g/m2)

IF (TensFactorBleached < REAL#0.1) OR (TensFactorBleached > REAL#2.0) THEN
    TensFactorBleached := REAL#0.54;
END_IF;

IF (TensFactorKraft < REAL#0.1) OR (TensFactorKraft > REAL#2.0) THEN
    TensFactorKraft := REAL#0.80;
END_IF;

IF PaperKraft THEN
    TensFactor := TensFactorKraft;
ELSE
    TensFactor := TensFactorBleached;
END_IF;

// ---- 2. Helper inputs (limits)

IF PaperGSM < REAL#10 THEN
    PaperGSM := REAL#10;
ELSIF PaperGSM > REAL#500 THEN
    PaperGSM := REAL#500;
END_IF;

IF WebWidth_mm < REAL#100 THEN
    WebWidth_mm := REAL#100;
ELSIF WebWidth_mm > REAL#3000 THEN
    WebWidth_mm := REAL#3000;
END_IF;

// ---- 3. Recommended tension (kg)
TensIdeal_N := PaperGSM / REAL#100 * WebWidth_mm * TensFactor;
TensIdeal_Disp := TensIdeal_N / REAL#9.81;

// ---- 4. Setup limits (maintenance popup)

IF TensTaper < REAL#0.0 THEN
    TensTaper := REAL#0.0;
ELSIF TensTaper > REAL#60.0 THEN
    TensTaper := REAL#60.0;
END_IF;

IF TensCoreBoost < REAL#0.0 THEN
    TensCoreBoost := REAL#0.0;
```

```
54  ELSIF TensCoreBoost > REAL#50.0 THEN
55      TensCoreBoost := REAL#50.0;
56  END_IF;
57
58  IF TensJogBoost < REAL#0.0 THEN
59      TensJogBoost := REAL#0.0;
60  ELSIF TensJogBoost > REAL#100.0 THEN
61      TensJogBoost := REAL#100.0;
62  END_IF;
63
64  IF TensRate_kg_s < REAL#1.0 THEN
65      TensRate_kg_s := REAL#1.0;
66  ELSIF TensRate_kg_s > REAL#20.0 THEN
67      TensRate_kg_s := REAL#20.0;
68  END_IF;
69
70  IF TensMin_kg < REAL#0.0 THEN
71      TensMin_kg := REAL#0.0;
72  END_IF;
73
74  IF TensMax_kg < TensMin_kg + REAL#5.0 THEN
75      TensMax_kg := TensMin_kg + REAL#5.0;
76  END_IF;
77
78  IF TensSPMin_kg < TensMin_kg THEN
79      TensSPMin_kg := TensMin_kg;
80  ELSIF TensSPMin_kg > TensMax_kg - REAL#5.0 THEN
81      TensSPMin_kg := TensMax_kg - REAL#5.0;
82  END_IF;
83
84  // ---- 5. Operator trim (HMI up/down, +/-10 kg), back to 0 when the paper changes
85
86  IF TensTrim_kg < REAL#-10.0 THEN
87      TensTrim_kg := REAL#-10.0;
88  ELSIF TensTrim_kg > REAL#10.0 THEN
89      TensTrim_kg := REAL#10.0;
90  END_IF;
91
92  IF (PaperGSM <> TensGSMMem) OR (WebWidth_mm <> TensWidthMem) OR
    (PaperKraft <> TensKraftMem) THEN
93      TensTrim_kg := REAL#0.0;
94  END_IF;
95
96  TensGSMMem := PaperGSM;
97  TensWidthMem := WebWidth_mm;
98  TensKraftMem := PaperKraft;
99
100 // ---- 6. Rewinder diameter fraction 0 (core) .. 1 (max diameter)
101
102 IF (RwMaxDiam > RwCoreDiameter + REAL#50.0) AND NOT RwSensFault THEN
103     TensDiaFrac := (RwDiam - RwCoreDiameter) / (RwMaxDiam - RwCoreDiameter);
104 ELSE
105     TensDiaFrac := REAL#0.0;                    // DT35 fault: no taper
106 END_IF;
107
108 IF TensDiaFrac < REAL#0.0 THEN
109     TensDiaFrac := REAL#0.0;
110 ELSIF TensDiaFrac > REAL#1.0 THEN
111     TensDiaFrac := REAL#1.0;
```

```
112 END_IF;
113
114 // ---- 7. Profile preview (HMI): core / mid / max
115
116 TensAtCore := TensIdeal_Disp * (REAL#1.0 + TensCoreBoost / REAL#100.0);
117 TensAtMid  := TensAtCore * (REAL#1.0 - TensTaper / REAL#100.0 * REAL#0.5);
118 TensAtMax  := TensAtCore * (REAL#1.0 - TensTaper / REAL#100.0);
119
120 // ---- 7b. Threading / knife adjustment preset: JOG raises the tension. START
    resumes normal
121
122 IF JogRun THEN
123     TensJogLatch := TRUE;
124 END_IF;
125
126 IF StartPB_OK AND NOT StartBlocked THEN          // START accepted -> normal
    tension
127     TensJogLatch := FALSE;
128 END_IF;
129
130 // ---- 8. Target: profile (taper) + trim, or manual; x JOG boost while threading
131
132 IF TensAuto THEN
133     TensProfile := TensAtCore * (REAL#1.0 - TensTaper / REAL#100.0 * TensDiaFrac);
134
135     // trim may not take the set point under TensSPMin_kg (also as the taper lowers
    the profile)
136
137     IF TensTrim_kg < TensSPMin_kg - TensProfile THEN
138         TensTrim_kg := TensSPMin_kg - TensProfile;
139     END_IF;
140
141     TensTarget := TensProfile + TensTrim_kg;
142 ELSE
143     TensTarget := TensManual_kg;
144 END_IF;
145
146 IF TensJogLatch THEN
147     TensTarget := TensTarget * (REAL#1.0 + TensJogBoost / REAL#100.0);
148 END_IF;
149
150 // Decel boost: more unwinder brake while the line slows down, so the web keeps
151 // the rewind coupling loaded (no knock) and the unwinder does not overrun.
152 // TOneDecel (Motor_Manager rung 17): ramping down / soft stop / end-of-roll
    ramp.
153 // BrakeHold: T-one kept in AUTO after the stop until < 5 m/min.
154
155 IF TensDecelBoost < REAL#0.0 THEN
156     TensDecelBoost := REAL#0.0;
157 ELSIF TensDecelBoost > REAL#60.0 THEN
158     TensDecelBoost := REAL#60.0;
159 END_IF;
160
161 TensDecelActive := TOneDecel OR (BrakeHold AND NOT StartMotor_Cmd);
162
163 IF TensDecelActive THEN
164     TensTarget := TensTarget * (REAL#1.0 + TensDecelBoost / REAL#100.0);
165 END_IF;
166
```

```
167  // lowest set point: AUTO >= TensSPMin_kg (10 kg), MANUAL down to TensMin_kg
     (0 kg)
168
169  IF TensAuto THEN
170      TensSPLow := TensSPMin_kg;
171  ELSE
172      TensSPLow := TensMin_kg;
173  END_IF;
174
175  IF TensTarget < TensSPLow THEN
176      TensTarget := TensSPLow;
177  ELSIF TensTarget > TensMax_kg THEN
178      TensTarget := TensMax_kg;
179  END_IF;
180
181  // ---- 9. Rate limit (4 ms scan): smooth change of GSM / width / type / mode /
     trim / JOG boost
182
183  IF TensDecelRate_kg_s < REAL#5.0 THEN
184      TensDecelRate_kg_s := REAL#5.0;
185  ELSIF TensDecelRate_kg_s > REAL#100.0 THEN
186      TensDecelRate_kg_s := REAL#100.0;
187  END_IF;
188
189  IF TensDecelActive THEN
190      TensStep := TensDecelRate_kg_s * REAL#0.004;      // fast while decelerating
191  ELSE
192      TensStep := TensRate_kg_s * REAL#0.004;
193  END_IF;
194
195  IF TensSet_kg < TensTarget - TensStep THEN
196      TensSet_kg := TensSet_kg + TensStep;
197  ELSIF TensSet_kg > TensTarget + TensStep THEN
198      TensSet_kg := TensSet_kg - TensStep;
199  ELSE
200      TensSet_kg := TensTarget;
201  END_IF;
202
203  // ---- 10. Analog output Ch1
204  // 1..10 V (400..4000) = TensMin_kg..TensMax_kg ; 0 V = wire broken (below range)
205
206  TensAOReal := REAL#400.0 + REAL#3600.0 * (TensSet_kg - TensMin_kg) /
     (TensMax_kg - TensMin_kg);
207
208  IF TensAOReal < REAL#400.0 THEN
209      TensAOReal := REAL#400.0;
210  ELSIF TensAOReal > REAL#4000.0 THEN
211      TensAOReal := REAL#4000.0;
212  END_IF;
213
214  AO_TOne_SetPoint := REAL_TO_INT(TensAOReal);
```

1-9.Data

1-9-1.Data Types

Name	Type	Comment

1-9-2.Global Variables

Name	Data Type	Initial Value	AT	Retain	Constant	Network Publish	Comment
VAR_GLOBAL							
Pot1_Work_Velocity	INT		BuiltInIO://opt/#0/Ch1_Analog_Input_Value	False	False	Publish Only	Potentiometer 1 value - Work speed -
InvRef	INT		BuiltInIO://opt/#0/Ch2_Analog_Output_Value	False	False	Publish Only	Inverter speed reference
AO_TOne_SetPoint	INT		BuiltInIO://opt/#0/Ch1_Analog_Output_Value	False	False	Publish Only	Analog O to T-one Tension SetPoint [Kg]
VelMax	REAL	800		True	False	Publish Only	Maximum speed
VelSet	REAL			False	False	Publish Only	Set speed
RwSlowDecelK	REAL	0.15		True	False	Publish Only	Learned slow-down decel per mm of rewinder diameter (mm/s2 per mm)
RwMaxSlowFallback	REAL	1285.0		True	False	Publish Only	Rewinder max: slow-down diameter while K not learned (mm)
Language	DINT			True	False	Publish Only	Language
RwThickMin_mm	REAL	0.02		True	False	Publish Only	Learning: min plausible paper thickness (mm)
RwThickMax_mm	REAL	2.0		True	False	Publish Only	Learning: max plausible paper thickness (mm)
RwDecelMin_mm	REAL	20.0		True	False	Publish Only	Learning: min plausible stop decel (mm/s2)
RwDecelMax_mm	REAL	5000.0		True	False	Publish Only	Learning: max plausible stop decel (mm/s2)
RwLearnFailMax	INT	12		True	False	Publish Only	Learning: failed windows before alarm (12 = 2 min)
RwStopFailMax	INT	3		True	False	Publish Only	Learning: failed stops before alarm
RwThick_mm	REAL			True	False	Publish Only	Learned paper thickness (mm)
OverSpeed	BOOL			False	False	Publish Only	Speed too high
WorkSpd_Max	REAL	500.0		True	False	Publish Only	Work speed pot full scale (m/min), 50..VelMax
WorkSpd_Max_Set	REAL	400.0		True	False	Publish Only	HMI request: work pot full scale (m/min), applied only with motor stopped
PaperKraft	BOOL	False		True	False	Publish Only	Paper type: TRUE = kraft, FALSE = bleached (tension factor)
TensFactorBleached	REAL	0.54		True	False	Publish Only	Tension factor bleached paper (N/mm width per 100 g/m2)
TensFactorKraft	REAL	0.80		True	False	Publish Only	Tension factor kraft paper (N/mm width per 100 g/m2), tune on machine
TensAuto	BOOL	False		True	False	Publish Only	T-one set point from the PLC profile (FALSE = TensManual_kg)
TensAOEnable	BOOL	False		True	False	Publish Only	AO Ch1 = T-one set point (FALSE = fixed 10 V as old Fix10Vdc)
TensManual_kg	REAL	40.0		True	False	Publish Only	T-one manual set point (kg) when TensAuto is OFF

TensTaper	REAL	30.0		True	False	Publish Only	Tension taper core -> max diameter (%), 0..60
TensCoreBoost	REAL	0.0		True	False	Publish Only	Extra tension at the core (%), 0..50
TensRate_kg_s	REAL	5.0		True	False	Publish Only	T-one set point max change (kg/s), 1..20
TensMin_kg	REAL	10.0		True	False	Publish Only	T-one set point at 1 V (= T-one F.38)
TensMax_kg	REAL	100.0		True	False	Publish Only	T-one set point at 10 V (= T-one F.39)
TensSet_kg	REAL	0.0		False	False	Publish Only	Live T-one set point sent on AO Ch1 (kg)
TensAtCore	REAL	0.0		False	False	Publish Only	Profile preview: tension at the core (kg)
TensAtMid	REAL	0.0		False	False	Publish Only	Profile preview: tension at mid diameter (kg)
TensAtMax	REAL	0.0		False	False	Publish Only	Profile preview: tension at max diameter (kg)
UwRawMin	INT	10		True	False	Publish Only	Unwinder sonar raw min (below = sensor fault, alarm 17)
UwRawMax	INT	3990		True	False	Publish Only	Unwinder sonar raw max (above = sensor fault / no echo, alarm 17)
TensCheckEnable	BOOL	False		True	False	Publish Only	Tension check ON (HMI switch); OFF = start as before
TensWaitSpd	REAL	15.0		True	False	Publish Only	Speed held after START until tension established (m/min)
TensionOK	BOOL	False		False	False	Publish Only	Tension established (debounced T-one Alarm 0)
TensWait	BOOL	False		False	False	Publish Only	Waiting for tension (speed held at TensWaitSpd, HMI lamp)
SonarAsk	BOOL	False		False	False	Publish Only	V10 HMI popup: sonar is OFF, turn it on? (HMI NO resets it)
UwSlow_at_D_Req	BOOL	True		True	False	Publish Only	V7c: unwinder diameter slow-down follows the speed (FALSE = fixed UwSlowDown_Diam)
UwSlowDiamAct	REAL	0.0		False	False	Publish Only	V7c: unwinder slows down at this diameter (mm), HMI display
FinalStopDecel	REAL	500.0		True	False	Publish Only	V12: end-of-roll stop ramp, web decel (mm/s2), 0 = drive DEC ramp as before, 50..2000
FinalStopRamp	BOOL	False		False	False	Publish Only	V12: end-of-roll stop ramp running (HMI lamp)
RwJogMaxDiam	REAL	400.0		True	False	Publish Only	V7d: jog blocked above this rewinder diameter (mm), 0 = no limit
JogInhibit	BOOL	False		False	False	Publish Only	V7d: jog blocked (rewinder roll too big), HMI lamp
SCurveTime	REAL	0.5		True	False	Publish Only	V7f: S-curve rounding of the AUTO ramps (s), 0 = off, max 2
TOneDecel	BOOL	False		False	False	Publish Only	V7f: line decelerating (for a T-one stop/boost input)
HornStartBee	REAL	1.0		True	False	Publish Only	V9 horn: beep when the slo

p_s							w-down starts (s), 0 = off, max 5
HornBeepLen_s	REAL	0.2		True	False	Publish Only	V9 horn: length of the short beeps during slow-down (s), 0 = off, max 2
HornBeepPeriod	REAL	5.0		True	False	Publish Only	V9 horn: one short beep every ... s during slow-down, 0 = off, max 30
HornStopBeep_s	REAL	3.0		True	False	Publish Only	V9 horn: beep when stopped after a slow-down (s), 0 = off, max 10
HornTest	BOOL	False		False	False	Publish Only	Buzzer
RwLockDiam	REAL	300.0		True	False	Publish Only	V9: paper thickness LOCKED at this rewinder diameter (mm), 0 = never lock
RwLockMinGrow_mm	REAL	50.0		True	False	Publish Only	V9: min diameter growth since the start of learning before locking (mm)
RwMatLocked	BOOL	False		True	False	Publish Only	V9: paper thickness locked (HMI lamp), NEW PAPER / GSM YES unlocks
LongThick_mm	REAL	0.0		False	False	Publish Only	V9: thickness from the long baseline (mm), HMI display
RwDiamRev	REAL	0.0		False	False	Publish Only	V9: rewinder diameter averaged over one revolution (mm)
RwWave_mm	REAL	0.0		False	False	Publish Only	V9: diameter wave peak-peak in the last revolution (mm), HMI
UwStopLenShow	BOOL	False		False	False	Publish Only	V9 HMI: show unwinder STOP LENGTH (AUTO STOP UNWINDER active)
UwSlowDiamShow	BOOL	False		False	False	Publish Only	V9 HMI: show unwinder SLOW AT diameter (normal operation, sonar ON)
SoftCancelAccel	REAL	100.0		True	False	Publish Only	V9: then ramp up at this web accel (mm/s2), 0 = AccelWeb
RwMatReset	BOOL	False		False	False	Publish Only	HMI NEW PAPER: forget the learned paper thickness RwMat_K (PLC resets it)
FinalSpd_Max_Set	REAL	80.0		True	False	Publish Only	HMI request: final pot full scale (m/min), applied only with motor stopped
FinalSpd_Max	REAL	80.0		True	False	Publish Only	Final speed pot full scale (m/min), 5..WorkSpd_Max
UnderSpeed	BOOL			False	False	Publish Only	Speed too low
RwLearnReset	BOOL	False		False	False	Publish Only	HMI / Watch: clear learned decels RwStopDecelK, RwStopDecel, RwSlowDecelK (PLC resets it)
RwCoreDiameter	REAL	100.0		True	False	Publish Only	Rewinder core outside diameter for TEACH CORE (mm)
RwTeachCore	BOOL	False		False	False	Publish Only	HMI: TEACH CORE request (PLC resets it)
RwTeachState	INT	0		False	False	Publish Only	TEACH CORE: 0 idle 1 teaching 2 done 3 refused
SpdDelta_Per	REAL	12.0		True	False	Publish Only	Speed difference percentage

cent							
Encoder_Pos	DINT		IOBus://unit#3/Ch1 Encoder Present Position	False	False	Publish Only	Encoder counter
UwTeachState	INT	0		False	False	Publish Only	Unwinder TEACH CORE: 0 idle 1 teaching 2 done 3 refused
Encoder_On	BOOL		IOBus://unit#3/Ch1 Encoder Counter Operation Command/Ch1 Counter Enable	False	False	Publish Only	Encoder enable
Encoder_Reset	BOOL		IOBus://unit#3/Ch1 Encoder Counter Operation Command/Ch1 Internal Reset Execution	False	False	Publish Only	Encoder reset
HMI_Screen	ARRAY[0..1] OF DINT			False	False	Publish Only	HMI panel Screen
Velocity_M_min	REAL			True	False	Publish Only	Speed in m/min
Current_Metre	REAL			True	False	Publish Only	Current metres
Speed_Display	REAL			True	False	Publish Only	Set speed (displayed)
I_ControlVoltage	BOOL		BuiltInIO://cpu/#0/Input_Bit_00	False	False	Publish Only	In_0 ControlVoltageON
I_Start_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_01	False	False	Publish Only	In_1 Start button
I_Stop_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_02	False	False	Publish Only	In_2 Stop button
I_Motor_Thermal	BOOL		BuiltInIO://cpu/#0/Input_Bit_03	False	False	Publish Only	In_3 Motor thermal switch
I_Prot_ByPass	BOOL		BuiltInIO://cpu/#0/Input_Bit_04	False	False	Publish Only	In_4 Guards-bypassed selector switch - Under Panel
I_Hub_Position	BOOL		BuiltInIO://cpu/#0/Input_Bit_05	False	False	Publish Only	In_5 Hub/chuck in extraction position
I_FrontGuard_Closed	BOOL		BuiltInIO://cpu/#0/Input_Bit_06	False	False	Publish Only	In_6 Front guard closed
I_Jog_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_07	False	False	Publish Only	In_7 Jog Push button
I_VFD_Alarm	BOOL		BuiltInIO://cpu/#0/Input_Bit_08	False	False	Publish Only	In_8 Inverter in alarm
I_Side_Door_	BOOL		BuiltInIO://c	False	False	Publish Only	In_Side Door is Closed

Closed			pu/#0/Input_Bit_13				
I_OR_Position_Sens	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN01 (Pin 1.1) _6000_01	False	False	Publish Only	OverRider position sens
I_OR_Hoover_Mode	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN02 (Pin 2.1) _6000_02	False	False	Publish Only	OverRider Hoover Mode
I_OR_Auto_OS	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN03 (Pin 1.2) _6000_03	False	False	Publish Only	OverRider in automatic
I_OR_Cont_OS	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN04 (Pin 2.2) _6000_04	False	False	Publish Only	OverRider in continuous
O_MotorStart	BOOL		BuiltInIO://cpu/#0/Output_Bit_00	False	False	Publish Only	Q_0 Motor start
I_FanTemp_Ok	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN06 (Pin 2.3) _6000_06	False	False	Publish Only	Motor (fan) thermal OK
O_Alarm_Array	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT 01 (Pin 1.1) _7000_01	False	False	Publish Only	Alarm lamp
O_Prot_ByPass_Lamp	BOOL			False	False	Publish Only	Guards bypassed lamp
O_Motor_Lamp	BOOL		BuiltInIO://cpu/#0/Output_Bit_01	False	False	Publish Only	Q_1 Motor ON lamp
O_RwUnload	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT 03 (Pin 1.2) _7000_03	False	False	Publish Only	Unloaders enable - opening
O_FrontGuard_ByPass	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-ECO Output_	False	False	Publish Only	Front guard bypass

			OUT04 (Pin 2.2) _7000_04				
O_Brake	BOOL		BuiltInIO://cpu/#0/Output_Bit_02	False	False	Publish Only	Q_2 Brake command
O_OR_Auto_Lamp	BOOL		BuiltInIO://cpu/#0/Output_Bit_06	False	False	Publish Only	Q_6 OverRider auto/continuous lamp
O_OverRider_Up	BOOL		BuiltInIO://cpu/#0/Output_Bit_09	False	False	Publish Only	Q_9 Raise OverRider ~10mm Over Round Rewinder roll
O_OverRider_Hold	BOOL		BuiltInIO://cpu/#0/Output_Bit_04	False	False	Publish Only	Q_4 OverRider Lock
O_CoolingFan	BOOL		BuiltInIO://cpu/#0/Output_Bit_03	False	False	Publish Only	Q_3 Brake and motor fans
Fan_Timeout	REAL	0.5		True	False	Publish Only	Fans stop timeout [min]
OverRider_TimeOFF	REAL	0.01		True	False	Publish Only	OverRider photocell OFF timeout (panel)
OverRider_TimeON	REAL	0.05		True	False	Publish Only	OverRider photocell ON timeout (panel)
AlarmOn	BOOL			False	False	Publish Only	Alarm active
BuzzerOn	BOOL			False	False	Publish Only	Panel buzzer command
Msg	INT			False	False	Publish Only	Message: Warnings / Alarms
Index_B	INT			False	False	Publish Only	Alarm index
MotStop	BOOL			False	False	Publish Only	Motor stopped
Clock1s	BOOL			False	False	Publish Only	1 s clock
ManMode	BOOL			True	False	Publish Only	Manual mode
Counter_Reset	BOOL			False	False	Publish Only	Counter reset from panel
I_FrontGuard_Open	BOOL		BuiltInIO://cpu/#0/Input_Bit_09	False	False	Publish Only	In_9 Front guard open
O_RwLoading_Enable	BOOL		BuiltInIO://cpu/#0/Output_Bit_07	False	False	Publish Only	Q_7 Unloaders Load enable - closing
JogSpd_SP	REAL	10.0		True	False	Publish Only	Speed in manual
O_AccDec2	BOOL		BuiltInIO://cpu/#0/Output_Bit_05	False	False	Publish Only	Q_5 Second acc / dec ramp
AccelTrim	REAL	0.08		True	False	Publish Only	Acceleration coefficient in run
DecelTrim	REAL	0.08		True	False	Publish Only	Deceleration coefficient in run
DecelCoarse	REAL	0.3		True	False	Publish Only	Acceleration coefficient at start (orig. comment; name suggests decel)
AccelCoarse	REAL	0.3		True	False	Publish Only	Acceleration coefficient at start (orig. comment; name suggests accel)
UwStopDiamShow	BOOL	False		False	False	Publish Only	V11 HMI: show unwinder STOP AT diameter (slow-down reached, decel / final speed)
O_AcousticSignal	BOOL		BuiltInIO://cpu/#0/Output_Bit_08	False	False	Publish Only	Q_8 Horn / Acoustic signal
Pot2_Slow_V	INT		BuiltInIO://o	False	False	Publish Only	Potentiometer 2 value - Fina

elocity			pt/#0/Ch2_Analog_Input_Value				l speed -
I_AutoStop_OS	BOOL		BuiltInIO://cpu/#0/Input_Bit_10	False	False	Publish Only	In_10 Stop on metre counter
I_Input_11_Spare_1255	BOOL		BuiltInIO://cpu/#0/Input_Bit_11	False	False	Do not publish	In_11 SPARE (wire 12.55) Under Panel
I_WebBreak_Monitor	BOOL		BuiltInIO://cpu/#0/Input_Bit_12	False	False	Publish Only	In_12 Paper break monitoring ON
I_Vacuum_OK	BOOL		IOBus://unit#3/Ch1 Reset/External Input Status/Ch1 External Input 0 Status	False	False	Publish Only	Extractor fan thermal OK
I_T_one_TensionOK	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN05 (Pin 1.3) _6000_05	False	False	Publish Only	T-one Alarm 0 (15-16): tension established (replaces I_SlowSens_PH)
I_WebBreak_PH	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN07 (Pin 1.4) _6000_07	False	False	Publish Only	Paper break photocell
UwStop_at_M_Val	REAL	5100		True	False	Publish Only	Unwinder Stop at metres
UwSlowMargin_m	REAL	10		True	False	Publish Only	Unwinder Slow-down metres
LowSpd_Display	REAL			True	False	Publish Only	Slow-down speed (displayed)
RwMaxDiam	REAL	1315.0		True	False	Publish Only	Rewinder max diameter protection (mm)
RwMaxStop	BOOL			False	False	Publish Only	Rewinder max diameter STOP (latched)
Encoder_Pulse	INT	1000		True	False	Publish Only	Encoder pulses/Rotation
RwSlowDeceILrn	REAL	0.0		True	False	Publish Only	Learned real slow-down decel (mm/s2) - 0 = not learned
CountReset_Req	BOOL			False	False	Publish Only	Counter reset requested
Psw	INT			False	False	Publish Only	Password Entry
I_RsetAlarm	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN08 (Pin 2.4) _6000_08	False	False	Publish Only	Alarm reset
EncoderWheel_Circumf	REAL	402		True	False	Publish Only	Encoder wheel (Cylinder / Roller) Circumference [mm]

AlarmCodeSV	ARRAY[0..32] OF BOOL			False	False	Publish Only	Panel alarms
SafeGuard_ByPass	BOOL			False	False	Publish Only	Guards bypassed
FrontGuard_Closed	BOOL			False	False	Publish Only	Front guard closed
SoftStop	BOOL	False		False	False	Publish Only	Soft stop in progress (HMI)
SoftStopEnable	BOOL	True		True	False	Publish Only	STOP button = soft stop (False = immediate as before)
SoftStopEndSpd	REAL	8.0		True	False	Publish Only	Soft stop: run command drops below this speed (m/min)
UwDiam_Measure	REAL			True	False	Publish Only	Measured distance
UwCoreDiameter	REAL	100		True	False	Publish Only	Core Outside diameter [mm]
Core_Measure	BOOL	False		True	False	Publish Only	Stored core value
UwCoreDistance	REAL	785.56		True	False	Publish Only	Sonar distance to empty core. [mm]
AI_Diam_Unwinder	INT		IOBus://unit#2/Ch1 Analog Input Value	False	False	Publish Only	Analog value - UnWinder Diameter
Measure_ON	BOOL	True		True	False	Publish Only	Unwinder Diameter measurement enabled
UwDiam_Act	REAL			True	False	Publish Only	Measured roll diameter
UwSlow_at_D_Val	REAL	200		True	False	Publish Only	Unwinder Slow-down diameter [mm]
UwStopDiam_Limit	REAL	110		True	False	Publish Only	Stop diameter
Alarm_Reset	BOOL			False	False	Publish Only	Alarm reset from panel
Psw22	BOOL			False	False	Publish Only	Password 22 for Popup 22
Psw44	BOOL			False	False	Publish Only	Password 44 for Popup 44
AI_Diam_Rewinder	INT		IOBus://unit#2/Ch2 Analog Input Value	False	False	Publish Only	Analog value - ReWinder Diameter
RwStopEnable	BOOL	False		True	False	Publish Only	Rewinder diameter stop enabled
RwTargetDiam	REAL	300.0		True	False	Publish Only	Rewinder final diameter (mm)
RwUseSlow	BOOL	True		True	False	Publish Only	Slow down before target
RwSlowMargin_m	REAL	10.0		True	False	Publish Only	Rewinder Metres at slow speed before stop
RwSlowDecel	REAL	0.0		True	False	Publish Only	Rewinder Slow-down decel mm/s2 - 0 = auto from DecStart
RwResetHyst	REAL	50.0		True	False	Publish Only	Reset requests below target - this [mm]
RwDist0V	REAL	0.0		True	False	Publish Only	DT35 distance at 0 V (mm)
RwDist10V	REAL	782.0		True	False	Publish Only	DT35 distance at 10 V (mm)
RwRawMin	INT	10		True	False	Publish Only	Raw below = sensor fault
RwRawMax	INT	3900		True	False	Publish Only	Raw above = sensor fault
RwStopOnFault	BOOL	True		False	False	Publish Only	Stop on sensor fault
RwAxisDist	REAL	727.0		True	False	Publish Only	Sensor to shaft centre (mm) - teach
RwFilter_K	REAL	0.2		True	False	Publish Only	Distance filter per 100 ms (

							0..1)
RwMinPulses	DINT	1000		True	False	Publish Only	Min pulses in window to learn
RwLearn_K	REAL	0.3		True	False	Publish Only	Material factor filter weight
RwMat_K	REAL	0.0		True	False	Publish Only	Learned material factor
RwLearnStop	BOOL	True		True	False	Publish Only	Learn stop decel on every stop
RwMinLearnSpd	REAL	15.0		True	False	Publish Only	Min speed to learn a stop (m/min)
RwStopDecel	REAL	0.0		True	False	Publish Only	Learned stop decel (pulses/s2)
RwTreact	REAL	0.03		True	False	Publish Only	Stop reaction time (s)
RwTstopDef	REAL	3.0		True	False	Publish Only	Stop time until decel learned (s)
RwTrim	REAL	1.0		True	False	Publish Only	Stop length trim (>1 = earlier)
RwDist	REAL			False	False	Publish Only	Filtered distance (mm)
RwDiam	REAL			False	False	Publish Only	Rewinder diameter (mm)
RwSpeed	REAL			False	False	Publish Only	Line speed from encoder (m/min)
RwRemain_m	REAL			False	False	Publish Only	Metres to target diameter
RwStopLen_m	REAL			False	False	Publish Only	Stopping length now (m)
RwSlowLen_m	REAL			False	False	Publish Only	Rewinder Slow-down length [m]
RwStop_at_D_Req	BOOL			False	False	Publish Only	Rewinder diameter STOP request
RwSlow_at_D_Req	BOOL			False	False	Publish Only	Rewinder diameter SLOW request
RwSensFault	BOOL			False	False	Publish Only	Rewinder DT35 sensor fault
UwStopEnable	BOOL	False		True	False	Publish Only	Unwinder Auto Stop - Enable
SlowAtDisp	REAL			False	False	Publish Only	HMI: slow-down starts at (m or mm, see SlowAtMode)
SlowAtMode	INT			False	False	Publish Only	HMI: 0=none 1=unwinder [m] 2=rewinder [mm]
SlowAtShow	BOOL			False	False	Publish Only	HMI: show slow-down display
UwSlow_at_M_Val	REAL			False	False	Publish Only	Unwinder: slow-down starts at [m]
UwSlow_at_M_Req	BOOL			False	False	Publish Only	Unwinder metre slow-down request (latched)
RwSlow_at_D_Val	REAL			False	False	Publish Only	Rewinder slow-down starts at diameter [mm]
RwStop_at_D_Val	REAL			False	False	Publish Only	Rewinder: stop command at diameter (mm)
RwMaxSlow	BOOL			False	False	Publish Only	Rewinder max diameter SLOW (latched)
UwEmptyHyst	REAL	30.0		True	False	Publish Only	Unwinder Empty Hysteresis
GSM_Learned_OK	BOOL			False	False	Publish Only	Paper [Grams / Square Meter] FALSE until GSM learned
RwMaxSlowAtDiam	REAL	1370.0		False	False	Publish Only	Rewinder max: slow-down starts at diameter (mm)
RwStopDecelK	REAL	0.0		True	False	Publish Only	Learned stop decel per mm of rewinder diameter (mm/s2 per mm)
SoftStopDece	REAL	500.0		True	False	Publish Only	Soft stop decel (mm/s2), 0

l							= DecelCoarse
SpdMax_Pending	BOOL	False		False	False	Publish Only	New pot range waiting for the motor to stop
RwAxisDist_Prev	REAL	727.0		True	False	Publish Only	Rewinder: RwAxisDist before the last TEACH (for UNDO)
UwCoreDist_Prev	REAL	785.56		True	False	Publish Only	Unwinder: UwCoreDistance before the last TEACH (for UNDO)
RwTeachUndo	BOOL	False		False	False	Publish Only	HMI: UNDO rewinder TEACH (PLC resets it)
UwTeachUndo	BOOL	False		False	False	Publish Only	HMI: UNDO unwinder TEACH (PLC resets it)
Psw33	BOOL			False	False	Publish Only	Password 33 for Popup 33
RwMatChangePct	REAL	35.0		True	False	Publish Only	Paper change detection: window deviation (%), 2 windows in a row = new paper, 0 = off
RwSlowFallbackMm	REAL	30.0		True	False	Publish Only	Rewinder: slow-down this many mm before the target while the paper thickness is not learned (0 = off)
RwAutoMaxSpd	REAL	500.0		True	False	Publish Only	Rewinder AUTO stop ON: maximum work speed (m/min), 0 = no limit
RwWinLen_m	REAL	60.0		True	False	Publish Only	Paper thickness learning: MAX window length (m of paper)
RwWinGrow_mm	REAL	5		True	False	Publish Only	Paper thickness learning: window = this diameter growth (mm), 4 m .. RwWinLen_m
GsmAsk	BOOL	False		False	False	Publish Only	HMI popup: new parent roll, did you change the paper? (HMI NO resets it)
GsmAskYes	BOOL	False		False	False	Publish Only	HMI popup YES: forget the learned thickness (PLC resets it)
DecelWeb	REAL	250.0		True	False	Publish Only	V7: AUTO decel ramp, real web mm/s2 (0 = DecelCoarse)
AccelWeb	REAL	200.0		True	False	Publish Only	AUTO accel ramp, real web mm/s2 (0 = AccelCoarse)
RwRatioD	REAL	3366.0		True	False	Publish Only	V7: drive counts per m/min x rewinder diameter (mm), adapted while running
FFEnable	BOOL	True		True	False	Publish Only	V7: feed-forward speed loop ON (FALSE = old loop of rung 16)
SpdHold	BOOL	False		False	False	Publish Only	V9: speed held after START during a soft stop, START again = ramp up (HMI lamp)
SonarAskYes	BOOL	False		False	False	Publish Only	V10 HMI popup YES: sonar ON (PLC resets it)
UwStop_at_D_Val	REAL	0.0		False	False	Publish Only	V11: unwinder stop command diameter (mm) so the web stops at UwDiamStop, HMI "STOP AT"
Clock10ms	BOOL			False	False	Publish Only	10mS clock
Clock100ms	BOOL			False	False	Publish Only	100mS clock

Sec_to_Min	REAL			False	False	Do not publish	
Psw48	BOOL			False	False	Publish Only	Password 48 for Popup 48
JogRun	BOOL			False	False	Publish Only	V10: JOG really running (button, not a normal run, not a soft stop)
GuardStopOK	BOOL			False	False	Publish Only	V11: stopped for real -> guard bypass allowed (Output 13)
SlowDown	BOOL			False	False	Publish Only	Slow down
WarningOn	BOOL			False	False	Publish Only	HMI Warning Display
AlarmSilenced	BOOL			False	False	Publish Only	Alarms Silencer
PaperGSM	REAL			True	False	Publish Only	
WebWidth_mm	REAL			True	False	Publish Only	Web Width [mm]
TensFactor	REAL			True	False	Publish Only	Tension Helper
TensIdeal_Disp	REAL			True	False	Publish Only	Tension Helper
TensLostWarn	BOOL	False		False	False	Publish Only	Tension lost while running (warning)
UwSensTest	BOOL	False		False	False	Publish Only	Unwinder sonar fault (raw out of range 1 s)
UwSensFault	BOOL			False	False	Publish Only	UnWinder Sensor Fault
Thickness_Fault	BOOL			False	False	Publish Only	Thickness Not Learned !
AutoStopWhy	INT			False	False	Publish Only	Reason for AutoStop Done
AutoStopDone	BOOL			False	False	Publish Only	AutoStop Complete
AutoEnRefused	BOOL			False	False	Publish Only	AutoStop Not Possible
AutoEnWhy	INT			False	False	Publish Only	
TensWaitHz	REAL	1.0		True	False	Publish Only	Motor Slow rev Control
StartBlocked	BOOL	True		False	False	Publish Only	Motor Start Blocked
GuardOpen	BOOL	False		False	False	Publish Only	Front Guard Opened
SideDoorOpen	BOOL	False		False	False	Publish Only	Side Security Door NOT CLOSED !
TensDiaFrac	REAL	0.0		False	False	Publish Only	Rewinder diameter fraction 0 (core) .. 1 (max)
Psw40	BOOL			False	False	Publish Only	Password 40 for Popup 40
CntResetMode	INT	0		True	False	Publish Only	Metre counter auto reset: 0 manual, 1 with new parent roll, 2 after auto-stop (at next START)
LastRoll_m	REAL	0.0		True	False	Publish Only	Length of the last roll (m), saved at the auto reset
CntResetPend	BOOL	False		True	False	Publish Only	Metre counter reset waiting for the next START (HMI lamp)
TensJogBoost	REAL	30.0		True	False	Publish Only	Threading preset: % extra tension from JOG until START, 0..100
TensJogLatch	BOOL	False		False	False	Publish Only	Threading tension active (JOG -> START), HMI lamp
TensTrim_kg	REAL	0.0		True	False	Publish Only	Operator trim on the AUTO tension (kg), +/-10, reset when paper changes
PulseRate	UDINT		IOBus://unit#3/Ch1 Pulse Rate	False	False	Publish Only	Pulses per 100 ms (encoder unit, HMI diagnostic)

RwSlowLearnMin_mm	REAL			False	False	Publish Only	Slow-down learning: min decel accepted (mm/s2), below = ignored
TensDecelBoost	REAL	0.0		True	False	Publish Only	Decel boost: extra T-one tension while the line slows down (%), 0 = off, 0..60
TensDecelRate_kg_s	REAL	40.0		True	False	Publish Only	Decel boost: T-one set point change rate while decelerating (kg/s), 5..100
TensDecelActive	BOOL	False		False	False	Publish Only	Decel boost active (HMI lamp)
TensSPMin_kg	REAL	10.0		True	False	Publish Only	Lowest T-one set point allowed (kg)